

臺灣自編大學教科書

大模型導論

第 2 篇

語言建模、臺灣語料與 Token 工程

Language Modeling, Taiwan Corpora, and Token Engineering

- 第 5 章 機率語言模型、訓練目標與基本評量
- 第 6 章 Unicode、繁體中文與臺灣語言現象
- 第 7 章 分詞器、詞彙表與臺灣語言效率
- 第 8 章 語料取得、授權、個資與資料治理
- 第 9 章 大規模資料清理、去重、混合與合成資料

李世淵（機器人叫獸） 編著

助教 李天宇、李宇晴

2026 年 8 月 第一版

序言 資料才是模型的第一級元件

很多人談大模型時，談的是架構：幾層、幾個頭、幾億參數。但如果你去問任何一個真正做過模型的人，時間花在哪裡，答案幾乎都是同一個——資料。

第 1 篇的第 1.5.3 節已經給出過一組數字：一個大模型專案的時間分布中，資料取得、授權確認與清理大約占四成，而模型訓練與適配只占一成五。這個比例在學生的想像中通常是顛倒的，而那個顛倒正是專題失敗最常見的起點。

你手上這一冊，就是在把那四成講清楚。

為什麼這五章要放在動手做模型之前

第 3 篇你就會親手寫出 Transformer 並訓練 MiniFormosaGPT。既然如此，為什麼不先做模型，等需要資料時再回頭處理？

因為順序不能顛倒。你的 tokenizer 決定了訓練成本與上下文容量，而 tokenizer 的輸入是正規化後的文字；正規化的規格又取決於你要保護哪些欄位；而哪些資料可以用，是在更上游的授權確認階段就決定的。任何一環先做，後面都得重來。

更根本的理由是：模型會忠實地反映你餵給它的東西。第 9 章的語料配比表其實是一份價值宣告——你認為這個模型應該擅長什麼。一個沒有想清楚配比就開始訓練的專案，等於把這個決定交給了資料的自然分布，而網路資料的自然分布對繁體中文與本土語言極不友善。

這五章各自處理什麼

第 5 章建立量測的紀律。它回答兩個問題：怎麼量（訓練目標、遮罩、困惑度、BPC）與怎麼比（切分、baseline、消融、污染）。這一章最重要的一句話是：量錯了通常會被發現，比錯了往往不會。往後三十五章的每一個數字都建立在這一章的規格上。

第 6 章處理文字本身。編碼、亂碼、Unicode 正規化、繁簡與異體字、全形半形、注音與臺羅、中英日混寫。這是最瑣碎、也最會在半夜三點咬你一口的一章。核心觀念只有一個：正規化是有代價的，代價要寫下來。

第 7 章處理切分。你會親手實作 BPE，並第一次把「token 稅」換算成新臺幣——那是每個繁體中文使用者天天在繳、卻沒有人立法通過的一筆稅。這一章也會讓你看到本土語言使用者付出的代價最重。

第 8 章處理權利與責任。資料從哪裡來、能不能用、有沒有個資、誰負責、要保存多久、別人要求移除時你做得到嗎。這一章不給法律意見，但它會讓你在被問到「這些資料哪裡來的」時，能打開一份

說得清楚的清冊。

第 9 章把前面全部串起來，產出一份真正能拿去訓練的語料。抽取、過濾、去重、去污染、配比、合成資料，每一步都要有統計、有樣本、有理由。

本冊的成果

讀完這五章並完成實作，你會交出五份東西：基準評量規格、臺灣文本工程規格與 100 筆測試資料、一個為繁體中文設計的 tokenizer、一份通過檢查的語料清冊與 Data Card、以及一份完整的語料統計報表。

這五份成果構成第一學期第一次階段評量的核心。它們看起來都不像「做出一個模型」，但第 13 章訓練出來的 MiniFormosaGPT 能學到什麼，此刻已經被它們決定了大半。

有一件事值得先說：這一冊裡有相當多的內容不會出現在國際教材中——繁體中文的異體字問題、臺灣的公文用語、民國紀年、臺羅拼音的四種寫法、法規多重轉載造成的污染。這些不是「加值內容」，而是任何人想在臺灣做出可用的模型時，繞不過去的工程現實。

關於用語

本書一律使用臺灣的專業用語與教育部標準用字。以下是本冊特別容易混淆的幾組，往後全書一致遵守：

本書用語	對應英文	說明與區別
推論	inference	指模型執行、產生輸出的階段。與下一列刻意區分。
推理	reasoning	指模型的思考與邏輯推導能力，如「推理模型」。
分散式	distributed	指多機多卡的訓練或部署，如「分散式訓練」。
分布式	distributional	指統計分布相關概念，如「分布式假說」。
正規化	normalization	涵蓋 Unicode 正規化與層正規化。不使用「歸一化」。
標準化欄位	canonical field	指保留原文之外另加的規範格式欄位，與上一列意義不同。
最佳化	optimization	不使用「優化」。最佳化器即 optimizer。
詞元 / 分詞器	token / tokenizer	不使用「標記」「令牌」「切詞器」。
資料	data	不使用「數據」。資料集即 dataset。

本書用語	對應英文	說明與區別
個資	personal data	個人資料的簡稱，兩者交替使用，意義相同。
去識別	de-identification	不使用「脫敏」。匿名化為其中一種手段。
去重 / 去污染	dedup / decontamination	兩者技術相似但目的不同，見第 9.4.1 節。
雜湊	hash	不使用「哈希」或「散列」。
覆核	review / double-check	指由非執行者進行的二次檢查。
閾值	threshold	不使用「門檻值」以避免與 gating 混淆。
管線	pipeline	不使用「流水線」。
品質 / 介面 / 快取	quality / interface / cache	不使用「質量 / 接口 / 緩存」。

英文專有名詞在首次出現時並列原文，其後視情況直接使用英文（例如 tokenizer、BPE、MinHash、Data Card）——這反映臺灣工程現場的真實習慣。

一則必須先說的提醒

第 8 章會大量談到著作權、個人資料與契約。這些內容僅供工程教學使用，目的是讓你知道「哪裡有風險、該問誰」，而不是讓你自行做出法律判斷。任何實際的資料使用決策，都必須經過所屬機構的法務、個資窗口或倫理審查程序。

此外，法規與實務見解會隨時間變動，尤其人工智慧訓練資料的相關規範在國內外都仍在發展中。書中所有涉及法律的段落都標示了需要確認的事項，請把它們當成待辦清單，而不是結論。

李世淵（機器人叫獸）

2026 年 8 月 於雲林

目 錄

序言 資料才是模型的第一級元件

目錄

全書架構總覽

第 5 章 機率語言模型、訓練目標與基本評量

Probabilistic Language Modeling, Training Objectives, and Basic Evaluation

學習目標

5.1 三種訓練目標

5.2 遮罩：因果遮罩與損失遮罩

5.3 困惑度：意義、計算與陷阱

5.4 資料切分：訓練、驗證、測試

5.5 baseline、消融實驗與比較的紀律

5.6 資料污染與 benchmark 洩漏

程式實作

系統案例：五領域 domain perplexity 面板

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

附：基準評量規格範本

本章小結

成果檢核

第 6 章 Unicode、繁體中文與臺灣語言現象

Unicode, Traditional Chinese, and Taiwan Language Phenomena

學習目標

6.1 四個層次：位元組、碼位、字元與字素

6.2 UTF-8、亂碼與編碼還原

6.3 Unicode 正規化：NFC、NFD、NFKC、NFKD

6.4 繁簡、異體字與臺灣正體

6.5 全形半形、標點與結構化欄位

6.6 注音、臺羅、客語與族語文字

6.7 中英混寫、程式碼與製造領域文本

6.8 正規化規格的設計

程式實作

系統案例：混合語料的清理與保護

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

附：正規化決策速查表

本章小結

成果檢核

第 7 章 分詞器、詞彙表與臺灣語言效率

Tokenizers, Vocabularies, and Taiwan Language Efficiency

學習目標

7.1 為什麼需要 tokenizer

7.2 BPE：從零實作

7.3 Pre-tokenization：切之前的切

7.4 特殊詞元與對話模板

7.5 量測：fertility、壓縮率與覆蓋率

7.6 從零訓練、詞彙擴充與嵌入初始化

7.7 token 稅：把 fertility 換算成錢

7.8 臺灣專屬的 tokenizer 公平性測試

程式實作

系統案例：為 MiniFormosaGPT 設計詞彙表

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

第 8 章 語料取得、授權、個資與資料治理

Corpus Acquisition, Licensing, Privacy, and Data Governance

學習目標

8.1 從用途反推資料需求

8.2 五種資料來源

8.3 智慧財產權的風險辨識

8.4 個人資料：定義、原則與實作

8.5 Provenance ledger：資料的身分證

8.6 Data Card、責任人與審批流程

8.7 退出機制與資料生命週期

8.8 資料隔離：科學問題也是治理問題

8.9 校園場域的特殊風險

程式實作

系統案例：臺灣教育語料庫的治理設計

章末評量
研究延伸與版本注意事項
常見問題
教師使用建議
附：資料治理成熟度自評
本章小結
成果檢核

第 9 章 大規模資料清理、去重、混合與合成資料

Cleaning, Deduplication, Mixtures, and Synthetic Data at Scale

學習目標
9.1 內容抽取：從原始檔案到純文字
9.2 品質過濾
9.3 去重：精確與近似
9.4 去污染與時間洩漏
9.5 語料配比：決定模型會變成什麼樣子
9.6 合成資料
9.7 規模、品質與多樣性的三角
程式實作
系統案例：MiniFormosaGPT 預訓練語料混合方案
章末評量
研究延伸與版本注意事項
常見問題
教師使用建議
本章小結
成果檢核

索引

以下為 Word 自動目錄欄位。在 Word 中開啟後，於下方按右鍵選擇「更新功能變數」即可產生含頁碼的三層目錄；亦可使用左側導覽窗格依書籤瀏覽。

序言 資料才是模型的第一級元件.....	2
為什麼這五章要放在動手做模型之前.....	2
這五章各自處理什麼	2
本冊的成果.....	3
關於用語	3
一則必須先說的提醒	4
目 錄	5

全書架構總覽	16
本冊五章的能力遞進	16
第 2 篇	18
機率語言模型、訓練目標與基本評量	19
學習目標	19
5.1 三種訓練目標	19
5.1.1 自回歸：預測下一個詞元	19
5.1.2 遮罩式：把挖空的詞填回來	20
5.1.3 去雜訊式序列到序列	20
5.1.4 從形狀看訓練訊號	21
5.1.5 損失的三種平均方式	22
5.2 遮罩：因果遮罩與損失遮罩	22
5.2.1 因果遮罩：不准看未來	22
5.2.2 損失遮罩：哪些位置該計分	23
5.2.3 把兩種遮罩放在一起看	24
5.3 困惑度：意義、計算與陷阱	25
5.3.1 從損失到困惑度	25
5.3.2 困惑度的直覺與參照點	25
5.3.3 三個必須成立的可比較前提	26
5.3.4 tokenizer 差異造成的假象：一個具體計算	26
5.3.5 BPC 的完整實作與三數字慣例	27
5.3.6 長度正規化與排序翻轉	28
5.4 資料切分：訓練、驗證、測試	28
5.4.1 三個集合各自的角色	28
5.4.2 隨機切分 vs 時間切分	29
5.4.3 切分比例與資料量	29
5.4.4 分域驗證集：本書的臺灣實踐	30
5.4.5 切分的實作	30
5.4.6 怎麼解讀分域困惑度的差異	31
5.4.7 切分的五條紀律	32
5.5 baseline、消融實驗與比較的紀律	32
5.5.1 沒有 baseline 的數字沒有意義	32
5.5.2 消融實驗：一次只改一件事	33
5.5.3 什麼叫「固定訓練預算」	34
5.5.4 實驗紀錄的最小要求	34
5.5.5 困惑度不是終點：評測的四個層次	35
5.5.6 一個完整的比較陷阱案例	35
5.6 資料污染與 benchmark 洩漏	36
5.6.1 污染是什麼，為什麼特別嚴重	36

5.6.2 一個讓學生印象深刻的示範	37
5.6.3 四種污染型態.....	37
5.6.4 用 n-gram 重疊偵測污染.....	38
5.6.5 防治污染的四層設計	39
5.6.6 臺灣情境的四個污染特例.....	39
程式實作	41
程式 5A 對數概似、困惑度與長度正規化.....	41
程式 5B 遮罩實作與計分範圍驗證	42
程式 5C 資料污染偵測器.....	43
三個實作如何串成一條線.....	44
系統案例：五領域 domain perplexity 面板	44
建置步驟	44
你應該會觀察到的現象（先預測、再驗證）	45
報告要求	45
章末評量	46
一、概念題.....	46
二、計算題.....	46
三、除錯題.....	46
四、系統設計題	47
五、臺灣情境與倫理題	47
評量提示（給自學者）	47
研究延伸與版本注意事項	47
常見問題	48
教師使用建議	48
附：基準評量規格範本.....	49
本章小結	50
成果檢核	51
Unicode、繁體中文與臺灣語言現象.....	52
學習目標	52
6.1 四個層次：位元組、碼位、字元與字素.....	52
6.1.1 四個層次的定義	53
6.1.2 為什麼這件事在大模型中特別重要	53
6.1.3 臺灣文本會用到的 Unicode 區塊.....	54
6.2 UTF-8、亂碼與編碼還原.....	55
6.2.1 UTF-8 的基本規則	55
6.2.2 亂碼的三種成因與判讀	55
6.2.3 截斷的正確做法	56
6.2.4 PDF 與 OCR 帶來的特殊問題	57
6.3 Unicode 正規化：NFC、NFD、NFKC、NFKD	58

6.3.1	四種形式的分工	58
6.3.2	NFKC 會做什麼：一份實測清單	58
6.3.3	不可逆性：正規化的核心風險	59
6.4	繁簡、異體字與臺灣正體	60
6.4.1	繁簡轉換不是一對一	60
6.4.2	臺灣正體字與其他繁體區的差異	60
6.4.3	罕用字、造字區與古籍文獻	61
6.4.4	本書的處理原則	61
6.5	全形半形、標點與結構化欄位	62
6.5.1	全形半形的取捨	62
6.5.2	結構化欄位的正規化	62
6.6	注音、臺羅、客語與族語文字	64
6.6.1	注音符號	64
6.6.2	臺灣台語羅馬字	64
6.6.3	客語與原住民族語	65
6.7	中英混寫、程式碼與製造領域文本	66
6.7.1	臺灣技術文本的混寫特徵	66
6.7.2	空白的語意	66
6.7.3	句子邊界：中文特有的難題	67
6.7.4	語言判定：漢字共用碼位的困境	68
6.8	正規化規格的设计	69
6.8.1	五個設計原則	69
6.8.2	管線的建議順序	69
6.8.3	處理紀錄的格式	70
6.8.4	量化正規化的效益與風險	70
	程式實作	72
	程式 6A 臺灣繁體中文 normalizer	72
	100 筆測試資料該怎麼設計	73
	程式 6B 亂碼與異常偵測器	74
	程式 6C 臺羅與注音的往返轉換測試	75
	系統案例：混合語料的清理與保護	76
	資料組成與挑戰	76
	交付要求	76
	評分重點	77
	章末評量	78
	一、概念題	78
	二、計算與判讀題	78
	三、除錯題	78
	四、系統設計題	78

五、臺灣情境與倫理題	79
評量提示（給自學者）	79
研究延伸與版本注意事項	79
常見問題	80
教師使用建議	80
附：正規化決策速查表	81
本章小結	82
成果檢核	83
分詞器、詞彙表與臺灣語言效率	84
學習目標	84
7.1 為什麼需要 tokenizer	84
7.1.1 模型看不見文字	84
7.1.2 五種切分粒度	85
7.1.3 中文的特殊處境	86
7.2 BPE：從零實作	86
7.2.1 演算法	86
7.2.2 用一個小例子走一遍	86
7.2.3 手動追蹤一次合併	87
7.2.4 完整實作	88
7.2.4 三種主流演算法的差異	89
7.3 Pre-tokenization：切之前的切	90
7.3.1 為什麼需要它	90
7.3.2 數字的處理	90
7.3.3 空白的表示	91
7.4 特殊詞元與對話模板	91
7.4.1 基本的特殊詞元	91
7.4.2 對話模板	92
7.4.3 工具呼叫與結構化輸出的標記	92
7.5 量測：fertility、壓縮率與覆蓋率	93
7.5.1 三個核心指標	93
7.5.2 分域 fertility：本章的核心量測	94
7.5.3 fertility 之外：切得好不好	94
7.5.4 看進詞彙表裡面	95
7.6 從零訓練、詞彙擴充與嵌入初始化	96
7.6.1 三條路線的選擇	96
7.6.2 嵌入初始化：三種策略	96
7.6.3 該加哪些詞	97
7.6.4 訓練 tokenizer 的語料配比	98
7.7 token 稅：把 fertility 換算成錢	99

7.7.1	三條成本傳導路徑.....	99
7.7.2	一個完整的試算.....	99
7.7.3	上下文容量的損失.....	99
7.7.4	訓練成本的傳導.....	100
7.8	臺灣專屬的 tokenizer 公平性測試.....	100
7.8.1	為什麼需要公平性測試.....	100
7.8.2	六類測試詞彙.....	100
7.8.3	判讀與行動.....	102
7.8.4	tokenizer 選型決策表.....	102
	程式實作.....	104
	程式 7A 迷你 BPE : trainer、encoder 與 decoder.....	104
	程式 7B 三種詞彙表大小的比較實驗.....	105
	程式 7C token 稅計算器.....	106
	系統案例：為 MiniFormosaGPT 設計詞彙表.....	107
	設計需求.....	107
	必須交付的六項.....	108
	評分重點.....	108
	章末評量.....	109
	一、概念題.....	109
	二、計算題.....	109
	三、除錯題.....	109
	四、系統設計題.....	109
	五、臺灣情境與倫理題.....	110
	評量提示（給自學者）.....	110
	研究延伸與版本注意事項.....	110
	常見問題.....	111
	教師使用建議.....	111
	本章小結.....	112
	成果檢核.....	113
	語料取得、授權、個資與資料治理.....	114
	學習目標.....	114
8.1	從用途反推資料需求.....	114
8.1.1	先問要做什麼，再問要什麼資料.....	115
8.1.2	一份資料需求規格.....	115
8.1.3	資料量要多少.....	116
8.2	五種資料來源.....	116
8.2.1	來源類型與風險輪廓.....	116
8.2.2	政府開放資料的實務注意事項.....	117
8.2.3	網路爬取：最需要謹慎的一類.....	117

8.2.4	合成資料的三個限制	118
8.2.5	取得資料的標準作業流程.....	118
8.3	智慧財產權的風險辨識	119
8.3.1	四種可能同時存在的權利.....	119
8.3.2	合理使用：一個必須謹慎對待的概念.....	120
8.3.3	風險分級與處置	120
8.4	個人資料：定義、原則與實作	121
8.4.1	什麼算個人資料	121
8.4.2	三項應該內化的原則	122
8.4.3	去識別的層次與極限	122
8.4.4	繁體中文個資偵測的難處.....	123
8.4.5	模型記憶：個資風險的特殊形態.....	123
8.5	Provenance ledger：資料的身分證	124
8.5.1	為什麼需要它.....	124
8.5.2	必要欄位	124
8.5.3	從 ledger 到可回答的問題	126
8.6	Data Card、責任人與審批流程	126
8.6.1	Data Card 與 provenance 的分工	126
8.6.2	Data Card 的必備段落	127
8.6.3	角色與審批.....	127
8.6.4	存取控制與稽核軌跡	127
8.7	退出機制與資料生命週期.....	128
8.7.1	為什麼退出很難	128
8.7.2	退出流程的設計	128
8.7.3	保存期限與到期處置	129
8.8	資料隔離：科學問題也是治理問題.....	130
8.8.1	五種資料的隔離	130
8.8.2	隔離的技術實作	130
8.9	校園場域的特殊風險.....	131
	程式實作.....	132
	程式 8A 語料清冊檢查器	132
	程式 8B 繁體中文個資掃描與遮罩	133
	程式 8C 資料退出處理器.....	135
	系統案例：臺灣教育語料庫的治理設計	136
	三種來源的治理差異	136
	必須交付的六項	137
	評分重點	137
	章末評量	138
	一、概念題	138

二、判斷與分析題.....	138
三、除錯題.....	138
四、系統設計題.....	138
五、臺灣情境與倫理題.....	139
評量提示（給自學者）.....	139
研究延伸與版本注意事項.....	139
常見問題.....	140
教師使用建議.....	140
附：資料治理成熟度自評.....	141
本章小結.....	142
成果檢核.....	143
大規模資料清理、去重、混合與合成資料.....	144
學習目標.....	144
9.1 內容抽取：從原始檔案到純文字.....	144
9.1.1 六種來源，六種毛病.....	144
9.1.2 網頁樣板的偵測與移除.....	145
9.1.3 抽取的品質標記.....	146
9.2 品質過濾.....	146
9.2.1 過濾的兩種思路.....	146
9.2.2 規則式過濾器的設計.....	147
9.2.3 每一階段都要看樣本.....	148
9.2.4 臺灣語料的特殊考量.....	149
9.3 去重：精確與近似.....	149
9.3.1 為什麼去重如此重要.....	149
9.3.2 精確去重.....	150
9.3.3 近似去重：MinHash 與 LSH.....	151
9.3.4 完整實作.....	152
9.3.5 去重的取捨.....	153
9.4 去污染與時間洩漏.....	153
9.4.1 去重與去污染的差別.....	153
9.4.2 把去污染接進管線.....	154
9.4.3 時間洩漏.....	155
9.5 語料配比：決定模型會變成什麼樣子.....	155
9.5.1 配比就是優先順序的宣告.....	155
9.5.2 取樣溫度：控制配比的數學工具.....	155
9.5.3 本書建議的臺灣語料配比.....	156
9.5.4 課程安排：資料的先後順序.....	157
9.6 合成資料.....	157
9.6.1 什麼時候值得用.....	157

9.6.2	三段式流程：生成、篩選、標註.....	158
9.6.3	人工覆核的比例	158
9.6.4	模型崩壞的風險	159
9.7	規模、品質與多樣性的三角.....	160
9.7.1	三者無法同時最大化	160
9.7.2	可觀測的指標.....	160
9.7.3	一份語料統計報表該長什麼樣.....	161
	程式實作	162
	程式 9A 可平行化的清理管線.....	162
	程式 9B MinHash 近似去重	163
	程式 9C 合成資料工廠.....	164
	系統案例：MiniFormosaGPT 預訓練語料混合方案.....	165
	完整流程	166
	必須交付的六項	166
	評分重點	166
	章末評量	168
	一、概念題.....	168
	二、計算題.....	168
	三、除錯題.....	168
	四、系統設計題	169
	五、臺灣情境與倫理題	169
	評量提示（給自學者）	169
	研究延伸與版本注意事項	169
	常見問題	170
	教師使用建議	170
	本章小結	171
	成果檢核	172
索 引.....		173
	英文詞條	173
	中文詞條	173

全書架構總覽

全書分為十篇四十章，另有十四組附錄。本冊為第 2 篇，是第一學期的資料工程主線，也是第一次階段評量（第 9 週）的範圍。

篇次	章次	主題	核心產出
第 1 篇	1–4	大模型視野、數學與工程基礎	需求規格書、技術演進圖、數學速查表、可重現環境
第 2 篇（本冊）	5–9	語言建模、臺灣語料與 Token 工程	基準評量規格、臺灣文本工程規格、自訓 tokenizer、語料清冊
第 3 篇	10–13	Transformer 從零實作	MiniFormosaGPT 與可生成繁中的模型
第 4 篇	14–17	預訓練、解碼與實驗科學	穩定訓練迴圈、尺度化推估與算力預算
第 5 篇	18–21	算力、系統與現代架構	效能剖析、分散式決策、臺灣算力盤點
第 6 篇	22–26	本土化後訓練與對齊	繁中適配模型、LoRA、偏好對齊、邊緣小模型
第 7 篇	27–30	推理、知識更新與提示工程	推理實驗、知識更新流程、RAG 原型
第 8 篇	31–34	應用系統：進階 RAG、Agent 與產品開發	知識系統、受限 Agent、可上線的應用
第 9 篇	35–37	部署、評測、安全與治理	量化部署、臺灣題庫、紅隊與影響評估
第 10 篇	38–40	多模態、具身智慧與總整專題	VLM 應用、臺語語音、VLA、總整專題

本冊五章的能力遞進

章	你會理解什麼	你會做出什麼	往後被誰使用
第 5 章	怎麼量、怎麼比才不會自欺	基準評量規格、五個分域驗證集	第 7、9、13、16、22、36 章

章	你會理解什麼	你會做出什麼	往後被誰使用
第 6 章	編碼、正規化與臺灣語言現象	臺灣文本工程規格、100 筆測試	第 7、8、9、30 章
第 7 章	token 稅從哪裡來、怎麼算	自訓 tokenizer、公平性報告	第 9、13、17、22、33 章
第 8 章	資料的權利、個資與可追溯性	語料清冊、Data Card、退出流程	第 9、22、36、37、40 章
第 9 章	清理、去重、配比如何決定模型	語料統計報表、可重現管線	第 13 章之後全部

第 2 篇

語言建模、臺灣語料與 Token 工程

Language Modeling, Taiwan Corpora, and Token Engineering

第 5–9 章

本篇承諾：把資料當成模型的第一級元件。從繁體中文文字規格、詞元效率、合法來源、去識別到可追溯治理，一次做完。

第 5 章

機率語言模型、訓練目標與基本評量

Probabilistic Language Modeling, Training Objectives, and Basic Evaluation

難度：核心 | 建議授課：第 5 週 | 硬體需求：A 級（一般筆電即可完成全部實作，不需 GPU）

叫獸開講

兩個模型的困惑度分別是 12.3 和 12.8，你能說第一個比較好嗎？如果它們用的是不同的 tokenizer，這兩個數字根本不能放在一起比。我看過太多實驗報告栽在這裡——不是模型做錯了，是量錯了。這一章要教的，是怎麼把「我覺得變好了」變成「我可以證明它變好了」。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 從機率鏈式法則寫出自回歸、遮罩式與序列到序列三種訓練目標，並說明各自適合什麼模型。
2. 正確實作 causal mask 與 loss mask，說明 padding、prompt 與 answer 三種詞元的計分範圍差異。
3. 計算句子級與詞元級的對數機率與困惑度，並解釋長度正規化為何會造成排序翻轉。
4. 說出困惑度可比較的三個前提，並判斷一組實驗數據是否構成有效比較。
5. 設計訓練、驗證、測試集的切分方式，包括隨機切分與時間切分的適用情境。
6. 建立 baseline 並設計消融實驗，區分「模型真的變好」與「量測方式改變」。
7. 偵測資料污染與 benchmark 洩漏，用 n-gram 重疊法量化污染程度。
8. 為臺灣的公文、教科書、社群、製造文件四種文本建立分域驗證集，並找出模型最陌生的領域。

這一章在全書的位置

第 2 章告訴你語言模型的目標函數是什麼，第 3 章給了你數學工具，第 4 章建好了環境。這一章要把三者接起來，並加上最關鍵的一塊：怎麼知道你做的事情有沒有用。往後三十五章的每一個實驗，都會回來引用本章建立的評量規格。

5.1 三種訓練目標

第 2 章介紹了三類架構，這一節把它們的訓練目標寫成可以實作的形式。目標函數決定了模型學到什麼，也決定了它擅長什麼——這是架構差異的根源。

5.1.1 自回歸：預測下一個詞元

$$L_{AR} = - (1/T) \sum_{t=1}^T \log P_{\theta}(w_t / w_{<t})$$

這是本書主線使用的目標。每個位置都在預測它的下一個位置，因此一條長度 T 的序列會產生 T 個訓練訊號——這是它訓練效率高的原因。

實作上有一個關鍵細節叫 **teacher forcing**：訓練時，模型預測第 t 個位置時看到的前文是「真實的」前文，而不是模型自己前面生成的內容。這讓訓練可以完全平行化（所有位置同時算），但也造成了訓練與推論的落差——推論時模型看到的是自己生成的內容，一旦某步出錯，錯誤會累積下去。這個落差稱為 **exposure bias**，第 14 章談解碼時會再遇到。

曝光偏差是一個必須知道、但不必過度焦慮的問題。在現代大模型的規模下，它主要透過三個方向被緩解：更好的解碼策略（第 14 章的取樣與重複懲罰）、指令微調與偏好對齊（第 23、25 章讓模型學會在自己的輸出上表現良好）、以及推理模型的自我檢查機制（第 27 章）。

階段	模型看到的前文	後果	緩解手段
訓練	資料集中的真實前文	穩定、可平行、收斂快	—
推論	模型自己生成的前文	錯誤會累積，長文本容易崩壞	解碼策略（第 14 章）
落差表現	生成到某處開始重複或語意漂移	常被誤認為模型能力不足	偏好對齊（第 25 章）

你會在第 13 章第一次親眼看到這個落差：訓練損失明明降得不錯，但一按下生成鍵，模型跑個幾十個字就開始重複同一句話。那不是模型壞了，那是曝光偏差加上解碼策略未調整的典型症狀。

5.1.2 遮罩式：把挖空的詞填回來

$$L_{MLM} = - (1/|M|) \sum_{t \in M} \log P_{\theta}(w_t / w_{\setminus M})$$

M 是被遮住的位置集合，通常占全部位置的 15%。模型可以看到左右兩邊的完整語境來填空，因此得到的表示比自回歸更「雙向」，適合分類與檢索。

但代價很明顯：一條序列只有 15% 的位置產生訓練訊號，其餘 85% 的計算沒有直接貢獻梯度。同樣的資料量與算力，自回歸能榨出約六倍的學習訊號。這是第 2 章說「Decoder-only 訓練效率高」的量化說明。

5.1.3 去雜訊式序列到序列

T5 一類的模型把輸入破壞掉（刪詞、替換片段、打亂順序），要求模型還原原文。它的形式介於前兩者之間：編碼器雙向讀取被破壞的輸入，解碼器自回歸產生還原結果。

目標	訓練訊號密度	語境方向	擅長	代表
自回歸	每個位置都算 (100%)	僅左側	生成、對話、少樣本學習	GPT、Llama
遮罩式	僅遮住的位置 (約 15%)	雙向	分類、抽取、句子表示	BERT
去雜訊 seq2seq	解碼端全部	編碼端雙向	翻譯、摘要、明確的轉換任務	T5、BART

本書第 11 到 13 章實作的是自回歸模型，但第 30 章做 RAG 時你會用到 Encoder-only 的嵌入模型——那正是遮罩式訓練的產物。理解目標函數的差異，你才知道為什麼一個模型適合生成、另一個適合檢索。

5.1.4 從形狀看訓練訊號

用第 3 章的形狀語言把自回歸訓練寫一次，會讓上面那些抽象的說法變得具體。

步驟	張量形狀	說明	注意
輸入詞元	(B, T)	一批序列的詞元索引	整數型別
模型輸出	(B, T, V)	每個位置一個詞彙表分布	記憶體大戶 (第 3.1.4 節)
位移對齊	$\text{logits}[:, :-1]$ 對 $\text{labels}[:, 1:]$	第 t 個位置預測第 $t+1$ 個詞元	忘記位移是最常見的錯誤
攤平計損	$(B \times (T-1), V)$ 與 $(B \times (T-1),)$	cross_entropy 需要二階與一階	第 3 章 shape tracing 第 9 題
有效訊號	約 $B \times (T-1)$ 個	扣掉 padding 與不計分的位置	下一節的 loss mask

請注意第三列。這個位移是整個自回歸訓練的核心機制，也是第 13 章實作時最常出錯的地方。忘記位移的後果是：模型被要求「預測當前這個詞」，而它已經看到當前這個詞了，所以 loss 會迅速掉到

接近 0——看起來訓練得非常成功，實際上模型什麼都沒學到。

5.1.5 損失的三種平均方式

這是一個看起來瑣碎、實際上會讓你的實驗數字對不上的細節。同一批資料，用不同的平均方式會得到不同的損失值。

方式	定義	特性	什麼時候用
詞元平均	總損失 ÷ 有效詞元數	長句與短句的每個詞元權重相同	預訓練的標準做法
序列平均	先算每句的平均，再對句子平均	短句的每個詞元權重較高	某些微調情境
總和	不做平均	受批次大小影響，難以比較	幾乎不用

本書統一使用詞元平均，因為它是預訓練的標準，也是困惑度定義所依據的方式。但你必須知道另外兩種存在——當你在第 23 章做指令微調、或是在第 25 章比較不同模型的序列機率時，這個選擇會直接影響結果。

梯度累積的隱形陷阱

在做梯度累積（第 3.5.8 節）時，如果每個小批次的有效詞元數不同，直接把各批次的平均損失再平均是錯的——那會讓詞元少的批次獲得不成比例的權重。正確做法是累加「損失總和」與「詞元總數」，最後才相除。這個錯誤很隱蔽，因為訓練仍會收斂，只是收斂到略為不同的地方。同樣的道理在 5.3.1 節的評測程式中也適用。

5.2 遮罩：因果遮罩與損失遮罩

這一節是本章最容易出錯、也最少被講清楚的部分。兩種遮罩名字相似但功能完全不同，混淆它們會造成很難察覺的錯誤。

5.2.1 因果遮罩：不准看未來

自回歸模型在訓練時，所有位置是同時計算的。如果不加限制，第 3 個位置在預測第 4 個詞時就能「看到」第 4 個詞本身——那等於直接抄答案，訓練出來的模型在推論時完全不能用。

因果遮罩的做法是：在注意力分數矩陣上，把所有「未來位置」的分數設成負無限大，經過 softmax 之後就變成 0。

因果遮罩的標準實作

```
import torch
```

```
def causal_mask(T: int, device=None) -> torch.Tensor:
    """回傳 (1, 1, T, T)，可廣播到 (B, H, T, T)"""
    # 上三角為 True (要被遮住的位置)，對角線不遮
    m = torch.triu(torch.ones(T, T, dtype=torch.bool, device=device),
                    diagonal=1)
    return m[None, None, :, :]

def apply_causal_mask(scores: torch.Tensor) -> torch.Tensor:
    """scores: (B, H, T, T)"""
    B, H, T, _ = scores.shape
    m = causal_mask(T, scores.device)
    # 用 dtype 的最小值而非 -inf，避免全遮的列產生 nan
    return scores.masked_fill(m, torch.finfo(scores.dtype).min)
```

請注意最後一行的細節：用 dtype 的最小值而不是負無限大。如果某一列的所有位置都被遮住（在某些 padding 情境下會發生），softmax 對一整列的 $-\infty$ 會產生 nan。用有限的最小值可以避免這個問題，代價是那一列會變成均勻分布——不正確但至少不會毀掉整個訓練。

一個具體的例子。T = 4 時，遮罩後的注意力可見範圍如下表（打勾表示可見）：

查詢位置 \ 鍵位置	位置 1	位置 2	位置 3	位置 4
位置 1	可見	遮住	遮住	遮住
位置 2	可見	可見	遮住	遮住
位置 3	可見	可見	可見	遮住
位置 4	可見	可見	可見	可見

5.2.2 損失遮罩：哪些位置該計分

因果遮罩控制「模型能看到什麼」，損失遮罩控制「哪些位置的預測要被評分」。兩者完全獨立，需要分開處理。

三種常見的不計分情況：

情況	為什麼不計分	若誤計分的後果	出現章節
padding 詞元	只是為了對齊長度的填充物	模型學會預測 PAD，損失被稀釋	第 13 章

情況	為什麼不計分	若誤計分的後果	出現章節
prompt 部分	指令微調時只評估回答的品質	模型學著背誦問題而非回答	第 23 章
系統提示	固定內容・重複出現會被過度學習	模型過度擬合系統提示的措辭	第 23 章

損失遮罩的正確實作

```
import torch.nn.functional as F

def masked_cross_entropy(logits, targets, loss_mask):
    """
    logits: (B, T, V)
    targets: (B, T)
    loss_mask: (B, T) 1 表示要計分，0 表示忽略
    """
    B, T, V = logits.shape
    losses = F.cross_entropy(
        logits.reshape(B * T, V),
        targets.reshape(B * T),
        reduction="none",          # 先不要平均
    ).reshape(B, T)

    m = loss_mask.to(losses.dtype)
    n_valid = m.sum().clamp(min=1.0) # 避免全部被遮時除以零
    return (losses * m).sum() / n_valid

# PyTorch 內建的簡便寫法：把要忽略的目標設成 ignore_index
IGNORE = -100
targets_masked = targets.masked_fill(loss_mask == 0, IGNORE)
loss = F.cross_entropy(logits.reshape(-1, V),
                       targets_masked.reshape(-1),
                       ignore_index=IGNORE)
```

一個極容易犯的錯誤

用 `losses.mean()` 而不是 `(losses * m).sum() / m.sum()`。前者把被遮住的位置（值為 0）也算進分母，導致損失被系統性低估。如果你的批次中有一半是 padding，你的損失會比真實值低一半——訓練曲線看起來很漂亮，但模型其實沒那麼好。這種錯誤不會報錯，只會讓你對模型有錯誤的印象。

5.2.3 把兩種遮罩放在一起看

用一個指令微調的例子把整件事串起來。假設一筆訓練資料是：

一筆對話資料的三種遮罩

序列： [BOS] 請 問 臺 南 在 哪 [SEP] 在 臺 灣 南 部 [EOS] [PAD] [PAD]
 位置： 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

attention_mask (能不能被看到)：

1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0

loss_mask (要不要計分):

0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0
 └── prompt 不計分 ───┘ └── answer 計分 ───┘ └── PAD ─┘

causal_mask: 額外套用，讓每個位置只能看到自己與左邊

三種遮罩同時作用：因果遮罩讓每個位置只看左邊，attention mask 讓 PAD 不被任何位置看到，loss mask 讓只有回答部分產生梯度。第 23 章你會實作完整版本，現在請先理解它們各自解決什麼問題。

5.3 困惑度：意義、計算與陷阱

5.3.1 從損失到困惑度

第 3 章已經給了定義，這裡補上實作與判讀。

$$PPL = \exp(L) = \exp(- (1/N) \sum \log P(w_t / w_{\{<t\}}))$$

N 是計分的詞元數——注意是「計分的」而不是「全部的」，這與上一節的 loss mask 直接相關。

正確的困惑度計算

```
@torch.inference_mode()
def perplexity(model, loader, device):
    """回傳整個資料集的困惑度（詞元加權，非批次平均）"""
    model.eval()
    total_nll, total_tokens = 0.0, 0

    for x, y, mask in loader:
        x, y, mask = x.to(device), y.to(device), mask.to(device)
        logits = model(x)
        nll = F.cross_entropy(
            logits.reshape(-1, logits.size(-1)),
            y.reshape(-1),
            reduction="none",
        ).reshape(y.shape)

        total_nll += (nll * mask).sum().item()
        total_tokens += mask.sum().item()

    return math.exp(total_nll / total_tokens)
```

請注意這裡累加的是「總損失」與「總詞元數」，最後才相除。一個常見的錯誤是先算每個批次的平均損失、最後平均那些平均值——當各批次的有效詞元數不同時（因為 padding 量不同），這會給出錯誤的答案。這叫做「平均的平均不等於平均」，在評測程式中是常見的隱形錯誤。

5.3.2 困惑度的直覺與參照點

困惑度	對應損失	意義	典型情境
等於 V	$\ln V$	完全沒學到，均勻猜測	隨機初始化的模型
約 1000	約 6.9	學到了基本的字頻分布	訓練初期
約 100	約 4.6	學到了局部的詞彙搭配	n-gram 等級
約 20–40	約 3.0–3.7	學到了語法與部分語意	小型模型的合理範圍
約 8–15	約 2.1–2.7	接近人類文本的可預測程度	中大型模型
接近 1	接近 0	幾乎完全預測正確	警訊：多半是記憶化或資料洩漏

最後一列值得特別注意。第 3 章講熵時說過，語言本身有不可消除的不確定性——「今天天氣很 _」後面可以接好幾個合理的字。所以一個在真正未見過的資料上困惑度趨近於 1 的模型，幾乎可以確定是測試資料洩漏到訓練集裡了。看到好得離譜的數字時，第一反應應該是懷疑而不是慶祝。

5.3.3 三個必須成立的可比較前提

這是本章的核心警告。兩個困惑度數字要能比較，必須同時滿足三個條件：

1. 同一個 tokenizer。切得越碎，每個詞元越好猜，困惑度越低——但這不代表模型更好。這是最常被忽略的一項。
2. 同一份測試資料，且同樣的前處理。換了正規化規則、換了截斷方式，數字就不能比。
3. 同樣的計分範圍。有沒有算 PAD、有沒有算 prompt、有沒有算第一個詞元，都會改變分母。

5.3.4 tokenizer 差異造成的假象：一個具體計算

假設同一句臺灣繁體中文，兩個 tokenizer 分別切成 30 個與 45 個詞元，而兩個模型對這句話賦予的「整句對數機率」完全相同，都是 -90 。

項目	模型 A	模型 B	說明
詞元數	30	45	B 的 tokenizer 切得比較碎
整句 $\log P$	-90	-90	兩者對這句話的理解完全相同
每詞元平均	-3.00	-2.00	除以不同的分母
困惑度	約 20.1	約 7.4	B 看起來好非常多

結論：B 的困惑度只有 A 的三分之一，看起來遙遙領先，但兩個模型對這句話的理解完全一樣。這不是理論上的極端例子——繁體中文的 tokenizer 差異就有這個量級，第 7 章會讓你實際量到。

那該怎麼辦？三個處置方式，由簡到繁：

- 最簡單：確保比較的模型使用同一個 tokenizer。這在你自己做消融實驗時完全做得到。
- 次佳：改用「每個字元的位元數」（bits per character）作為指標，把 tokenizer 的影響除掉。
- 最務實：不要只看困惑度。用下游任務的表現來比較——第 36 章會建立完整的評測體系。

$$BPC = (\text{總 } \log_2 \text{ 機率的負值}) / \text{字元數} \quad (\text{與 tokenizer 無關})$$

5.3.5 BPC 的完整實作與三數字慣例

既然 BPC 可以跨 tokenizer 比較，那就把它做進評測程式裡。分子是整段文字的總資訊量（與怎麼切無關），分母是原始字元數（與怎麼切也無關），所以這個指標具有可比性。

同時計算 PPL、BPC 與 tokens_per_char

```
import math

@torch.inference_mode()
def evaluate_lm(model, tokenizer, texts, device):
    total_nll = 0.0          # 自然對數下的負對數概似總和
    total_tokens = 0
    total_chars = 0

    model.eval()
    for text in texts:
        ids = tokenizer.encode(text)
        total_chars += len(text)          # 原始字元數，與切法無關
        x = torch.tensor(ids, device=device)[None, :]
        logits = model(x[:, :-1])
        nll = F.cross_entropy(
            logits.reshape(-1, logits.size(-1)),
            x[:, 1:].reshape(-1),
            reduction="sum",              # 用 sum 才能正確累加
        )
        total_nll += nll.item()
        total_tokens += x.size(1) - 1

    return {
        "ppl": math.exp(total_nll / total_tokens),
        "bpc": (total_nll / math.log(2)) / total_chars,
        "tokens_per_char": total_tokens / total_chars,
    }
```

最後回傳的 tokens_per_char 就是第 7 章要深入處理的 token fertility 的倒數。在本章你只需要知道：把它一起記錄下來，你就能事後解釋為什麼兩個模型的 PPL 不同。

本書的評測慣例

從本章開始，所有語言模型的評測都必須同時報告三個數字：詞元級 PPL、字元級 BPC、以及 `tokens_per_char`。只報 PPL 的實驗結果在本課程中視為不完整。這個要求看起來嚴格，但它會在第 7 章（比較三種詞彙表大小）與第 22 章（做繁中適配）時證明它的價值——沒有這三個數字，你根本無法區分「模型變好了」與「切法變了」。

5.3.6 長度正規化與排序翻轉

另一個常見陷阱出現在「比較兩個候選句子哪個比較好」的場景，例如 beam search 或多候選重排序。

整句的對數機率會隨長度單調下降——因為每多一個詞元就多加一個負數。所以直接用 $\log P$ 排序，永遠偏好短句。

候選句	詞元數	整句 $\log P$	每詞元平均
「好。」	3	-4.5	-1.50
「這個方案在成本上比較有利。」	15	-18.0	-1.20

用整句 $\log P$ 排序，第一句勝出（ $-4.5 > -18.0$ ）；用每詞元平均排序，第二句勝出（ $-1.20 > -1.50$ ）。同一組候選，兩種計分方式給出完全相反的答案——這就是排序翻轉。

實務上兩種都不完美：整句機率偏好短句，長度平均偏好長句（因為長句中往往有很多容易預測的虛詞把平均拉高）。折衷做法是加入一個可調的長度懲罰項，第 14 章會展開。現在你只需要記得：看到任何涉及「比較不同長度輸出」的指標，一定要問它怎麼處理長度。

5.4 資料切分：訓練、驗證、測試

5.4.1 三個集合各自的角色

集合	用途	可以看幾次	被污染的後果
訓練集	更新模型參數	無限次	無（本來就是給它學的）
驗證集	選超參數、決定何時停止、比較方案	很多次，但每次都在消耗它	過度擬合驗證集，選出的方案在真實情境下不佳

集合	用途	可以看幾次	被污染的後果
測試集	最終報告的數字	理想上只有一次	報告的數字失去意義，結論不可信

第三欄是關鍵。驗證集每被使用一次，你就對它多知道一點；用了五十次之後，你選出來的超參數其實是「最適合這個驗證集」而不是「最適合這個任務」。這叫做過度擬合驗證集，它是隱形的，因為你的驗證分數看起來一直在進步。

本書的要求是：測試集在整個開發過程中最多用兩次——期中一次確認方向、最終一次寫進報告。所有的日常比較都用驗證集。這條紀律在第 16、36、40 章會反覆強調。

5.4.2 隨機切分 vs 時間切分

不是所有資料都適合隨機切分。選錯切法會讓你的評測結果嚴重高估。

切法	做法	適用	不適用的情況
隨機切分	把所有樣本打亂後按比例分配	樣本彼此獨立，且未來與過去分布相同	有時間性、有群組結構的資料
時間切分	用某個時間點之前的當訓練，之後的當測試	新聞、公告、法規、社群、產線資料	資料量太少、時間跨度不足
群組切分	同一來源、同一作者的樣本不跨集合	同一份文件被切成多段時	無明顯群組結構時

臺灣情境中最典型的例子是校務公告與法規文件：同一份公告可能被切成十段落，如果隨機切分，其中幾段在訓練集、幾段在測試集——模型在測試時等於看過非常相似的內容，分數會虛高。正確做法是群組切分：整份文件要嘛全在訓練集，要嘛全在測試集。

另一個例子是新聞與社群資料。如果你的模型會被用來處理未來的內容，那就必須用時間切分——因為真實情境中，模型面對的永遠是它訓練時看不到的未來。用隨機切分會讓你的評測樂觀許多，而那個樂觀在上線後會消失。

5.4.3 切分比例與資料量

常見的比例是 80/10/10 或 90/5/5，但比例不是重點，重點是驗證集與測試集要「夠大到能區分差異」。

一個粗略的判斷方法：如果你想偵測 1% 的差異，測試集至少要有數千個樣本，否則差異會被隨機波動淹沒。第 16 章會用信賴區間把這件事講精確。現在只要記得一個直覺：測試集只有 50 題時，答對 40 題與答對 42 題之間的差異幾乎沒有統計意義。

5.4.4 分域驗證集：本書的臺灣實踐

一個總體的困惑度數字會把模型在不同領域的巨大差異平均掉。所以本書要求你不只切出一份驗證集，而是切出五份分域驗證集。這五個領域構成往後所有評測的基本面板。

領域	文本來源	語言特徵	為什麼要單獨看
公文與法規	政府公告、法規條文、機關函釋	正式、固定句式、大量專有名詞與條號	模型最容易在此犯制度性錯誤
教科書與教材	課文、講義、習題、學習單	結構化、循序、術語密集	教育應用的核心場景
社群與口語	論壇、留言、對話、訊息	口語、縮寫、表情符號、中英混寫	模型的臺灣語感最容易在此露餡
製造技術文件	SOP、規格書、工單、維修紀錄	料號、單位、中英混寫、表格	臺灣產業的差異化資料所在
本土語言	臺語、客語文本與教材	漢字、羅馬字、聲調符號	低資源，需要單獨觀察不被平均掉

你會在第 7 章看到不同 tokenizer 在這五個領域的效率差異，在第 9 章看到語料配比如何影響它們，在第 22 章看到繁中適配讓哪些領域進步、哪些領域退步。一個只報告單一數字的實驗，會完全看不見這些訊息。

5.4.5 切分的實作

把前面的原則寫成程式。重點不在演算法有多難，而在於「切分本身也是一個要被版本控制與記錄的產出物」。

formosa/splits.py — 支援群組與時間切分

```
import hashlib, json
from pathlib import Path

def stable_bucket(key: str, n_buckets: int = 100) -> int:
    """用雜湊決定分組，同一個 key 永遠落在同一桶—
    這比 random.shuffle 更好：新增資料不會打亂舊的分配"""
```

```

h = hashlib.blake2b(key.encode("utf-8"), digest_size=8).digest()
return int.from_bytes(h, "big") % n_buckets

def group_split(records, group_key="doc_id",
                 ratios=(80, 10, 10), seed_salt="v1"):
    """同一份文件的所有段落必定落在同一個集合"""
    assert sum(ratios) == 100
    tr_hi, va_hi = ratios[0], ratios[0] + ratios[1]
    out = {"train": [], "valid": [], "test": []}
    for r in records:
        b = stable_bucket(seed_salt + str(r[group_key]))
        key = "train" if b < tr_hi else ("valid" if b < va_hi else "test")
        out[key].append(r)
    return out

def time_split(records, cutoff: str, date_key="published_at"):
    """cutoff 之前為訓練，之後為測試—模擬真實部署"""
    train = [r for r in records if r[date_key] < cutoff]
    test = [r for r in records if r[date_key] >= cutoff]
    return {"train": train, "test": test}

def write_manifest(splits, out: Path, meta: dict):
    """切分本身也要被記錄：規模、方式、種子、時間"""
    manifest = {
        "method": meta["method"],
        "seed_salt": meta.get("seed_salt"),
        "cutoff": meta.get("cutoff"),
        "created_at": meta["created_at"],
        "sizes": {k: len(v) for k, v in splits.items()},
        "chars": {k: sum(len(r["text"]) for r in v)
                  for k, v in splits.items()},
    }
    out.write_text(json.dumps(manifest, ensure_ascii=False, indent=2),
                   encoding="utf-8")

```

注意 `stable_bucket` 的設計理由。用 `random.shuffle` 打亂再切的問題是：當你之後新增了一批資料重新切分，原本在訓練集的樣本可能跑到測試集去——那會讓你先前所有的實驗結果失效。用雜湊決定分桶則沒有這個問題：每筆資料的歸屬只取決於它自己的 ID，新增資料不會影響既有分配。這是一個小細節，但在專案跑了三個月、資料改版了五次之後，它的價值會非常明顯。

5.4.6 怎麼解讀分域困惑度的差異

拿到五個領域的數字之後，真正的工作才開始：把數字翻譯成行動。困惑度高不一定代表要補資料，得先判斷它高的原因。

觀察到的現象	可能的原因	如何進一步確認	對應處置
某領域 PPL 特別高	語料中該領域比例過低	檢查語料統計與領域分布	補資料 (第 9 章)
PPL 高但 BPC 正常	tokenizer 對該領域不友善	看該領域的 <code>tokens_per_char</code>	調整詞彙表 (第 7 章)
PPL 與 BPC 都高	模型確實不熟悉這個領域	人工抽樣看生成品質	持續預訓練 (第 22 章)
某領域異常低	可能有污染或高度重複	跑污染檢查與去重統計	重建驗證集 (本章 5.6 節)

第二列是繁體中文使用者最常遇到、也最容易誤判的情況。本土語言 (臺語漢字、羅馬字) 的困惑度往往極高，但其中相當一部分不是「模型不懂臺語」，而是「tokenizer 把臺語切得極碎」。這兩者的處置完全不同——前者要補語料與微調，後者要改詞彙表。分不清楚就會做白工。

所以本書堅持三數字並列的理由在這裡變得具體：PPL 告訴你「以詞元為單位模型有多不確定」，BPC 告訴你「以文字為單位模型有多不確定」，`tokens_per_char` 告訴你「這兩個單位之間的換算率」。三個一起看，你才能診斷；只看一個，你只能猜。

5.4.7 切分的五條紀律

1. 切分必須在任何實驗開始之前完成並固定。中途改變切分等於作廢先前所有結果。
2. 切分的隨機種子必須記錄在設定檔中，並寫進環境指紋 (第 4.7.3 節) 。
3. 測試集在專案結束前最多看兩次，理想上一次。
4. 額外保留一份「私有保留集」，連你自己都不看，直到最終驗收——這是防止自欺的最後一道防線，第 36 章會展開。
5. 每一份切分都要記錄來源、時間範圍、文件數與詞元數，寫進 Data Card (第 8 章) 。

5.5 baseline、消融實驗與比較的紀律

前面四節在講「怎麼量」，這一節開始講「怎麼比」。這兩件事同樣重要，而後者更常被做錯——因為量錯了通常會被發現，比錯了往往不會。

5.5.1 沒有 baseline 的數字沒有意義

「我的模型困惑度是 18.3」這句話不包含任何資訊。18.3 是好是壞？跟什麼比？如果一個 unigram

模型在同一份資料上是 19.0，那你的 Transformer 幾乎沒學到東西。

本書要求每個實驗至少有三層 baseline：

層級	內容	回答什麼問題	本書位置
退化 baseline	unigram 或 bigram 模型	我的模型有沒有超越單純的統計？	第 2 章的程式 2A
同類 baseline	相同規模、標準設定的模型	我做的改動有沒有效？	第 12、16 章
上界參考	更大的模型或商業 API	我離天花板還有多遠？	第 1、36 章

退化 baseline 特別重要，因為它會抓出「模型根本沒在學」的情況。我看過學生的 Transformer 訓練得很順利、loss 一路下降，最後困惑度比 bigram 還差——原因是某個地方的形狀寫錯，模型其實只看得到最後一個詞元。有 baseline，這個問題在第一天就會被發現。

5.5.2 消融實驗：一次只改一件事

消融 (ablation) 就是把某個元件拿掉或換掉，看結果變多少，藉此判斷它的貢獻。原則只有一條，但很難遵守：一次只改一個變因。

做法	問題	正確做法	嚴重度
同時改架構與學習率	不知道是哪個造成差異	分兩次實驗，各改一項	高
改了設定也換了資料版本	結論完全無效	固定資料版本，記錄雜湊值	極高
比較時訓練步數不同	「比較久當然比較好」	固定訓練預算 (步數或詞元數)	高
只跑一個 seed	差異可能只是運氣	至少三個 seed，報告中位數與範圍	高

第三列的「固定訓練預算」值得展開。比較兩個架構時，什麼叫公平？固定步數？固定詞元數？固定 GPU 小時？固定 FLOPs？這四種都合理，但會得出不同的結論——一個計算量較小的架構在「固定 FLOPs」下會顯得更好，在「固定步數」下則不一定。

本書的建議是：明確寫出你固定的是什麼，並且如果可能，報告兩種以上。誠實地說出「在固定步數

下 A 較好，在固定 FLOPs 下 B 較好」，遠比含糊地說「A 比較好」有價值。第 16 章會把這件事變成一套可執行的實驗規範。

5.5.3 什麼叫「固定訓練預算」

上一節提到公平比較要固定訓練預算，但預算可以有四種定義，而它們會給出不同的結論。這一節把選擇的後果講清楚，因為你在第 16、17、20 章都會遇到這個抉擇。

固定什麼	意思	偏袒誰	適合回答的問題
訓練步數	兩者都跑 20000 步	偏袒每步計算量大的模型	「同樣的迭代次數下誰學得快」
總詞元數	兩者都看過 100 億詞元	偏袒參數多的模型	「資料受限時誰比較有效率」
總 FLOPs	兩者的總計算量相同	偏袒架構效率高的模型	「算力受限時該選哪個架構」
牆鐘時間	兩者都訓練 24 小時	偏袒實作優化好的模型	「實務上一天內能做到什麼」

對臺灣的多數情境而言，第三種（固定 FLOPs）與第四種（固定牆鐘時間）通常最有意義，因為我們的限制是算力與時間，不是資料或耐心。但無論你選哪一種，關鍵是在報告中明確寫出來——一份沒說明固定了什麼的比較，讀者無法判斷它是否公平。

一個進階的做法是報告兩種以上。「在固定步數下 A 較好，但在固定 FLOPs 下 B 較好」是一個資訊量遠高於「A 比較好」的結論，因為它告訴讀者在什麼條件下該選哪一個。第 17 章講尺度定律時會把這件事推到極致。

5.5.4 實驗紀錄的最小要求

一次可被引用的實驗，紀錄中必須包含以下九項。這份清單直接對應第 4 章的環境指紋，你可以把它做成表單。

1. 模型設定：層數、維度、頭數、參數量。
2. tokenizer：哪一個、詞彙表大小、版本雜湊。
3. 資料：哪一版、切分方式、切分種子、各集合的詞元數。
4. 訓練預算：步數、有效批次、總詞元數、GPU 小時。

5. 最佳化設定：學習率、排程、weight decay、梯度裁切。
6. 隨機種子：跑了哪幾個，以及是否有失敗的種子被排除（若有必須說明）。
7. 評測方式：計分範圍、平均方式、是否含特殊詞元。
8. 結果：PPL、BPC、tokens_per_char，以及分域的分項數字。
9. 環境指紋：程式碼 commit、是否 dirty、函式庫版本、硬體。

一條殘酷但必要的規則

任何一項缺漏的實驗，在本課程中都不能被寫進最終報告的結論。這聽起來嚴格，但請想想：如果第六項（種子）沒記錄，你怎麼知道報告中那個漂亮的數字不是二十次裡挑出來的最好一次？科學的可信度不是靠誠實的意圖，是靠可查核的紀錄。

5.5.5 困惑度不是終點：評測的四個層次

本章花了很大篇幅講困惑度，但必須把它放在正確的位置上：它是最底層的指標，不是最終的判準。

層次	量什麼	優點	限制
第一層 困惑度	模型對文本的預測不確定度	便宜、連續、直接反映訓練目標	與下游表現的相關性不完美
第二層 自動指標	任務層級的自動評分	可大量執行、可重複	無法評估開放式生成的品質
第三層 模型評審	用另一個模型評分	可處理開放式輸出	有偏誤，需要校準
第四層 人工盲評	人類判斷	最接近真實價值	昂貴、慢、需要一致性管理

四個層次的成本由上而下遞增，可信度也由上而下遞增。實務上的做法是：日常開發用第一層快速迭代，階段性驗收用第二層，重要決策用第三層，最終結論用第四層。

一個常見的錯誤是只用第一層就下結論。困惑度下降不必然代表使用者體驗變好——一個把所有回答都寫得又長又囉嗦的模型，困惑度可能很漂亮，但沒有人想用它。第 36 章會把四個層次完整展開，並教你設計人工盲評的評分規準。

5.5.6 一個完整的比較陷阱案例

把前面的原則放進一個具體情境。假設某組同學要證明「加入臺灣公文語料能提升模型的公文處理能

力」，他們的實驗報告寫著：

項目	對照組	實驗組	報告的結論
訓練語料	通用語料 10 億詞元	通用 10 億 + 公文 2 億	「加入公文語料有效」
訓練步數	20000	24000	(未在報告中提及)
tokenizer	通用 32K	重新訓練的 32K	(未在報告中提及)
公文驗證集 PPL	28.4	19.7	「下降 31%，效果顯著」

這份報告的結論不成立，至少有四個理由：

1. 訓練步數不同。實驗組多訓練了 20%，光是這一項就可能造成困惑度下降，與加不加公文語料無關。
2. tokenizer 不同。重新訓練的 tokenizer 對公文中的專有名詞切得更好，PPL 自然較低——這是 5.3.4 節的假象，不是能力提升。
3. 沒有報告種子與變異。19.7 可能是三次裡最好的一次，而 28.4 可能是最差的一次。
4. 沒有做污染檢查。加入的公文語料可能與驗證集來自同一批文件。

正確的做法是：固定 tokenizer、固定訓練步數（或固定總詞元數並說明選了哪一種）、跑三個種子、報告 BPC 而非只有 PPL、並在加入語料前先對驗證集做污染檢查。修正之後，效果可能仍然存在，但幅度多半會小很多——而那個較小的數字才是真的。

一個對學生的提醒

這個案例不是虛構的教學例子，它是我在課堂上反覆看到的模式。學生並非有意造假，而是因為「加語料」與「多訓練一點」在直覺上都是「讓模型更好」，所以不覺得需要分開。但科學的價值正在於區分這兩者——如果你不知道效果來自哪裡，你就無法在下一次做出更好的決策。

5.6 資料污染與 benchmark 洩漏

5.6.1 污染是什麼，為什麼特別嚴重

資料污染指的是：測試資料（或與它高度相似的內容）出現在訓練資料裡。這會讓模型的測試表現虛高，因為它不是在推理，是在回憶。

在大模型時代，這個問題比以往嚴重得多，原因有三個：

- 訓練語料規模達到兆級詞元，幾乎不可能人工檢查裡面有什麼。
- 公開的評測題庫本身就在網路上，很容易被爬進訓練語料。
- 模型有很強的記憶能力，只要見過一次就可能記住——第 16 章會讓你量測這件事。

後果是：排行榜上的高分可能來自記憶而非能力。一個在公開題庫上表現優異的模型，換成一套從未公開的等難度題目，分數可能大幅下滑。這也正是第 36 章要求你自己出題、並保留一份私有題目的根本理由。

5.6.2 一個讓學生印象深刻的示範

如果你想在三分鐘內讓人理解污染有多嚴重，做這個實驗：

1. 準備一個小模型與一份 200 題的評測集，量測它的表現，記下數字。
2. 把評測集中的 40 題（20%）加進訓練資料，只訓練一個 epoch。
3. 重新量測。你會看到那 40 題的正確率大幅上升，而其餘 160 題幾乎不變。
4. 整體分數上升了，但模型的實際能力沒有任何改變。

這個示範的價值在於它把「污染」從一個抽象的警告變成一個可以看見的數字。而且它揭示了一件更重要的事：污染造成的分數提升是不均勻的——它只出現在被污染的樣本上。這給了我們一個實用的偵測訊號：如果某個模型在某個子集上的表現異常好於其他子集，那個子集可能被污染了。

第 16 章會把這個想法發展成正式的記憶化偵測方法，第 36 章則會用「公開題組 vs 私有題組的分數落差」作為污染的診斷指標。

5.6.3 四種污染型態

型態	描述	偵測難度	處置
完全重複	測試樣本逐字出現在訓練集	低（精確比對即可）	從訓練集移除
近似重複	改寫、翻譯、格式不同但實質相同	中（需 n-gram 或嵌入相似度）	設定閾值後移除
答案洩漏	題目沒出現但答案來源出現	高（需人工判斷）	換題或標註為受污染
時間洩漏	訓練資料包含測試期之後的資訊	中（需檢查時間戳）	改用時間切分

第三種最棘手。假設你的測試題是「某校 114 學年度的錄取分數是多少」，而訓練語料中沒有這道題，

但有那份公告的原文。這算不算污染？技術上模型是在「查資料」而不是「背答案」，但它確實見過答案。這種情況必須靠人工判斷，並在報告中明確揭露。

5.6.4 用 n-gram 重疊偵測污染

最實用的自動化方法是 n-gram 重疊：把測試樣本切成 n-gram，檢查有多少比例出現在訓練集中。

ch05/contamination.py (節錄)

```
from collections import defaultdict
import hashlib

def char_ngrams(text: str, n: int = 13):
    """中文用字元級 n-gram；13 是常用的閾值長度"""
    text = "".join(text.split()) # 移除空白，避免格式差異干擾
    return {text[i:i + n] for i in range(max(0, len(text) - n + 1))}

def build_index(train_texts, n=13):
    """把訓練集的 n-gram 存成雜湊集合，省記憶體"""
    index = set()
    for t in train_texts:
        for g in char_ngrams(t, n):
            index.add(hashlib.blake2b(g.encode("utf-8"),
                                      digest_size=8).digest())
    return index

def contamination_ratio(test_text, index, n=13):
    """回傳這筆測試樣本有多少比例的 n-gram 在訓練集出現過"""
    grams = char_ngrams(test_text, n)
    if not grams:
        return 0.0
    hit = sum(1 for g in grams
              if hashlib.blake2b(g.encode("utf-8"),
                                  digest_size=8).digest() in index)
    return hit / len(grams)

def audit(test_texts, index, threshold=0.5):
    """產出可人工覆核的污染報告"""
    flagged = []
    for i, t in enumerate(test_texts):
        r = contamination_ratio(t, index)
        if r >= threshold:
            flagged.append({"idx": i, "ratio": round(r, 3),
                           "preview": t[:60]})
    print(f"共 {len(test_texts)} 筆，標記 {len(flagged)} 筆 "
          f"({len(flagged)/len(test_texts):.1%}) ")
    return flagged
```

關於參數的兩點說明。n = 13 是一個經驗值：對中文而言，連續十三個字完全相同而彼此獨立產生的機率極低，所以命中通常代表真的重複。閾值 0.5 表示超過一半的 n-gram 都見過就標記——這個值

可以依你的容忍度調整，但務必在報告中寫明。

注意這個方法只能抓完全重複與部分的近似重複。改寫過的內容、翻譯過的內容、以及答案洩漏都抓不到。它是第一道防線，不是保證。

5.6.5 防治污染的四層設計

層次	做法	成本	本書章節
事前	評測題目自行撰寫，不從網路取材	高（要出題）	第 36 章
事中	訓練語料建立時就排除已知題庫	中（需維護排除清單）	第 9 章
事後	n-gram 重疊偵測與人工覆核	低	本章程式 5C
保險	保留一份完全不公開的私有題組	低（但需紀律）	第 36 章

第四層是最便宜也最有效的。你只要在建立評測集時多切一份出來、鎖在抽屜裡（比喻上的），到最終驗收時才拿出來用。如果公開題組與私有題組的分數落差很大，那就是污染的強烈訊號。

5.6.6 臺灣情境的四個污染特例

本地的資料生態有一些特殊性，會讓污染問題以不同的形式出現。這四種情況在國際教材中不會被提到，但你在做臺灣的專案時一定會遇到。

特例	情況	為什麼麻煩	處置
法規多重轉載	同一條法規被十幾個網站轉載	去重後仍有格式微異的版本殘留	正規化後再比對（第 6 章）
公文範本流通	公文格式範本在網路上大量流傳	模型可能背下範本而非理解格式	評測時改用真實公文而非範本
考古題與解答	升學考題與詳解在網路上完整可得	任何用歷屆試題做的評測都可能失效	自行命題，或用尚未公開的題目
繁簡轉換文本	同一內容有繁體與簡體兩版	簡單的字元比對抓不到跨字集的重複	先統一轉換再比對

第三列對教育領域的應用特別關鍵。如果你要做一個「協助學生準備學測」的系統，用歷屆試題來評測幾乎一定會高估——那些題目與詳解在網路上完整流通，模型很可能已經見過。這不是技術問題，是評測設計問題，而它的解法只有一個：自己出題，或取得尚未公開的題目並妥善保管。

第四列則是一個技術細節，但影響不小。程式 5C 的字元級 n-gram 比對，遇到繁簡不同的同一份文本會完全抓不到。正確做法是在建立索引前先做統一的正規化——這正是第 6 章要處理的主題，屆時你會發現正規化不只是為了清理資料，也是污染偵測的前提。

程式實作

本章三個實作都在 A 級硬體上完成。程式碼位於課程倉庫的 ch05/ 目錄，並會被第 13、16、36 章直接引用。

程式 5A 對數概似、困惑度與長度正規化

目標：計算句子級與詞元級的對數機率，並實際觀察長度正規化造成的排序翻轉。你可以用第 2 章訓練的 bigram 模型，也可以用任何一個開放權重的小模型。

ch05/lab5a_likelihood.py (節錄)

```
import math
import torch
import torch.nn.functional as F

@torch.inference_mode()
def score_sentence(model, tokenizer, text, device):
    """回傳一句話的多種評分方式"""
    ids = tokenizer.encode(text)
    x = torch.tensor(ids, device=device)[None, :]
    logits = model(x[:, :-1]) # (1, T-1, V)
    nll = F.cross_entropy(
        logits.reshape(-1, logits.size(-1)),
        x[:, 1:].reshape(-1),
        reduction="none",
    ) # (T-1,)

    total_nll = nll.sum().item()
    n_tok = nll.numel()
    n_char = len(text)
    return {
        "text": text,
        "n_tokens": n_tok,
        "n_chars": n_char,
        "logp_total": -total_nll,
        "logp_per_token": -total_nll / n_tok,
        "ppl": math.exp(total_nll / n_tok),
        "bpc": (total_nll / math.log(2)) / n_char,
        "tokens_per_char": n_tok / n_char,
    }

def compare(model, tokenizer, candidates, device):
    """用三種排序方式比較候選，觀察排序是否一致"""
    rows = [score_sentence(model, tokenizer, c, device) for c in candidates]
    for key in ["logp_total", "logp_per_token", "bpc"]:
        rev = (key != "bpc") # bpc 越小越好，其餘越大越好
        order = sorted(rows, key=lambda r: r[key], reverse=rev)
        print(f"\n依 {key} 排序：")
        for rank, r in enumerate(order, 1):
            print(f" {rank}. [{r[key]:+8.3f}] {r['text'][:36]}")
```

實驗要求與思考題：

- 準備至少六個候選句，長度從 5 字到 60 字不等，且語意上都是合理的回答。觀察三種排序是否一致。
- 找出一組會發生排序翻轉的候選，並解釋為什麼。把它寫進報告——這是你親手驗證教科書結論的第一次。
- 對同一句話，用兩個不同的 tokenizer 計算 PPL 與 BPC。確認 PPL 差很多而 BPC 接近，這驗證了 5.3.4 節的說法。
- 把長度懲罰係數 α 引入評分（分數除以 n_tokens 的 α 次方），掃描 α 從 0 到 1.5，觀察排序如何隨之改變。 $\alpha = 0$ 與 $\alpha = 1$ 分別對應哪兩種評分方式？

程式 5B 遮罩實作與計分範圍驗證

目標：實作 causal mask 與 loss mask，並用測試證明它們正確。這三個測試會直接進入你的 tests/ 目錄，第 13 章實作模型時會用到。

ch05/lab5b_masks.py (測試部分)

```
import torch, pytest
import torch.nn.functional as F
from formosa.masks import apply_causal_mask, build_loss_mask

def test_causal_no_future():
    """未來位置的注意力權重必須為 0"""
    scores = torch.randn(2, 4, 8, 8)
    attn = torch.softmax(apply_causal_mask(scores), dim=-1)
    upper = torch.triu(torch.ones(8, 8, dtype=torch.bool), diagonal=1)
    assert attn[..., upper].abs().max() < 1e-8
    assert torch.allclose(attn.sum(-1), torch.ones(2, 4, 8), atol=1e-6)

def test_loss_mask_counts():
    """有效詞元數必須等於 answer 長度"""
    ids = torch.tensor([[1, 2, 3, 4, 5, 6, 7, 0, 0]]) # 0 為 PAD
    prompt_len, pad_id = 4, 0
    labels = build_loss_mask(ids, prompt_len, pad_id)
    n_valid = (labels != -100).sum().item()
    assert n_valid == 3, f"預期 3 個計分位置，實際 {n_valid}"

def test_padding_invariance():
    """同一句話單獨算與湊批次算，結果應相同"""
    from formosa.model import MiniFormosaGPT
    from formosa.config import ModelConfig
    m = MiniFormosaGPT(ModelConfig(vocab_size=50, d_model=32,
                                    n_layers=2, n_heads=4,
                                    d_ff=64, max_seq_len=16)).eval()
    short = torch.randint(1, 50, (1, 6))
    padded = F.pad(short, (0, 4), value=0) # 補 4 個 PAD
    with torch.no_grad():
        a = m(short)[0, :6]
        b = m(padded, pad_id=0)[0, :6]
```

```
assert torch.allclose(a, b, atol=1e-5), "padding 影響了實質位置的輸出"
```

第三個測試是最有價值的，因為 padding 造成的錯誤最隱蔽：模型仍然會訓練、loss 仍然會下降，只是批次組成不同時結果會微妙地不一致。這種錯誤在單卡開發時可能完全不會被發現，直到你換了批次大小或上了多卡才爆發。

延伸挑戰：故意把 causal mask 的 diagonal 參數從 1 改成 0（也就是連自己都遮住），觀察 loss 會如何變化並解釋原因。再改成 2（可以看到下一個詞），觀察 loss 掉到多低——這會讓你對「偷看答案」的後果有具體的感受。

程式 5C 資料污染偵測器

目標：實作 n-gram 重疊偵測，量化你的測試集有多少比例可能被污染，並產出可人工覆核的報告。

ch05/lab5c_contamination.py (主流程)

```
import json
from pathlib import Path
from formosa.contamination import build_index, contamination_ratio

def main(train_path: Path, test_path: Path, out: Path,
         n: int = 13, threshold: float = 0.5):
    train_texts = [json.loads(l)["text"]
                   for l in train_path.open(encoding="utf-8")]
    test_items = [json.loads(l)
                  for l in test_path.open(encoding="utf-8")]

    print(f"建立索引：{len(train_texts):,} 筆訓練文本，n={n}")
    index = build_index(train_texts, n)
    print(f"索引大小：{len(index):,} 個 n-gram 雜湊")

    rows, flagged = [], 0
    for item in test_items:
        r = contamination_ratio(item["text"], index, n)
        rows.append(**item, "contamination": round(r, 4),
                    "flagged": r >= threshold)
        flagged += r >= threshold

    # 分域統計：哪個領域污染最嚴重？
    by_domain = {}
    for r in rows:
        d = r.get("domain", "unknown")
        by_domain.setdefault(d, []).append(r["contamination"])
    print("\n分域污染率：")
    for d, vals in sorted(by_domain.items()):
        hi = sum(v >= threshold for v in vals)
        print(f" {d:<12} n={len(vals):<5} 平均 {sum(vals)/len(vals):.3f} "
              f"標記 {hi} 筆 ({hi/len(vals):.1%}) ")

    out.write_text(json.dumps(rows, ensure_ascii=False, indent=2),
                   encoding="utf-8")
    print(f"\n總計標記 {flagged}/{len(rows)} 筆，報告已寫入 {out}")
```

實驗要求：

- 先做一個對照組：把訓練集中的十筆文本直接放進測試集，確認偵測器能抓到它們（污染率應接近 1.0）。這是驗證工具本身是否正確的必要步驟。
- 再做一個反向對照：放進十筆確定沒見過的文本，確認污染率接近 0。如果不接近 0，代表 n 設得太小，常見片語造成誤判。
- 掃描 n 從 5 到 20，畫出誤判率與漏判率的曲線，選出適合你的語料的 n 值。中文與英文的最佳值不同，請自己量。
- 分域報告：哪一個領域的污染最嚴重？通常是公文與法規，因為那些文本在網路上大量重複出現。請解釋這對評測設計的意涵。

三個實作如何串成一條線

這三個程式不是各自獨立的練習，它們構成本書評測基礎設施的第一版。理解它們的關係，你才知道為什麼順序是這樣安排的。

程式	產出什麼	被誰使用	往後的演化
5A	評分函式 (PPL / BPC / 三數字)	5B 的驗證、系統案例的面板	第 13 章評測 MiniFormosaGPT
5B	遮罩實作與三個測試	第 11、13 章的模型實作	進入 tests/ 目錄長期使用
5C	污染索引與偵測報告	系統案例、第 9 章的去污染	第 16 章的記憶化偵測共用同一套技術

一個建議的做法：把這三個程式直接放進第 4 章建立的 `src/formosa/` 套件，而不是留在 `ch05/` 目錄裡。它們是基礎設施，不是一次性的作業——你會用它們用到第 40 章。

系統案例：五領域 domain perplexity 面板

本章的系統案例是建立一套分域評測面板：對同一個模型，分別量測它在公文、教科書、社群、製造文件與本土語言五個領域的困惑度，找出模型最陌生的臺灣文本類型，並據此排出補資料的優先序。

建置步驟

1. 為五個領域各收集至少 200 段文本（每段 200 字以上），並依第 8 章的原則記錄來源與授權。本

章階段可先用公開授權的資料。

2. 對五份資料各做一次污染檢查（程式 5C），移除或標記受污染的樣本。
3. 選定至少兩個模型：一個臺灣本土適配的、一個國際開放權重的。記錄版本與取得日期。
4. 對每個模型、每個領域，計算 PPL、BPC 與 tokens_per_char 三個數字。
5. 製作對照表與雷達圖，並回答：哪個領域最陌生？兩個模型的強弱項是否不同？

你應該會觀察到的現象（先預測、再驗證）

領域	預期現象	可能的原因	對應章節
公文與法規	本土模型明顯較好	訓練語料包含臺灣公部門文本	第 9、22 章
社群與口語	兩者都不理想	口語與網路用語在正式語料中稀少	第 9 章
製造技術文件	兩者都很差	這類文本幾乎不在公開語料中	第 22、31 章
本土語言	BPC 極高	低資源語言，tokenizer 也不友善	第 7、39 章
教科書	差距最小	這類文本在各語料中都有一定比例	—

報告要求

- 必須同時報告 PPL、BPC 與 tokens_per_char，並解釋當 PPL 與 BPC 給出不同排序時，你相信哪一個、為什麼。
- 必須說明污染檢查的結果與處置。
- 必須提出補資料的優先序，並附上理由與粗略的取得難度評估。這份優先序會成為你在第 9 章建立語料混合方案的起點。
- 必須寫出限制：哪些結論不成立、樣本數是否足夠、有哪些你沒測到的領域。

章末評量

一、概念題

1. 說明自回歸、遮罩式與去雜訊三種訓練目標的訓練訊號密度差異，並解釋這個差異對資料稀缺的語言（如繁體中文）有什麼意涵。
2. 什麼是 teacher forcing？它帶來什麼好處與什麼代價？曝光偏差在實務上如何被緩解？
3. 因果遮罩與損失遮罩的功能有何不同？各自寫錯會產生什麼徵兆？
4. 為什麼「先算每批次的平均損失、再平均那些平均值」是錯的？什麼情況下這個錯誤特別嚴重？
5. 列出困惑度可比較的三個前提，並各舉一個違反該前提的實際情境。
6. 為什麼驗證集「每用一次就消耗一次」？這與測試集只能看一兩次的規定有什麼關聯？

二、計算題

1. 某模型在驗證集上的詞元平均損失為 2.85（自然對數）。計算困惑度。若詞彙表大小為 32000，說明模型相對於隨機猜測縮小了多少倍的候選範圍。
2. 同一段 1000 字的繁體中文，tokenizer A 切成 700 個詞元、B 切成 1200 個詞元。兩個模型的整段對數概似都是 -2100（自然對數）。分別計算兩者的 PPL 與 BPC，並說明哪個指標可以拿來比較。
3. 一個批次有 4 條序列，有效詞元數分別為 100、50、200、30，各自的平均損失為 3.0、3.5、2.8、4.0。用正確的方式計算這個批次的整體平均損失，並與「四個平均值再平均」的錯誤做法比較。
4. 一份測試集有 300 筆，n-gram 偵測標記出 24 筆污染。若這 24 筆的模型正確率是 92%，其餘 276 筆是 71%，計算整體正確率，以及移除污染樣本後的正確率。差距說明了什麼？

三、除錯題

1. 同學的模型訓練到第 200 步時 loss 就掉到 0.05，遠低於任何合理範圍。請列出三個最可能的原因，並依檢查的難易度排序。
2. 同學的驗證損失一直低於訓練損失，而且差距穩定。請列出三個可能原因（提示：其中一個與 dropout 有關，一個與資料切分有關）。
3. 同學報告「換了 tokenizer 之後困惑度從 24 降到 11，模型大幅進步」。請指出這個結論的問題，並說明應該補做什麼實驗才能下結論。
4. 同學的評測程式對同一個模型跑兩次得到不同的困惑度。模型已設 eval() 模式且用了 no_grad。請列出三個可能原因。

四、系統設計題

1. 為一個「校務公告問答系統」設計完整的資料切分方案。需說明：用哪種切分方式、為什麼、如何避免同一份公告的不同段落跨集合、以及如何處理公告會被修訂的問題。
2. 你要比較三種 tokenizer (8K、16K、32K 詞彙表) 對繁中生成品質的影響。請設計這個實驗：固定什麼、變動什麼、用什麼指標、跑幾個種子、以及如何確保結論不是由 tokenizer 造成的量測假象。

五、臺灣情境與倫理題

1. 公文與法規類的文本在網路上大量重複出現 (同一份法規會被多個網站轉載)。請說明這對訓練語料去重、對評測污染偵測、以及對困惑度判讀各造成什麼影響。
2. 有人主張「我們的模型在公開排行榜上贏過某國際模型，所以更適合臺灣」。請提出至少三個追問，說明在什麼條件下這個結論才成立。
3. 假設你在建立評測題庫時，發現有些題目的答案來自尚未公開的內部文件。使用這些題目會讓評測更貼近真實需求，但也涉及資訊揭露問題。請說明你會如何處理，並指出哪些判斷需要交給誰。

評量提示 (給自學者)

- 計算題第 1 題： $PPL = \exp(2.85) \approx 17.3$ ；相對於 32000 縮小約 1850 倍。
- 計算題第 2 題：A 的 $PPL = \exp(2100/700) = \exp(3.0) \approx 20.1$ ；B 的 $PPL = \exp(2100/1200) = \exp(1.75) \approx 5.75$ 。兩者的 BPC 相同，都是 $(2100/\ln 2)/1000 \approx 3.03$ ，所以 BPC 才是可比較的指標。
- 計算題第 3 題：正確做法為 $(100 \times 3.0 + 50 \times 3.5 + 200 \times 2.8 + 30 \times 4.0) \div 380 = 1155/380 \approx 3.04$ ；錯誤做法為 $(3.0 + 3.5 + 2.8 + 4.0)/4 = 3.325$ 。錯誤做法高估了短序列的權重。
- 計算題第 4 題：整體 $(24 \times 0.92 + 276 \times 0.71)/300 \approx 0.7268$ ；移除後為 0.71。約 1.7 個百分點的虛高完全來自污染樣本。
- 除錯題第 1 題排序建議：logits 與 labels 沒有錯開一格 (最容易檢查) > causal mask 沒生效 > 訓練與驗證資料重疊。
- 除錯題第 2 題：dropout 在訓練時開啟而驗證時關閉 > 驗證集恰好比訓練集簡單 (切分不當) > 訓練損失是移動平均而驗證損失是瞬時值。

研究延伸與版本注意事項

- 評測方法論：機器學習領域近年對「評測危機」有大量討論，包括排行榜過擬合、benchmark 飽和與污染。這些討論對本章的觀念有直接補充，第 36 章會深入。
- 資料污染研究：關於大模型記憶訓練資料的實證研究值得一讀，它們同時說明了污染偵測的方法與極限。第 16 章的記憶化偵測會用到相同的技術。
- 困惑度與下游表現的關係：困惑度低不一定代表下游任務好，這個落差本身就是研究題目。在你做第 22 章的適配實驗時會親身遇到。
- 長度正規化與解碼：beam search 的長度懲罰有多種形式，第 14 章會展開；若想提早了解，可查閱神經機器翻譯相關的解碼文獻。
- 本章的所有觀念都不會過時，但具體的評測工具與題庫會持續更新。課程使用的分域驗證集版本請以倉庫為準，並在每學期重新檢查授權狀態與污染情形。

常見問題

問：既然困惑度有這麼多陷阱，為什麼還要用它？答：因為它便宜、連續、且直接反映訓練目標。它是很好的「訓練是否正常」的監控指標，也是很好的「同一設定下 A 改動有沒有效」的比較指標。它不好的地方是跨模型、跨 tokenizer 的比較——那需要 BPC 或下游任務評測。工具沒有好壞，只有用對用錯。

問：我的專題只用 RAG，不訓練模型，還需要這一章嗎？答：需要，而且很需要。5.4 到 5.6 節（切分、baseline、消融、污染）完全不涉及訓練，它們是任何評測工作的基礎。事實上 RAG 系統的評測污染問題更嚴重——你的檢索文件庫很可能就包含測試題的答案來源。

問：分域驗證集要多大才夠？答：每個領域至少 200 段、每段 200 字以上，是本課程的最低要求。這個規模足以看出領域間的顯著差異，但不足以偵測 1% 以內的細微改進。若你的專題需要更精細的比較，第 16 章會教你用統計方法算出所需的樣本數。

問：n-gram 污染偵測抓不到改寫過的內容，那意義何在？答：它是四層防治中成本最低的一層，能抓到最常見也最嚴重的完全重複。真正的防線是自己出題（第 36 章）與保留私有題組。任何單一手段都不足夠，這正是為什麼要做四層。

教師使用建議

本章排在第 5 週，建議 2 小時講授加 1 小時實驗。本章是全書「觀念密度最高、但學生最容易覺得不重要」的一章，建議用具體的失敗案例來建立動機。

時段	內容	互動設計	注意事項
前 25 分鐘	5.1 至 5.2 節，訓練目標與遮罩	在白板上手畫 4×4 的因果遮罩表	遮罩必須讓每個學生都畫過一次
中間 35 分鐘	5.3 節，困惑度與三個陷阱	用 5.3.4 節的計算表帶全班算一次	這是本章最核心的觀念
後 30 分鐘	5.4 至 5.5 節，切分與比較紀律	拿一篇真實論文的實驗表，請學生找出缺漏項	找缺漏比講原則有效
最後 30 分鐘	5.6 節，污染與作業說明	示範把訓練樣本放進測試集後分數如何虛高	這個示範的震撼效果很好
實驗課 1 小時	程式 5A 與 5B	5A 兩人一組，各自找一組排序翻轉的例子	5C 可作為課後作業

一個效果很好的課堂活動：事先準備兩份「實驗報告」，一份記錄完整、一份缺了 5.5.4 節清單中的三四項，但結論寫得更漂亮。請學生分組判斷哪一份可信。多數學生第一次會選漂亮的那份，這正是教學的切入點。

高中授課的調整：5.1、5.2 完整教（遮罩的概念用畫格子的方式很容易懂）；5.3 只教到困惑度的直覺與 tokenizer 陷阱，BPC 的公式可略；5.4 至 5.6 濃縮成一句話——「不能拿考過的題目當考題」，並用程式 5C 的對照組實驗來示範。這個示範對高中生特別有說服力。

附：基準評量規格範本

成果檢核要求的規格書可依下表製作。這份規格會在往後六章被反覆引用，建議做成可填寫的表單而非散落的文字。

區塊	應填寫的內容	為什麼需要	引用章節
指標定義	PPL、BPC、tokens_per_char 的計算式與計分範圍	沒有這個，數字無法被別人重算	第 7、13 章
切分方案	切法、種子鹽值、比例、五個分域的規模與來源	切分變了，先前結果全部失效	第 9、22 章
baseline	退化、同類、上界三層的實際數字	沒有 baseline 的數字沒有意義	第 13、16 章

區塊	應填寫的內容	為什麼需要	引用章節
污染檢查	n 值、閾值、對照組驗證、分域污染率	虛高的分數比沒有分數更糟	第 9、36 章
實驗紀錄表單	九項欄位的空白表格	讓每次實驗都以同一格式留下紀錄	第 16 章
判讀規則	什麼情況下可以宣稱「有改進」	事前訂好，避免事後找理由	第 16、36 章

最後一列最容易被略過，但它其實是整份規格的靈魂。請在做任何實驗之前，先寫下：「我認為改進成立的條件是：在三個種子下，該指標的中位數改善超過 X，且分域結果中沒有任何一域顯著退步。」事前訂好判準，才不會在看到數字之後才決定要相信哪一個。這是本書從第 5 章到第 40 章一貫的要求。

本章小結

- 三種訓練目標的差異在訓練訊號密度與語境方向：自回歸每個位置都計分、遮罩式只有 15%、去雜訊介於兩者之間。這解釋了 Decoder-only 在資料稀缺情境下的優勢。
- Teacher forcing 讓訓練可平行化，代價是訓練與推論之間的曝光偏差，主要靠解碼策略與後訓練緩解。
- 因果遮罩控制「能看到什麼」，損失遮罩控制「哪些位置計分」，兩者獨立且都必須測試。causal mask 寫錯會讓 loss 異常低卻毫無用處。
- 計算損失與困惑度時必須累加總損失與總詞元數後再相除；先平均再平均是常見的隱形錯誤。
- 困惑度可比較的三個前提是同 tokenizer、同測試資料、同計分範圍；違反任一項，數字就沒有比較意義。
- 跨 tokenizer 比較應使用 BPC，因為它的分子與分母都與切法無關。本書要求同時報告 PPL、BPC 與 tokens_per_char。
- 長度正規化會造成排序翻轉：總對數機率偏好短句，詞元平均偏好長句，兩者都不完美。
- 資料切分必須在實驗前固定；隨機切分不總是對的，時間切分與群組切分在臺灣的公告、法規、新聞情境中更誠實。
- 五個分域驗證集（公文法規、教科書、社群口語、製造文件、本土語言）構成本書往後所有評測的基本面板。
- 每個實驗至少要有退化、同類與上界三層 baseline；消融實驗一次只改一件事並固定訓練預算；

實驗紀錄九項缺一不可。

11. 資料污染有完全重複、近似重複、答案洩漏與時間洩漏四種型態；n-gram 重疊是成本最低的偵測手段，但真正的防線是自己出題與保留私有題組。

成果檢核

完成本章後你必須繳交一份「基準評量規格」，內容包括：

- 評量指標定義：明確寫出 PPL、BPC、tokens_per_char 的計算方式與計分範圍，並附上可執行的程式。
- 資料切分方案：切法、種子、比例、五個分域驗證集的規模，以及為什麼選這種切法的理由。
- baseline 清單：退化、同類、上界三層各是什麼，以及它們在你的資料上的實際數字。
- 污染檢查報告：程式 5C 的分域結果，含對照組驗證（放進去的樣本有被抓到嗎）與 n 值的選擇理由。
- 實驗紀錄表單：依 5.5.4 節的九項製作，可直接填寫。
- 一頁分析：你的模型（或所選的參考模型）在五個領域中最陌生的是哪一個？補資料的優先序為何？

這份規格會在第 7 章（tokenizer 比較）、第 9 章（語料配比）、第 13 章（MiniFormosaGPT 的第一次評測）、第 16 章（實驗科學）、第 22 章（適配前後的回歸測試）與第 36 章（完整評測體系）反覆被引用。它是本書所有量化結論的共同地基——寫得越嚴謹，往後三十五章的每個數字就越站得住腳。

第 6 章

Unicode、繁體中文與臺灣語言現象

Unicode, Traditional Chinese, and Taiwan Language Phenomena

難度：核心 | 建議授課：第 6 週 | 硬體需求：A 級（一般筆電即可完成全部實作，不需 GPU）

叫獸開講

把「臺」統一改成「台」，看起來很貼心，直到有人發現公文裡的「臺北市政府」變成了一個不存在的機關名稱。正規化是有代價的，代價要寫下來。這一章講的是最不性感、卻最會在半夜三點咬你一口的東西：文字本身。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 區分位元組、碼位、字元與字素叢集四個層次，並說明各自在什麼場合是正確的計算單位。
2. 說明 UTF-8 的編碼方式，判斷亂碼的成因並還原常見的編碼錯誤。
3. 比較 NFC、NFD、NFKC、NFKD 四種正規化形式，並判斷哪些轉換是不可逆的。
4. 辨識繁簡差異、臺灣正體字、異體字與一對多轉換的風險，並設計安全的處理策略。
5. 處理全形半形、標點、空白、日期、地址、電話、幣值與 SI 單位的正規化。
6. 正確處理注音符號、臺灣台語羅馬字、客語拼音與原住民族語文字的編碼問題。
7. 處理中英日混寫、程式碼、型號、料號與製造領域縮寫的邊界。
8. 設計具有 provenance、可逆轉換與保留欄位的正規化規格，並寫出對應的測試。
9. 完成一份「臺灣文本工程規格」與不少於 100 筆的單元測試資料。

這一章為什麼放在 tokenizer 之前

第 7 章要訓練 tokenizer，而 tokenizer 的輸入就是本章的輸出。如果你的正規化把「臺」全部改成「台」，那麼你的模型永遠學不會「臺北市政府」這個正式名稱；如果你沒有處理全形數字，那麼「1 2 3」與「123」會被切成完全不同的詞元，白白浪費詞彙表空間。順序不能顛倒：先決定文字長什麼樣，才能決定怎麼切。

6.1 四個層次：位元組、碼位、字元與字素

「這段文字有多長」這個問題沒有唯一答案，因為「長度」可以用四種單位來算，而它們給出的數字不同。搞混它們是中文處理的第一個坑。

6.1.1 四個層次的定義

層次	定義	「臺」的答案	什麼時候用它
位元組 byte	實際儲存的位元組數	3 (UTF-8 下)	檔案大小、網路傳輸、byte-level tokenizer
碼位 code point	Unicode 指派的編號	1 (U+81FA)	Python 的 len()、字元級模型
字元 character	使用者認知的「一個字」	1	日常溝通，但定義模糊
字素叢集 grapheme	顯示上不可分割的單位	1	游標移動、截斷、顯示寬度

對純中文而言四個數字往往一致（除了位元組），所以問題不明顯。但一旦混入其他內容，差異就出來了。

四個層次的實測

```

s = "臺"
len(s)           # 1    (碼位數, Python 的 len 算的是這個)
len(s.encode("utf-8")) # 3    (位元組數)

e = "👨👩👦"
len(e)           # 5    (三個人 + 兩個連接符 U+200D)
len(e.encode("utf-8")) # 18
# 但使用者看到的是「一個」圖案 — 字素叢集為 1

f = "TW"
len(f)           # 2

a = "ā"
import unicodedata as ud
len(ud.normalize("NFC", a)) # 1    (單一碼位 U+0101)
len(ud.normalize("NFD", a)) # 2    (U+0061 + U+0304 組合符號)

```

最後那組值得注意：同一個「ā」可以用一個碼位表示，也可以用「a」加上一個組合用的長音符號表示。兩者在螢幕上看起來完全一樣，但在程式中是不同的字串——比對會失敗、切詞會不同、去重會漏掉。這正是下一節正規化要解決的問題，而它對臺羅與族語文本影響極大。

6.1.2 為什麼這件事在大模型中特別重要

場景	正確的單位	用錯的後果	章節
計算 BPC	字元 (碼位)	指標無法跨 tokenizer 比較	第 5.3.5 節
截斷長文件	字素叢集	把表情符號或組合字截成一半，產生亂碼	第 9 章
byte-level BPE	位元組	無法處理未見過的字元	第 7 章
去重比對	正規化後的碼位	同一份文件因編碼差異被視為兩份	第 9 章
計算 token 稅	碼位對詞元的比值	成本估算失準	第 7 章

6.1.3 臺灣文本會用到的 Unicode 區塊

知道你的文字住在 Unicode 的哪個區域，是撰寫偵測規則與品質統計的基礎。以下是本書處理臺灣語料時會用到的區塊。

區塊	範圍	內容	在臺灣語料中的角色
基本拉丁	U+0000–007F	ASCII	英文、數字、程式碼
拉丁擴充	U+00C0–024F	帶變音符的字母	臺羅、客語拼音、族語
修飾字母	U+02B0–02FF	聲調與修飾符號	注音的四聲符號
注音符號	U+3105–312F	ㄅ到ㄺ	教材、輸入法、發音標註
中日韓標點	U+3000–303F	、。『』《》全形空白	中文正式標點
中日韓統一表意	U+4E00–9FFF	常用漢字	繁體中文主體
擴充 A 區	U+3400–4DBF	罕用漢字	古籍、罕見人名地名
擴充 B 區以上	U+20000 起	極罕用漢字	需 4 位元組，常是造字或古文
全形符號	U+FF01–FF5E	全形英數與標點	排版遺留，通常要轉半形

這張表可以直接轉成程式 6B 的字元分類規則。特別留意最後三列：擴充區的漢字在舊系統中常常無法顯示或被替換成方框，如果你的語料中這些字的比例異常高，多半代表編碼處理出了問題，而不是真的有那麼多罕用字。

6.2 UTF-8、亂碼與編碼還原

6.2.1 UTF-8 的基本規則

UTF-8 是變長編碼：ASCII 字元用 1 個位元組，大部分中文用 3 個，部分罕用字與表情符號用 4 個。它的設計有兩個重要性質，都與大模型直接相關。

範圍	位元組數	涵蓋	對大模型的意涵
U+0000–007F	1	ASCII、英文、數字	英文的位元組成本最低
U+0080–07FF	2	拉丁擴充、希臘、西里爾	含臺羅的帶調字母
U+0800–FFFF	3	中日韓、注音、全形符號	繁體中文的主要區域
U+10000–10FFFF	4	罕用字、表情符號、部分族語	成本最高，也最容易出錯

第一個重要性質是自我同步：從任意位置往前掃，可以判斷出字元邊界。這讓 byte-level 的 tokenizer (第 7 章) 能夠安全運作。第二個是 ASCII 相容：純英文的 UTF-8 與 ASCII 完全相同，這是為什麼英文在 byte 層級的成本天然較低。

從第三列可以直接推出一個結論：同樣的語意，繁體中文的位元組數大約是英文的三倍。這是在 tokenizer 之前就已經存在的結構性劣勢，第 7 章講 token 稅時會把它換算成新臺幣。

6.2.2 亂碼的三種成因與判讀

亂碼不是隨機的，它有明確的模式。學會判讀模式，就能反推原始編碼並還原。

症狀	成因	典型長相	處置
歐洲字母亂碼	UTF-8 被當成 Latin-1 讀	「臺灣」變成六個怪字母	重新編碼再解碼
問號或方框	目標編碼不支援該字元	「𐄂」變成「？」或「□」	無法還原，需回到原始檔
解碼錯誤	Big5 被當成 UTF-8 讀	直接拋出 UnicodeDecodeError	改用正確編碼
半個字	在多位元組中間截斷	句尾出現破碎符號	按字元而非位元組截斷

亂碼的實測與還原

```
# 情況一：UTF-8 被當成 Latin-1 讀（最常見）
bad = "臺灣".encode("utf-8").decode("latin-1")
print(bad)                # è°çŒ ← 這就是典型的歐洲字母亂碼

# 還原：把它編回 latin-1 的位元組，再用 utf-8 解
fixed = bad.encode("latin-1").decode("utf-8")
print(fixed)              # 臺灣

# 情況二：Big5 被當成 UTF-8 讀 — 通常直接報錯
raw = "臺灣".encode("big5") # b'\xbb0\xc6W'
try:
    raw.decode("utf-8")
except UnicodeDecodeError as e:
    print("解碼失敗：", e.reason)

# 正確做法：偵測後再解
def smart_decode(raw: bytes) -> tuple[str, str]:
    for enc in ("utf-8", "big5", "cp950", "gb18030"):
        try:
            return raw.decode(enc), enc
        except UnicodeDecodeError:
            continue
    return raw.decode("utf-8", errors="replace"), "utf-8-lossy"
```

Big5 在臺灣仍然沒有絕跡

許多政府單位的舊系統、企業的歷史文件、以及部分工控軟體的輸出仍使用 Big5 或 CP950。如果你的語料來源包含這些系統，smart_decode 這類函式是必要的，而且必須記錄每個檔案實際使用的編碼——這是第 8 章 provenance 的一部分。順帶一提，Big5 本身也有多個變體（Big5-HKSCS、CP950），對罕用字的支援不同，這是罕用字遺失的常見來源。

6.2.3 截斷的正確做法

三種截斷方式的差異

```
text = "臺灣科技大學智慧機器人學程"

# 錯誤：按位元組截斷，會把中文切成一半
bad = text.encode("utf-8")[:20].decode("utf-8", errors="replace")
# 結果尾端出現替換字元

# 正確：按碼位截斷
ok = text[:8]                # 臺灣科技大學智慧

# 更正確：按位元組上限但保持字元完整
def truncate_bytes(s: str, max_bytes: int) -> str:
    b = s.encode("utf-8")
    if len(b) <= max_bytes:
        return s
    b = b[:max_bytes]
    # 往回退到合法的字元邊界（UTF-8 的自我同步特性）
    while b and (b[-1] & 0xC0) == 0x80:
        b = b[:-1]
    return b.decode("utf-8")
```


6.2.4 PDF 與 OCR 帶來的特殊問題

臺灣的公文、法規、規格書大量以 PDF 流通，而 PDF 抽取出來的文字有一批獨特的毛病。這些問題在第 9 章的清理管線中會反覆出現，這裡先建立辨識能力。

症狀	成因	辨識方式	處置
字與字之間有空白	PDF 為排版而插入的字距	中文字之間出現單一空白	中文字之間的空白可安全移除
斷行在句中	按版面而非語意換行	換行後不是句末標點	合併行，但保留段落邊界
直書變亂序	直排文件的抽取順序錯誤	文字順序完全不通	需改用版面分析工具
表格變成一串字	表格結構遺失	數字與文字混雜無分隔	標記為表格，另行處理
OCR 形近字錯誤	辨識錯誤	「日」與「曰」、「己」與「巳」	需人工或語言模型校正
缺字或造字	字型嵌入問題	出現替換字元或空白	回到原始 PDF 確認

第一列的處理有一個中文特有的技巧：因為中文本來就不需要空白，所以「兩個漢字之間的單一空白」幾乎一定是排版產物，可以安全移除。但「漢字與英文之間的空白」則要保留，因為那可能是有意義的分隔。

中文字距空白的安全移除

```
import re

CJK = r"[\u4e00-\u9fff\u3400-\u4dbf]"

def remove_cjk_spacing(s: str) -> str:
    """只移除兩個漢字之間的單一空白，保留中英之間的空白"""
    prev = None
    while prev != s:
        prev = s
        s = re.sub(f"({CJK}) ({CJK})", r"\1\2", s)
    return s

def merge_broken_lines(text: str) -> str:
    """合併句中斷行：前一行不以句末標點結束就合併"""
    lines = text.split("\n")
    out = []
    for line in lines:
        line = line.rstrip()
```

```

    if out and out[-1] and out[-1][-1] not in "。！？；：」』)\n":
        out[-1] += line.lstrip()
    else:
        out.append(line)
    return "\n".join(out)

```

注意 `remove_cjk_spacing` 用了一個 `while` 迴圈反覆套用。原因是正規表示式的比對會消耗字元：處理「臺灣科技」時，一次 `sub` 只能處理不重疊的配對，需要多跑幾次才能全部合併。這是一個常見的細節錯誤，寫測試時務必包含連續多個空白的案例。

6.3 Unicode 正規化：NFC、NFD、NFKC、NFKD

6.3.1 四種形式的分工

正規化解決的問題是：同一個「看起來一樣」的文字，可能有多種內部表示。四種形式沿著兩個維度組合。

形式	組合方向	是否做相容分解	典型效果
NFC	盡量合成	否	a + 長音符 → ā (單一碼位)
NFD	盡量分解	否	ā → a + 長音符 (兩個碼位)
NFKC	盡量合成	是	A → A、1 → 1、① → 1、(株) → (株)
NFKD	盡量分解	是	同上，但保持分解狀態

K 代表 *compatibility* (相容性)。做相容分解的兩種形式會把「視覺上不同但語意相同」的字元統一，這通常是好事——但它也是不可逆的，而這正是危險所在。

6.3.2 NFKC 會做什麼：一份實測清單

輸入	碼位	NFKC 輸出	影響評估
A (全形)	U+FF21	A	通常想要——避免全半形分裂詞彙表
1 (全形)	U+FF11	1	通常想要
% (全形)	U+FF05	%	通常想要
() 全形括號	U+FF08/09	()	要小心——中文排版習慣用全形

輸入	碼位	NFKC 輸出	影響評估
① (圈數字)	U+2460	1	會遺失「這是條列編號」的資訊
(株)	U+3231	(株)	一個字變三個字，長度改變
... (省略號)	U+2026	...	中文排版通常保留全形
ii (羅馬數字)	U+2171	ii	法規條號中可能有意義
、。(中文標點)	U+3001/3002	不變	好消息：中文標點不受影響

這張表的關鍵訊息是：NFKC 不是「清理」，而是「统一到某個標準」，而那個標準未必是你要的。圈數字變成普通數字之後，「①②③」與「123」就分不出來了；如果你的語料是教材或法規，那個編號結構可能是重要資訊。

6.3.3 不可逆性：正規化的核心風險

這是本章最重要的觀念。NFC 與 NFD 之間可以來回轉換，但 NFKC 與 NFKD 是單向的——資訊被丟掉了，回不去。

轉換	可逆嗎	為什麼	處置原則
NFC ↔ NFD	可逆	只是組合方式不同，資訊等價	安全，可自由使用
→ NFKC	不可逆	① 與 1 合併後無法區分	必須保留原文
繁簡轉換	不可逆	多對一映射（後述）	必須保留原文
全形轉半形	大多不可逆	轉換後無法知道原本是全形	視情境決定
移除空白與換行	不可逆	結構資訊遺失	僅用於比對，不用於儲存

本書的原則是：正規化後的文字用於訓練與比對，但原始文字必須保留，並且每一筆資料都要記錄「套用了哪些轉換」。這是第 8 章 provenance 的一部分，也是程式 6A 的核心設計要求。

一個真實的教訓

有團隊為了統一格式，對整份語料做了 NFKC 加繁簡轉換，然後刪掉了原始檔案以節省磁碟空間。三個月後客戶問「為什麼模型把我們公司的『株』寫成『(株)』」，他們無法回答，也無法還原。磁碟很便宜，原始

資料無價——這句話請寫在你的專案 README 第一行。

6.4 繁簡、異體字與臺灣正體

6.4.1 繁簡轉換不是一對一

很多人以為繁簡轉換就是查表替換，這在簡轉繁時會出大問題，因為簡體字經常是多個繁體字的合併。

簡體	對應的繁體	判斷依據	錯誤範例
发	發 / 髮	語意（發生 vs 頭髮）	「理发」→「理發」（應為理髮）
台	台 / 臺 / 檯 / 颱	語境（台灣 vs 檯燈 vs 颱風）	「台风」→「台風」（應為颱風）
干	干 / 乾 / 幹	語意（干擾 vs 乾燥 vs 幹部）	「干燥」→「干躁」（應為乾燥）
里	里 / 裡	語意（公里 vs 裡面）	「里面」→「里面」（應為裡面）
面	面 / 麵	語意（表面 vs 麵條）	「面条」→「面條」（應為麵條）
板	板 / 闆	語意（木板 vs 老闆）	「老板」→「老板」（應為老闆）

這意味著：簡轉繁必須依賴語境判斷，純查表一定會出錯。而繁轉簡相對安全（多對一），但也不是完全無損——轉完之後就回不去了。

對本書的讀者而言，這件事有一個直接的實務結論：如果你的語料來源包含簡體文本，最安全的做法不是「轉成繁體再訓練」，而是「標記它是簡體，並在語料配比中控制它的比例」。轉換引入的錯誤會直接進入模型的知識，而且極難察覺。

6.4.2 臺灣正體字與其他繁體區的差異

即使同樣是繁體，臺灣、香港與其他地區的用字標準也不完全相同。

類別	臺灣標準	其他常見	說明
異體字選擇	裡	裏	「裡面」在臺灣教育部標準為「裡」
異體字選擇	為	爲	字形細節差異，皆為合法漢字
正式用字	臺	台	公文與正式名稱應用「臺」
部件差異	啟	啓	教育部標準字體與異體字

類別	臺灣標準	其他常見	說明
詞彙差異	滑鼠、程式	滑鼠、程序	見第 1.6.1 節的用語表

第三列是臺灣特有的難題。「臺」與「台」在日常使用中幾乎可以互換，但在公文、機關名稱、法規條文中，「臺」是正式用字。如果你的正規化把它們統一了，模型就再也學不會這個區別——而這個區別在公文生成的場景中是硬需求。

6.4.3 罕用字、造字區與古籍文獻

臺灣的教育與文史語料中，罕用字的比例高於一般網路文本。這帶來三個具體問題。

問題	情況	後果	處置
擴充區漢字	人名地名用字在 U+20000 以上	tokenizer 可能不認識，變成 UNK	byte fallback (第 7 章)
私人造字區	舊系統用 PUA 存自造字	換系統後變成亂碼或空白	需原始造字對照表
缺字替代	用組字式或注音代替缺字	「𠄎 亻 政」或「ㄇ 尤」	標記並保留，勿自動替換
異體字並存	同一人名在不同文件寫法不同	去重與檢索失效	建立別名表，不改原文

私人造字區 (Private Use Area, U+E000 起) 特別麻煩：那個區域的碼位沒有標準意義，各家系統各自定義。舊的戶政、學籍、醫療系統常用它來存放罕用人名字。如果你的語料來自這些系統，看到 PUA 字元時不要當成亂碼刪掉——那可能是某個人的名字，而且沒有對照表就永遠還原不了。

本書的處理原則：偵測到 PUA 字元時，標記該文件為「需要來源方提供對照表」，並在 Data Card 中記錄。這是一個典型的「工程上無解、必須回到組織層面處理」的問題，第 8 章會把這類情況歸類為資料取得階段就該問清楚的事項。

6.4.4 本書的處理原則

1. 不做繁簡轉換。若語料含簡體，標記來源並在配比中控制，而非轉換。
2. 不統一「臺」與「台」。兩者都保留，讓模型從語境中學會何時用哪一個。
3. 異體字 (裡/裏、為/爲) 可依教育部標準做有限度的統一，但必須記錄轉換清單並保留原文。
4. 所有轉換都要有對應的測試案例，證明它在你的語料上不會產生語意錯誤。
5. 涉及法規條文、機關名稱、人名、地名的文本，一律不做任何字形轉換。

為什麼第五條要單獨列出

因為法規與機關名稱是「字面即效力」的文本。「臺北市政府」與「台北市政府」在法律文件中不是同一個字串，人名的異體字（例如「杰」與「傑」）更是不能改的——那是身分識別資訊。第 8 章談個資時會回到這個問題。工程上的判斷是：涉及識別與效力的欄位，一律保持原樣，並在資料結構中標記為「不可正規化」。

6.5 全形半形、標點與結構化欄位

6.5.1 全形半形的取捨

全形字元占兩個字位寬度，是中文排版的傳統。問題是同一個字元有兩種形式，會讓詞彙表被切成兩半——「A 1」與「A1」在 tokenizer 眼中是完全不同的東西。

類別	建議處置	理由	例外
全形英數字	轉半形	減少詞彙表分裂，語意無損	無
全形空白 U+3000	轉半形空白	避免斷詞混亂	需保留排版格式時
中文標點（、。」「」）	保留	中文的正式標點，語意有別於半形	無
全形括號（）	視情境	中文排版習慣用全形	技術文件中英混寫時可轉
全形冒號、分號	視情境	公文格式可能有規定	法規文本保留

第三列必須特別強調：不要把中文標點轉成英文標點。「，」與「,」在中文語境中不是同一個符號，而 NFKC 剛好不會動它們——這是 NFKC 對中文相對友善的地方。但要注意 NFKC 會把全形括號、全形冒號、全形百分號轉成半形，如果你不想要，就得在 NFKC 之後把它們改回來，或者根本不用 NFKC 而是自訂轉換表。

6.5.2 結構化欄位的正規化

臺灣文本中有一批高度結構化的內容，它們的格式變化多但語意固定。統一它們可以大幅提升 tokenizer 效率與模型的一致性。

欄位	常見變體	建議標準形	注意事項
日期	114/8/20、民國 114 年 8 月 20 日、2025-08-20	保留原文，另加標準化欄位	民國與西元轉換需明確記錄
電話	05-5342601、 (05)5342601、055342601	統一為 05-5342601	分機格式需另行處理
地址	雲林縣斗六市大學路三段 123 號	保留原文	地址是識別資訊，見第 8 章
幣值	NT\$1,000、新臺幣一千 元、1000 元	保留原文	大寫數字在契約中有法律意義
數量單位	10 公分、10 cm、10CM	統一空白與大小寫	SI 單位建議統一為小寫標準符號
百分比	50%、50%、百分之五十	統一符號形式	法規文本中的中文數字保留

請注意日期那一列：民國紀年是臺灣文本的特色，也是模型最容易出錯的地方。「114 年」對一個以西元為主的模型而言是一個不知所云的數字。建議的做法不是把它轉成西元（那會遺失原文），而是保留原文並在資料結構中另加一個標準化欄位，讓模型同時看到兩種表示。

結構化欄位的識別與標註

```
import re

PATTERNS = {
    "roc_date": re.compile(r"(?:民國)?(\d{2,3})年(\d{1,2})月(\d{1,2})日"),
    "roc_short": re.compile(r"\b(\d{3})/(\d{1,2})/(\d{1,2})\b"),
    "phone_tw": re.compile(r"0\d{1,2}[-\s]?\d{6,8}"),
    "law_ref": re.compile(r"第\s*[一二三四五六七八九十百\d]+\s*條"),
    "id_like": re.compile(r"[A-Z][12]\d{8}"),          # 身分證格式
    "unified_no": re.compile(r"\b\d{8}\b"),           # 統一編號
}

def annotate(text: str) -> list[dict]:
    """回傳所有命中的結構化欄位，不修改原文"""
    hits = []
    for name, pat in PATTERNS.items():
        for m in pat.finditer(text):
            hits.append({"type": name, "span": m.span(),
                        "raw": m.group(0)})
    return sorted(hits, key=lambda h: h["span"][0])

def roc_to_ce(y: int, m: int, d: int) -> str:
    """民國轉西元，僅用於加註標準化欄位，不取代原文"""
    return f"{y + 1911:04d}-{m:02d}-{d:02d}"
```

注意 `annotate` 的設計：它只標註不修改。這是本章反覆強調的原則——正規化管線的每一步都應該是「加註」而非「破壞」，直到最後一步才產生訓練用的文字。這樣你隨時可以回到任何一個中間狀態。

另外請留意 `id_like` 與 `unified_no` 這兩個樣式。它們抓的是身分證字號與統一編號的格式——這不是為了正規化，而是為了在第 8 章做個資遮罩時能夠識別。本章先把偵測建好，第 8 章再決定怎麼處理。

6.6 注音、臺羅、客語與族語文字

6.6.1 注音符號

注音符號在 Unicode 中有獨立區塊（U+3105 起），聲調符號則借用了修飾字母區。這造成一個實務問題：聲調符號與一般的重音符號共用碼位，容易被錯誤處理。

符號	碼位	名稱	注意事項
ㄅ ㄆ ㄇ	U+3105 起	Bopomofo Letter	獨立區塊，處理單純
ˊ（二聲）	U+02CA	Modifier Letter Acute	與重音符號同區
ˇ（三聲）	U+02C7	Caron	也用於捷克語等，非注音專用
ˋ（四聲）	U+02CB	Modifier Letter Grave	同上
˙（輕聲）	U+02D9	Dot Above	NFKC 會把它變成組合用符號，需留意

最後一列是一個具體的陷阱：輕聲符號在 NFKC 之下會被轉成「空白加組合用上點」，長度從 1 變成 2，而且視覺上可能不同。如果你的語料包含注音教材（這在教育應用中很常見），做 NFKC 之前務必先測試。

6.6.2 臺灣台語羅馬字

臺羅使用帶調符號的拉丁字母，這帶來一個 NFC/NFD 的經典問題：同一個字可以有兩種內部表示。

臺羅的正規化實測

```
import unicodedata as ud

word = "Tâi-uân"          # 臺灣的臺羅拼法

nfc = ud.normalize("NFC", word)
nfd = ud.normalize("NFD", word)
print(len(nfc), len(nfd)) # 7 9 ← â 與 â 在 NFD 下各變成兩個碼位
```



```
print(nfc == nfd)          # False ← 看起來一樣，比對卻不相等

# 結論：臺羅文本一律先做 NFC，否則去重與比對都會出錯
def normalize_tailo(s: str) -> str:
    s = ud.normalize("NFC", s)
    # 常見的錯誤輸入：用 ASCII 撇號代替連字號、用數字調號
    s = s.replace("'", "-").replace("’", "-")
    return s
```

臺羅的第二個問題是輸入法多樣性：有人用組合鍵直接打出帶調字母，有人用「數字調號」（例如 Tai5-uan5）再轉換，有人混用。同一個詞在語料中可能有三四種寫法，而它們在 tokenizer 眼中是完全不同的詞。

寫法	範例	常見來源	處置
帶調字母 (NFC)	Tâi-uân	正式教材、辭典	標準形，其餘轉成這個
帶調字母 (NFD)	Tâi-uân (分解)	某些系統輸出	轉 NFC
數字調號	Tai5-uan5	輸入法中繼、論壇	需轉換規則
無調	Tai-uan	非正式書寫	無法還原調號，標記為低品質

第四列的處置值得說明：無調的臺羅在語意上是歧義的（不同聲調是不同的詞），所以不應該直接補上調號。正確做法是標記為低品質資料，在語料配比中降低權重，或只用於特定任務。這是第 9 章品質過濾的一個具體案例。

6.6.3 客語與原住民族語

語言	書寫系統	技術問題	本書處理
客語	漢字 + 客語拼音方案	拼音方案有多套，腔調差異大	標記腔調，不強制統一
原住民族語	拉丁字母書寫系統	族語各異，部分字母罕見	保留原樣，勿做相容分解
共通問題	—	語料極少、標準化程度低	第 39 章的低資源策略

原住民族語的書寫系統使用一些在其他語言中罕見的字母組合，做 NFKC 相容分解時有機會被改變。本書的原則很簡單：族語文本一律只做 NFC，不做 NFKC，並且不做任何字元替換。這類語料極為稀少且珍貴，寧可保守處理。

第 39 章會把這些語言當成一個完整的工程題目來處理（語音、轉寫、低資源微調）。本章的任務是確保它們在進入語料庫時沒有被文字處理環節破壞——這是最基礎、也最容易被忽略的一步。

6.7 中英混寫、程式碼與製造領域文本

6.7.1 臺灣技術文本的混寫特徵

臺灣的技術文件有一個鮮明的特徵：中英日文與符號高密度混寫。這對 tokenizer 與模型都是挑戰。

類型	實例	處理難點	章節
中英混寫	請執行 SPC 分析並更新 CIM 系統	英文縮寫的邊界與大小寫	第 7 章
料號與型號	M8×1.25P、SKD11、FANUC-0iMF	含符號、數字、大小寫混合	第 7 章詞彙表設計
單位與規格	±0.02mm、Ra1.6、HRC58-62	正負號、小數、範圍表示	本章 6.5.2 節
日文殘留	設備手冊中的假名與漢字	日文漢字與中文漢字同碼位	需語言偵測
程式碼片段	巨集、PLC 梯形圖註解、G-code	空白與換行有語意	不可移除空白

第四列是一個容易被忽略的問題：日文漢字與中文漢字在 Unicode 中共用碼位（中日韓統一表意文字）。所以你無法用字元範圍判斷一段文字是中文還是日文——必須看假名、看詞彙、或用語言偵測模型。臺灣的工具機、半導體設備手冊中日文殘留相當常見，若不處理會混入語料。

6.7.2 空白的語意

中文不需要空白，所以很多人習慣性地把空白全部移除。這在含程式碼、表格或英文的文本中是災難。

情境	空白的角色	移除的後果	正確處置
純中文散文	無語意	無影響	可壓縮連續空白
中英混寫	分隔英文詞	英文黏在一起	保留單一空白
程式碼	縮排即語法	程式碼失效	完全保留
表格與對齊	欄位分隔	結構遺失	轉成結構化格式

情境	空白的角色	移除的後果	正確處置
公文格式	首行縮排、對齊	格式資訊遺失	保留或另存格式欄位

本書的處理原則是分型處理：先判斷文本類型（散文、程式碼、表格、公文），再套用對應的空白規則。這需要在清理管線的早期就做類型判定，第 9 章會把它變成一個完整的分流機制。

6.7.3 句子邊界：中文特有的難題

切句看起來簡單——看到句號就切。但臺灣的實際文本會讓這條規則失效。

情況	例子	為什麼難	處置
法規條號	第 3 條之 1 規定.....	「之 1」中的數字與句號混淆	先標註條號再切句
小數與版本	ISO 9001:2015 第 4.2 節	小數點不是句末	用前後字元判斷
英文縮寫	Dr. Lee 與 Prof. Wang	英文句點的歧義	維護縮寫清單
引號內的句子	他說：「這樣不行。」然後離開	句號在引號內	引號需成對處理
條列式文本	一、..... 二、.....	沒有句末標點	把條列標記視為邊界
公文分項	(一)(二)(三)	括號編號	同上

中文切句的實用實作

```
import re

SENT_END = "。！？；"
CLOSERS = "「」『』〉〉〉\'"

def split_sentences(text: str, protected_spans=None) -> list[str]:
    """protected_spans 來自 6.5.2 節的標註，這些區段內不切"""
    protected = set()
    for sp in (protected_spans or []):
        protected.update(range(sp["span"][0], sp["span"][1]))

    out, buf = [], []
    i = 0
    while i < len(text):
        ch = text[i]
        buf.append(ch)
        if ch in SENT_END and i not in protected:
            # 把緊接在後的收尾符號一起吃掉
            j = i + 1
            while j < len(text) and text[j] in CLOSERS:
                buf.append(text[j]); j += 1
            out.append(''.join(buf))
            buf = []
            i = j
```

```

        out.append("".join(buf)); buf = []
        i = j
        continue
    i += 1
    if buf:
        out.append("".join(buf))
    return [s.strip() for s in out if s.strip()]

```

注意 `protected_spans` 這個參數——它把 6.5.2 節的結構化標註接了進來。這是本章一再強調的設計：各個處理步驟不是獨立的黑盒，而是共享同一份標註資訊。條號、料號、日期一旦被標出來，後面的切句、切塊、正規化都能繞開它們。

切句的品質會直接影響第 30 章的 RAG 效果——切塊的邊界若落在句子中間，檢索到的片段會缺頭少尾，生成的答案就會怪怪的。這也是為什麼本章要放在 RAG 之前很久的地方：地基的每一塊都會往上傳導。

6.7.4 語言判定：漢字共用碼位的困境

前面提過中日韓漢字共用碼位，所以無法用字元範圍分辨中文與日文。但語料清理必須分辨——你不會想讓大量日文手冊混進繁中語料而不自知。

線索	判斷方式	可靠度	限制
假名	出現平假名或片假名即為日文	高	純漢字的日文段落抓不到
韓文字母	出現諺文即為韓文	高	同上
專有字形	日文特有的簡化字（駅、図、単）	中高	需維護字表
標點習慣	日文用「，」較少、頓號用法不同	中	容易誤判
繁簡特徵字	簡體特有字（们、这、国）即為簡中	高	繁簡混寫文本會誤判
詞彙特徵	臺灣用語 vs 其他中文區用語	中	需詞表，見第 1.6.1 節

實用的語言判定啟發式

```

import re

KANJI = re.compile(r"[\u3040-\u30ff]")
HANGUL = re.compile(r"[\uac00-\ud7af\u1100-\u11ff]")
# 簡體特有字（僅列示意，實務上需完整字表）
SIMP_ONLY = set("们这国说时会个业务经济发实现")

```

```
# 日文特有簡化字
JP_ONLY = set("駅図单壳価済応")

def guess_language(text: str) -> dict:
    n = len(text) or 1
    kana = len(KANA.findall(text)) / n
    hangul = len(HANGUL.findall(text)) / n
    simp = sum(1 for c in text if c in SIMP_ONLY) / n
    jp = sum(1 for c in text if c in JP_ONLY) / n

    if hangul > 0.01:
        label = "ko"
    elif kana > 0.01 or jp > 0.005:
        label = "ja"
    elif simp > 0.005:
        label = "zh-Hans"
    else:
        label = "zh-Hant"
    return {"label": label, "kana": kana, "simp": simp, "jp_char": jp,
            "confidence": "low" if max(kana, simp, jp) < 0.002 else "high"}
```

請注意 `confidence` 這個欄位。一段純漢字、沒有任何特徵字的短文，可能是繁中、也可能是純漢字的日文標題——這時候應該誠實標記為低信心，而不是硬猜。低信心的文件送去人工抽樣，這比讓錯誤資料靜靜混進語料好得多。

第 9 章的清理管線會用這個函式做語言配比統計，第 22 章做繁中適配時也會用它來確認訓練語料的組成。一個常見的驚訝是：許多號稱「繁體中文」的公開語料，其中有相當比例其實是簡體轉繁體的產物——而那正是本章 6.4.1 節說的一對多錯誤的來源。

6.8 正規化規格的设计

6.8.1 五個設計原則

1. 可組合：每一項轉換是獨立的小函式，可依需求開關，而非一個大黑盒。
2. 可追溯：每一項轉換都記錄「改了什麼、改了幾處」，寫進處理紀錄。
3. 保留原文：正規化的輸出是新欄位，不覆寫原始文字。
4. 有例外機制：法規、人名、地名、識別碼等欄位可標記為不可轉換。
5. 有測試：每一項轉換至少三個測試案例，包含一個「不該被改動」的反例。

6.8.2 管線的建議順序

順序很重要，因為前一步的輸出是後一步的輸入。以下是本書建議的順序與理由。

順序	步驟	做什麼	為什麼在這個位置
1	編碼偵測與解碼	確保得到正確的 Unicode 字串	編碼錯了後面全錯

順序	步驟	做什麼	為什麼在這個位置
2	亂碼偵測	標記或修復已知的亂碼模式	趁還能看出模式時處理
3	結構化欄位標註	標出日期、電話、條號、識別碼	在文字被改動前先定位
4	NFC 正規化	統一組合形式	安全且可逆，應盡早做
5	選擇性相容轉換	全形英數轉半形等	逐項開關，非整份 NFKC
6	空白與標點處理	依文本類型套用規則	需先知道類型
7	品質檢查	亂碼率、罕用字率、語言比例	產生可供第 9 章使用的統計

第三步放在第四步之前是刻意的：先標註結構化欄位，之後即使文字被修改，你仍然知道原本的位置與內容。反過來做的話，正規化可能改變字元數，讓先前記錄的位置全部失效。

6.8.3 處理紀錄的格式

每筆文件的處理紀錄

```
{
  "doc_id": "gov-2026-0815-0031",
  "source_encoding": "big5",           // 偵測到的原始編碼
  "raw_sha256": "a3f2...",           // 原文雜湊，可驗證未被竄改
  "transforms": [
    {"op": "decode", "from": "big5", "to": "utf-8"},
    {"op": "nfc", "changed": 12},       // 改了 12 處
    {"op": "fullwidth_alnum", "changed": 47},
    {"op": "collapse_space", "changed": 8}
  ],
  "protected_spans": [                // 標記為不可轉換的區段
    {"type": "law_ref", "span": [120, 128], "raw": "第二十七條"},
    {"type": "org_name", "span": [3, 10], "raw": "臺北市政府"}
  ],
  "quality": {
    "replacement_char_ratio": 0.0,     // 亂碼指標
    "rare_char_ratio": 0.003,
    "lang_ratio": {"zh": 0.82, "en": 0.16, "ja": 0.02}
  }
}
```

這份紀錄會直接進入第 8 章的 Data Card，也會被第 9 章的清理管線使用。請注意 `protected_spans` 這個欄位——它是本章與第 8 章個資處理的介面：本章負責找出它們，第 8 章決定要遮罩、保留還是刪除。

6.8.4 量化正規化的效益與風險

正規化不能憑感覺做。每一項轉換都應該有數字支撐：它省下了多少詞元？它改動了多少字元？它有沒有破壞任何受保護的內容？

轉換	效益指標	風險指標	判斷準則
NFC	去重命中率提升	幾乎無風險	一律執行
全形英數轉半形	詞彙表中重複條目減少	排版資訊遺失	效益通常遠大於風險
壓縮連續空白	詞元數下降	程式碼與表格失效	僅對散文型文本執行
移除中文字距	詞元數明顯下降	可能誤刪有意義的空白	需測試中英交界處
異體字統一	同詞不同寫法合併	人名地名可能被改	必須配合保護清單

一個具體的量化方法：在正規化前後各訓練一次 `tokenizer`（第 7 章），比較同一份測試文本的詞元數。如果正規化讓詞元數下降 5%，那就等於你的上下文長度增加了 5%、API 成本下降了 5%——這是實實在在的收益，而且只需要做一次前處理。

風險面的量化則靠破壞測試：準備一份含有所有受保護樣式的測試集，跑完整條管線，統計有幾筆被改動。本書的要求是零破壞——不是「破壞率很低」，而是零。因為那些欄位一旦被改，錯誤會直接進入模型知識，而且極難察覺。

一個判斷原則

當你不確定某項轉換該不該做時，問三個問題：一、它可逆嗎？二、它會影響受保護的欄位嗎？三、它帶來的詞元節省有多少？如果答案是「不可逆、可能影響、節省不到 1%」，那就不要做。保守處理文字的代價通常只是多一點磁碟與詞元，而激進處理的代價是無法還原的資訊遺失。

程式實作

本章三個實作都在 A 級硬體上完成，且不需要大量資料。程式碼位於 ch06/，並會被第 7、8、9 章直接引用。

程式 6A 臺灣繁體中文 normalizer

目標：建立一個可組合、可追溯、保留原文的正規化管線，每一項轉換都記錄影響範圍。

formosa/normalize.py (節錄)

```
import unicodedata as ud
import re
from dataclasses import dataclass, field

@dataclass
class NormResult:
    text: str
    transforms: list = field(default_factory=list)

    def apply(self, name: str, fn):
        before = self.text
        after = fn(before)
        if after != before:
            changed = sum(1 for a, b in zip(before, after) if a != b)
            changed += abs(len(after) - len(before))
            self.transforms.append({"op": name, "changed": changed})
        self.text = after
        return self

# — 個別轉換（每一項都獨立、可測試） —

def to_nfc(s: str) -> str:
    return ud.normalize("NFC", s)

FULLWIDTH_ALNUM = {chr(c): chr(c - 0xFEE0)
                    for c in range(0xFF10, 0xFF5B)} # 0 - z

def fullwidth_alnum_to_half(s: str) -> str:
    """只轉英數字母，不動中文標點"""
    return "".join(FULLWIDTH_ALNUM.get(c, c) for c in s)

def normalize_space(s: str, keep_indent: bool = False) -> str:
    s = s.replace("\u3000", " ") # 全形空白
    if keep_indent:
        return re.sub(r"[ \t]{2,}", " ", s)
    return re.sub(r"[ \t]+", " ", s)

def normalize(text: str, *, fullwidth=True, space=True,
```



```

        keep_indent=False) -> NormResult:
    r = NormResult(text)
    r.apply("nfc", to_nfc)
    if fullwidth:
        r.apply("fullwidth_alnum", fullwidth_alnum_to_half)
    if space:
        r.apply("space", lambda s: normalize_space(s, keep_indent))
    return r

```

測試要求（本實作至少要寫 100 筆測試資料，這是成果檢核的硬性要求）：

- 正向測試：每一項轉換至少三筆，確認它做了該做的事。
- 反向測試：每一項轉換至少三筆「不該被改動」的案例。例如中文標點在 `fullwidth_alnum` 之後必須完全不變。
- 組合測試：多項轉換一起套用時，結果與逐項套用相同。
- 臺灣特例：「臺北市政府」「第二十七條」「Tâi-uân」「ㄅㄆㄇ」在預設設定下都不應被破壞。
- 追溯測試：`transforms` 紀錄的 `changed` 數量與實際差異相符。

100 筆測試資料該怎麼設計

成果檢核要求不少於 100 筆測試資料。這個數字不是隨便訂的——它剛好足以涵蓋本章列出的所有陷阱，而且不至於讓人放棄。以下是建議的配額。

類別	筆數	內容	至少要包含
編碼與亂碼	10	UTF-8、Big5、亂碼還原、截斷	一筆無法還原的案例
正規化基本	15	NFC/NFD、全形半形、空白	一筆 NFKC 會出錯的案例
繁簡與異體	15	一對多映射、臺 / 台、異體字	「理发」「台风」「面条」
受保護欄位	20	機關名稱、條號、料號、單位、人名	每種樣式至少三筆
結構化欄位	15	民國日期、電話、統編、百分比	至少三種日期寫法
本土語言	15	注音、臺羅四種寫法、客語、族語	輕聲符號與 NFKC 的衝突
混寫與程式碼	10	中英日混寫、G-code、PLC 註解	一筆縮排有語意的案例

每一筆測試資料的格式建議如下，重點是要寫清楚「為什麼這筆重要」——因為半年後你自己會忘記。

tests/data/normalize_cases.jsonl (範例)

```
{ "id": "PROT-003",
```

```

"category": "受保護欄位",
"input": "臺北市政府教育局函，依據本市高級中等學校免試入學實施要點第七條規定辦理。",
"expect_unchanged": ["臺北市政府教育局", "第七條"],
"expect_changed": [],
"why": "機關全稱中的「臺」不可轉為「台」；條號不可被切句或改寫。",
"added_by": "class-2026", "date": "2026-08-20"}

{"id": "LANG-011",
"category": "本土語言",
"input": "ㄉㄞˊ ㄌㄞˊ",
"forbid_ops": ["nfkc"],
"why": "輕聲符號 U+02D9 在 NFKC 下會被轉成組合用符號，長度與外觀皆改變。",
"added_by": "class-2026", "date": "2026-08-20"}

```

expect_unchanged 這個欄位是本套測試的核心：它斷言的不是「輸出應該長什麼樣」，而是「這些子字串必須原封不動地出現在輸出中」。這種寫法比逐字比對整個輸出更穩健，因為你可以自由調整其他部分的處理方式而不必改測試。

程式 6B 亂碼與異常偵測器

目標：掃描語料，找出亂碼、罕用字、編碼異常與可疑的繁簡混雜，產出可人工審查的報表。

ch06/lab6b_detect.py (節錄)

```

import unicodedata as ud
import re
from collections import Counter

# 典型的 UTF-8 被當 Latin-1 讀之後產生的字元範圍
MOJIBAKE_HINT = re.compile(r"[ÃÄà@è¥ã|§`]{2,}")
REPLACEMENT = "\ufffd"

def char_profile(text: str) -> dict:
    """統計字元組成，作為品質判斷依據"""
    n = len(text) or 1
    cnt = Counter()
    for ch in text:
        cp = ord(ch)
        if ch == REPLACEMENT:
            cnt["replacement"] += 1
        elif 0x4E00 <= cp <= 0x9FFF:
            cnt["cjk"] += 1
        elif 0x3400 <= cp <= 0x4DBF:
            cnt["cjk_ext_a"] += 1
        elif 0x2000 <= cp <= 0x2A6D:
            cnt["cjk_ext_b"] += 1
        elif 0x3105 <= cp <= 0x312F:
            cnt["bopomofo"] += 1
        elif 0x3040 <= cp <= 0x30FF:
            cnt["kana"] += 1
        elif ch.isascii() and ch.isalpha():
            cnt["latin"] += 1
        elif 0xFF01 <= cp <= 0xFF5E:
            cnt["fullwidth"] += 1
    return {k: v / n for k, v in cnt.items()}

def try_fix_mojibake(s: str) -> tuple[str, bool]:
    """嘗試還原 UTF-8→Latin-1 的亂碼"""
    if not MOJIBAKE_HINT.search(s):
        return s, False

```

```

try:
    fixed = s.encode("latin-1").decode("utf-8")
except (UnicodeEncodeError, UnicodeDecodeError):
    return s, False
# 只有在中文比例提升時才採信
before = char_profile(s).get("cjk", 0)
after = char_profile(fixed).get("cjk", 0)
return (fixed, True) if after > before + 0.1 else (s, False)

def audit(docs, thresholds=None):
    th = thresholds or {"replacement": 0.001, "kana": 0.05,
                        "cjk_ext_b": 0.01}
    flagged = []
    for i, d in enumerate(docs):
        p = char_profile(d)
        reasons = [k for k, lim in th.items() if p.get(k, 0) > lim]
        if reasons:
            flagged.append({"idx": i, "reasons": reasons,
                           "profile": {k: round(v, 4)
                                         for k, v in p.items()},
                           "preview": d[:60]})
    return flagged

```

注意 `try_fix_mojibake` 的設計：它不會盲目修復，而是先檢查修復後中文比例有沒有提升。這是一個通用的工程模式——自動修復必須有驗證條件，否則會把好資料改壞。

延伸挑戰：把 kana 比例超標的文件挑出來人工檢視，你會發現臺灣的設備手冊中日文殘留的比例可能高於預期。統計這些文件的來源分布，作為第 8 章語料清冊的一部分。

程式 6C 臺羅與注音的往返轉換測試

目標：驗證聲調符號在不同正規化路徑下的存活率，找出會破壞本土語言文本的操作。

ch06/lab6c_roundtrip.py (節錄)

```

import unicodedata as ud

CASES = [
    ("臺羅", "Tâi-uân ê bûn-hòa"),
    ("臺羅長音", "chhut-hoat, kám-siā, tsò-hué"),
    ("注音", "ㄊㄞˊ ㄄ㄢˊ ㄜˊ ㄅㄨˋ ㄏㄨㄚˊ"),
    ("輕聲", "ㄊㄞˊ ㄄ㄢˊ ㄜˊ ㄅㄨˋ"),
    ("客語拼音", "Thòi-vàn Hak-ngi"),
]

FORMS = ["NFC", "NFD", "NFKC", "NFKD"]

def roundtrip_report():
    print(f"{'案例':<10}{'原長':>5}", end="")
    for f in FORMS:
        print(f"{'f':>8}", end="")
    print(f"{' NFC 可還原':>12}")

```

```

for name, s in CASES:
    print(f"{name:<10}{len(s):>5}", end="")
    for f in FORMS:
        print(f"{len(ud.normalize(f, s)):>8}", end="")
    # 檢查 NFD 之後能否還原成 NFC
    ok = ud.normalize("NFC", ud.normalize("NFD", s)) == \
        ud.normalize("NFC", s)
    kc_ok = ud.normalize("NFKC", s) == s
    print(f"{'是' if ok else '否':>12}",
          f" NFKC 不變: {'是' if kc_ok else '否'}")

```

你應該觀察到的現象與必須寫進報告的結論：

- NFC 與 NFD 之間的長度不同，但可以互相還原——這證實了 6.3.3 節說的可逆性。
- 注音的輕聲符號在 NFKC 下會改變（U+02D9 被轉成組合用符號），長度可能改變。這是一個具體的「NFKC 會破壞本土文本」的證據。
- 臺羅的帶調字母在 NFKC 下不變，但如果你的管線之後又做了「移除變音符號」這類操作，就會全毀。
- 結論：本土語言文本的處理管線，一律只做 NFC，且不做任何字元替換。這條規則要寫進你的文本工程規格。

系統案例：混合語料的清理與保護

本章的系統案例是清理一份真實的混合語料：包含公文、校務文件與電子製造規格書，並在清理過程中保護法規條號、料號與量測單位不被破壞。

資料組成與挑戰

來源	典型內容	技術挑戰	保護重點
政府公文	函、公告、法規引用	Big5 舊檔、全形排版、條號	機關名稱、條號、日期
校務文件	學則、課程大綱、公告	格式雜亂、PDF 轉出的斷行	學分數、課號、系所名稱
製造規格書	料號、公差、設備型號	中英日混寫、符號密集	料號、單位、公差符號

交付要求

1. 清理管線：依 6.8.2 節的七步順序實作，每一步可獨立開關。
2. 保護清單：列出所有被標記為不可轉換的樣式（至少十種），並附上偵測用的正規表示式與測試

案例。

3. 處理紀錄：每份文件產生一份 6.8.3 節格式的紀錄，含原文雜湊與轉換清單。
4. 品質報表：亂碼率、罕用字率、語言比例的分布圖，並標出需人工審查的文件。
5. 破壞測試：故意關閉保護機制跑一次，統計有多少個條號、料號、單位被破壞。這個數字是你的保護機制價值的量化證明。

評分重點

- 保護機制是否真的有效：助教會用一組刁鑽的測試案例（含「臺北市政府」「M8×1.25P」「±0.02mm」「第二十七條」）驗證。
- 是否保留原文與處理紀錄：無法還原到任一中間狀態者不及格。
- 破壞測試的誠實度：如果報告說「零破壞」，助教會特別仔細檢查。
- 規格文件是否可被他人執行：另一組同學能否照著你的規格重現相同結果。

章末評量

一、概念題

1. 說明位元組、碼位、字元、字素叢集四個層次的差異，並各舉一個「用錯單位會出錯」的實際情境。
2. 為什麼 NFC 與 NFD 之間可逆，而 NFKC 不可逆？請舉兩個 NFKC 會遺失資訊的例子。
3. 為什麼簡轉繁比繁轉簡危險？請舉三組一對多的例子並說明判斷依據。
4. 為什麼本書主張不要統一「臺」與「台」？在什麼情境下這個區別是硬需求？
5. 「先標註結構化欄位，再做正規化」的順序理由是什麼？反過來做會有什麼問題？
6. 為什麼本土語言（臺羅、族語）文本只做 NFC 而不做 NFKC？請用程式 6C 的結果佐證。

二、計算與判讀題

1. 一段 1000 字的純繁體中文，計算它在 UTF-8 下的位元組數。若同樣語意的英文為 600 個 ASCII 字元，比較兩者的位元組成本比。
2. 某語料檔案大小 3.0 GB，經 char_profile 統計 replacement 比例為 0.008。估算受影響的字元數，並判斷這個比例是否需要處理（本書建議閾值為 0.001）。
3. 一份文件經過 NFKC 後長度從 5000 變成 5120。請提出至少三種可能的原因，並說明如何逐一驗證。
4. 某臺羅語料有 40% 使用 NFC、35% 使用 NFD、25% 使用數字調號。若不做正規化就訓練 tokenizer，估計詞彙表會被浪費多少（定性說明即可），並提出處置。

三、除錯題

1. 同學的清理程式跑完後，所有的「臺北市政府」都變成了「台北市政府」。請找出最可能的兩個原因，並說明如何在管線中加入防護。
2. 同學的去重程式報告「找不到任何重複」，但人工檢查明顯有重複文件。請列出三個可能原因（提示：其中一個與本章的正規化有關）。
3. 一份 Big5 的舊公文用 UTF-8 讀取後拋出 UnicodeDecodeError，同學改用 errors="ignore" 就沒事了。請說明這個做法的問題與正確處置。
4. 同學把所有空白移除後，發現製造規格書中的「10 mm」變成「10mm」沒問題，但 PLC 註解中的縮排全毀。請說明應該如何設計分型處理。

四、系統設計題

1. 為一個「校務公告與法規問答系統」設計文本正規化規格。需明確列出：哪些轉換要做、哪些不做、哪些欄位受保護、以及每一項的測試案例。特別說明你如何處理「臺 / 台」與民國紀年。
2. 你要建立一份包含臺語教材的語料庫。請設計一套流程，能處理四種臺羅寫法（NFC、NFD、數字調號、無調），並說明無調文本的處置理由。

五、臺灣情境與倫理題

1. 本章的 PATTERNS 中包含身分證字號與統一編號的偵測樣式。請說明：偵測到之後應該做什麼？在什麼情況下可以保留、什麼情況下必須遮罩？哪些判斷需要交給誰決定？（提示：本章負責偵測，第 8 章負責決策）
2. 有人主張「為了統一，我們把所有簡體語料轉成繁體再訓練」。請提出三個技術反駁，以及這個做法在什麼條件下可能是合理的。
3. 原住民族語語料極為稀少。若你取得了一批族語文本，但來源單位希望你先做「標準化」以符合某個拼寫方案。請說明你會提出哪些技術上的疑慮，以及哪些決定不應由工程師單方面做出。

評量提示（給自學者）

- 計算題第 1 題：1000 個中文字約 3000 位元組，英文 600 字元為 600 位元組，比值 5:1。注意這只是位元組層面，第 7 章的 token 層面比值不同。
- 計算題第 2 題：0.008 遠高於建議閾值 0.001，需要處理。以 3 GB 約 10 億字元估算，受影響字元約 800 萬個——這通常代表有整批檔案的編碼判定錯誤，而非零星問題。
- 計算題第 3 題：可能原因包括圈數字被展開為多字元、ㄅ一類的組合字被展開、羅馬數字被展開為多個字母。驗證方式是逐字元比對 NFKC 前後的差異並統計類型。
- 除錯題第 2 題：未做 NFC 導致同一份文件的兩種表示不相等 > 未統一全形半形 > 比對前未移除格式差異（如換行）。這正是 5.6.6 節說的繁簡雙版問題的變體。

研究延伸與版本注意事項

- Unicode 標準本身：正規化附錄（UAX #15）與字素叢集切分規則（UAX #29）是本章的權威來源，建議至少瀏覽正規化部分。
- 中日韓統一表意文字的設計與爭議：理解為什麼中日文漢字共用碼位，以及這對語言偵測的影響。
- 教育部國語辭典、臺灣台語常用詞辭典與客語辭典：這些是判斷臺灣標準用字的權威依據，也是建立轉換清單的來源。
- 原住民族語書寫系統的相關規範：族語文本處理前務必查閱，並與相關單位確認。

- 本章的 Unicode 行為在不同 Python 版本間相當穩定，但 unicodedata 模組所依據的 Unicode 版本會隨 Python 版本更新。請在環境指紋（第 4.7.3 節）中記錄 unicodedata.unidata_version。

常見問題

問：直接用 NFKC 一次搞定不行嗎？答：對純英文資料多半可以，對臺灣的中文資料不行。本章列出的清單顯示 NFKC 會改動圈數字、組合字、羅馬數字、輕聲符號與全形括號，其中好幾項在教材、法規與注音文本中是有意義的。正確做法是逐項選擇你要的轉換，而不是套用一個包裹方案。

問：這些細節真的會影響模型品質嗎？答：會，而且影響的方式是隱形的。全形半形不統一會讓同一個詞占用兩個詞元位置，浪費詞彙表；未做 NFC 會讓去重失效，導致重複資料進入訓練；繁簡轉換錯誤會把錯字寫進模型知識。這些都不會讓程式報錯，只會讓模型悄悄變差。

問：我的專題只做 RAG，也需要這一章嗎？答：需要。檢索的比對、切塊的邊界、引用的原文還原，全都建立在文字處理之上。實際上 RAG 對正規化更敏感——查詢用了全形數字而文件用半形，就檢索不到。

問：保留原文會不會太占空間？答：純文字的壓縮率很高，而且相對於模型權重與 checkpoint，原始語料通常不是磁碟大宗。就算真的很大，也應該是壓縮存檔或移到冷儲存，而不是刪除。這一條沒有妥協空間。

教師使用建議

本章排在第 6 週，建議 2 小時講授加 1 小時實驗。這一章的內容瑣碎但關鍵，建議大量使用現場示範——每個抽象規則都配一個當場跑出來的例子。

時段	內容	互動設計	注意事項
前 25 分鐘	6.1 至 6.2 節，四個層次與編碼	現場示範表情符號的 len() 與亂碼還原	亂碼還原的示範效果極好
中間 30 分鐘	6.3 至 6.4 節，正規化與繁簡	請學生猜 NFKC 會不會改動某個字元，再當場驗證	猜錯的例子最有教學價值
後 30 分鐘	6.5 至 6.7 節，欄位、本土語言、混寫	請學生找出自己領域的三個保護欄位	讓學生連結到自己的專題
最後 35 分鐘	6.8 節與作業說明	示範「關掉保護機制」的破壞測試	破壞測試是本章最有說服力的一段

時段	內容	互動設計	注意事項
實驗課 1 小時	程式 6A 與 6C	兩人一組互相出刁鑽測試案例	100 筆測試資料可由全班共建

一個效果很好的做法：讓全班共同建立那 100 筆測試資料，每人貢獻五筆並附上「為什麼這筆重要」的說明。收集起來之後，這份測試集會成為班級的共同資產，往後每一屆都可以繼續擴充。這也讓學生第一次體驗到「建立測試集」本身就是有價值的工作——這個觀念在第 36 章會被推到極致。

高中授課的調整：6.1、6.2 完整教（亂碼與表情符號長度的例子對高中生很有吸引力）；6.3 只教 NFC/NFKC 的差別與不可逆性，四種形式的細節可略；6.4 的繁簡一對多用「理髮」的例子講清楚即可；6.5 至 6.7 挑一兩個與學生生活相關的例子（例如注音、臺語）；6.8 簡化成「三個原則：保留原文、逐項可控、每項有測試」。程式 6C 特別適合高中生，因為它只是跑幾行就能看到結果。

附：正規化決策速查表

這張表是本章所有原則的濃縮，可以直接印出來貼在螢幕旁邊，也可以作為規格文件的第一頁。每一列的判斷都在本章有對應的討論。

項目	本書建議	可逆	理由與例外
NFC 正規化	一律做	是	解決同形不同碼的問題，無風險
NFKC 整份套用	不做	否	會破壞圈數字、組合字、輕聲符號
全形英數轉半形	做	否	避免詞彙表分裂；效益明顯
全形標點轉半形	不做	否	中文標點有其語意與排版意義
繁簡轉換	不做	否	一對多映射必然出錯；改為標記來源
臺 / 台統一	不做	否	公文與正式名稱中是硬性區別
異體字統一	有限度做	否	需配合保護清單與轉換紀錄
壓縮連續空白	分型做	否	程式碼與表格不可做
移除中文字距	分型做	否	僅對 PDF 抽取的散文
合併句中斷行	分型做	否	需保留段落邊界

項目	本書建議	可逆	理由與例外
移除變音符號	絕不做	否	會摧毀臺羅與族語文本
刪除罕用字與 PUA	絕不做	否	可能是人名，且無法還原
法規條號改寫	絕不做	否	字面即效力
機關名稱正規化	絕不做	否	同上
人名地名正規化	絕不做	否	身分識別資訊，見第 8 章

注意最後五列都是「絕不做」。這五項構成本書文字處理的紅線——任何一項被違反，資料就永久受損。如果你的規格文件只能寫一頁，請把這五行寫上去。

本章小結

1. 文字長度有四個層次：位元組、碼位、字元、字素叢集。用錯單位會造成截斷破損、指標不可比、去重失效。
2. UTF-8 下繁體中文約為 3 位元組、英文 1 位元組，這是 tokenizer 之前就存在的結構性成本差異。
3. 亂碼有明確模式：UTF-8 被當 Latin-1 讀會產生歐洲字母、Big5 被當 UTF-8 讀會直接報錯。自動修復必須附帶驗證條件。
4. NFC 與 NFD 可逆，NFKC 與 NFKD 不可逆。NFKC 會改動圈數字、組合字、羅馬數字、輕聲符號與全形括號，但不動中文標點。
5. 簡轉繁是一對多的映射，純查表必然出錯；本書主張不做繁簡轉換，改為標記來源並控制配比。
6. 「臺」與「台」在公文與正式名稱中不可統一；法規、人名、地名、識別碼一律標記為不可轉換。
7. 全形英數字建議轉半形以避免詞彙表分裂，但中文標點必須保留。
8. 民國紀年、電話、條號、料號等結構化欄位應先標註再正規化，並保留原文另加標準化欄位。
9. 注音的輕聲符號在 NFKC 下會改變；臺羅有四種寫法需統一為 NFC；族語文本只做 NFC 且不做任何字元替換。
10. 空白在程式碼、表格與公文中有語意，必須分型處理；中日韓漢字共用碼位，無法用字元範圍判斷語言。
11. 正規化規格的五個原則是可組合、可追溯、保留原文、有例外機制、有測試；管線的七步順序中，結構化標註必須在正規化之前。

成果檢核

完成本章後你必須繳交「臺灣文本工程規格 v1.0」，內容包括：

- 規格文件：逐項列出要做與不做的轉換，每一項附上理由、影響評估與可逆性標註。
- 保護清單：至少十種不可轉換的樣式，含偵測規則與測試案例。
- normalizer 實作（程式 6A）：可組合、有 transforms 紀錄、保留原文。
- 測試資料：不少於 100 筆，含正向、反向、組合、臺灣特例、追溯五類。
- 偵測報表（程式 6B）：對課程語料的亂碼率、罕用字率、語言比例統計。
- 往返測試報告（程式 6C）：四種正規化形式對注音、臺羅、客語的影響對照表。
- 破壞測試：關閉保護機制後被破壞的欄位統計，量化你的保護機制的價值。

這份規格是第 7 章訓練 tokenizer 的前置條件，也會被第 8 章的 Data Card、第 9 章的清理管線、第 30 章的檢索前處理直接引用。文字處理是整條資料鏈的地基——地基歪了，上面蓋什麼都會斜。

第 7 章

分詞器、詞彙表與臺灣語言效率

Tokenizers, Vocabularies, and Taiwan Language Efficiency

難度：核心 | 建議授課：第 7 週 | 硬體需求：A 級（一般筆電即可完成全部實作，不需 GPU）

叫獸開講

同一句話，英文用 12 個 token，繁體中文卻用了 31 個。你付的 API 費用是英文的兩倍多，而模型能記住的上下文只有人家的一半。這就是 token 稅——一種沒有人立法通過、卻由每個中文使用者天天在繳的稅。這一章要教你怎麼算出這筆稅，以及怎麼少繳一點。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 比較字元、詞、subword、byte 與 byte fallback 五種切分粒度的取捨。
2. 說明 BPE、WordPiece 與 Unigram LM 的訓練目標差異，並手寫一個可運作的 BPE。
3. 處理 pre-tokenization 的邊界問題，說明為什麼中日韓文字的處理與英文不同。
4. 設計特殊詞元集合，包括 BOS、EOS、PAD、UNK、對話模板與工具呼叫標記。
5. 量測 token fertility、壓縮率與詞彙覆蓋率，並解讀它們的差異。
6. 從零訓練一個 tokenizer，或對既有 tokenizer 做詞彙擴充與 embedding 初始化。
7. 設計臺灣專屬的 tokenizer 公平性測試，涵蓋地名、人名、法規、晶片、機械與校務詞彙。
8. 把 token fertility 換算成 API 成本、上下文容量與訓練 GPU 小時，做出可稽核的決策。
9. 為一個實際專案選定詞彙表大小，並寫出每一個設計決策的理由。

這一章與前後章的關係

第 6 章決定了「文字長什麼樣」，本章決定「文字怎麼切」，第 9 章決定「餵多少、餵什麼」。三者環環相扣：正規化沒做好，tokenizer 會浪費詞彙表在全形半形的重複條目上；tokenizer 沒設計好，第 9 章的語料再乾淨也會被低效的切法吃掉一半價值。這也是為什麼本篇的順序不能對調。

7.1 為什麼需要 tokenizer

7.1.1 模型看不見文字

神經網路吃的是數字，不是文字。所以在任何模型看到你的句子之前，必須有一個東西把文字變成一串整數索引。這個東西就是 tokenizer，而它的設計會影響後面所有環節。

tokenizer 決定了什麼	直接影響	間接影響	章節
序列長度	計算量與記憶體	可用的上下文長度	第 3、11 章
詞彙表大小	嵌入層與輸出層參數量	模型總參數量	第 12 章
切分粒度	模型要學多少組合	收斂速度與資料需求	第 15、17 章
未知字處理	罕用字能否被表示	人名地名的正確性	第 6 章
每字元詞元數	API 成本與訓練成本	對繁中使用者的公平性	本章 7.7 節

請注意第二列：詞彙表大小同時影響輸入端的嵌入表與輸出端的投影層。以 $d = 768$ 為例，詞彙表從 32K 增加到 128K，這兩層加起來會多出約 1.47 億參數——對一個小模型而言，這可能占了總參數量的一大半。詞彙表不是免費的。

7.1.2 五種切分粒度

粒度	一個詞元是	詞彙表大小	序列長度	主要問題
字元	一個字	數千至數萬	很長	序列太長，長距依賴難學
詞	一個詞	數十萬以上	很短	未登錄詞爆炸、中文需斷詞
subword	常見字串片段	數萬	中等	目前的主流平衡點
byte	一個位元組	256	極長	中文成本是英文的三倍
byte fallback	subword 為主，未知時退回 byte	數萬	中等	兼顧覆蓋率與效率

最後一列是現代大模型的標準做法：平常用 subword 切，遇到詞彙表裡沒有的字（例如罕用漢字、族語字母、新出現的表情符號）就退回到位元組層級。這保證了永遠不會出現 UNK，代價是那些罕用內容會占用較多詞元。

對繁體中文而言，byte fallback 特別重要。第 6.4.3 節談的擴充區漢字、PUA 造字、族語文字，在沒有 byte fallback 的 tokenizer 中會全部變成 UNK——那等於把人名地名整個抹掉。

7.1.3 中文的特殊處境

英文有天然的空白邊界，所以 subword 切分是在「詞」的內部進行。中文沒有空白，所以 tokenizer 必須自己決定哪些字要黏在一起。這造成三個實務差異。

差異	英文	中文	後果
邊界資訊	空白提供強線索	完全沒有	切分完全由統計決定
基礎單位數	26 個字母	數千個常用漢字	基礎詞彙就吃掉大量空間
組合爆炸	字母組合有限	漢字組合近乎無限	難以涵蓋所有詞
位元組成本	1 字元 = 1 位元組	1 字元 = 3 位元組	byte 層級天然吃虧

第二列值得展開。一個 32K 的詞彙表，如果要涵蓋常用漢字（約 5000 字）加上標點、數字、英文字母與常見英文詞片，光是基礎單位就用掉了相當比例的空間。剩下的名額才能拿來存「臺北」「機器人」「智慧製造」這類多字詞。這是繁中 tokenizer 設計的核心限制。

7.2 BPE：從零實作

7.2.1 演算法

BPE (Byte Pair Encoding) 原本是資料壓縮演算法，被借來做 subword 切分。它的核心只有一句話：反覆找出最常一起出現的相鄰配對，把它們合併成一個新單位。

1. 把文字轉成最基礎的單位序列（本書用位元組，因此初始詞彙表為 256）。
2. 統計所有相鄰配對的出現次數。
3. 選出最高頻的配對，指派一個新的詞元編號，記錄這次合併。
4. 把序列中所有該配對替換成新編號。
5. 重複第 2 到 4 步，直到達到目標詞彙表大小。

訓練的產出是一份「合併規則表」，記錄了每次合併的配對與順序。編碼時按同樣的順序套用這些規則，就能重現訓練時的切法。

7.2.2 用一個小例子走一遍

假設語料是「臺灣科技大學智慧機器人學程，臺灣的智慧製造。臺灣科技大學。」，共 29 個字元、

87 個位元組。

階段	單位數	詞彙表大小	發生了什麼
初始	87 (位元組)	256	每個中文字是 3 個位元組
前幾次合併	約 58	約 262	常見漢字的三個位元組被合併成單一詞元
中間	約 40	約 272	「臺灣」「科技」等常見詞被合併
24 次合併後	31	280	「臺灣科技大學」幾乎成為單一單位

從 87 降到 31，壓縮率約 2.8 倍。注意這個過程的前幾步都在做同一件事：把三個位元組組成一個漢字。這是 byte-level BPE 在中文上必然要付出的「開場成本」——前幾百個合併規則幾乎都花在把位元組還原成漢字上。

這也解釋了一個常見的觀察：詞彙表很小的 byte-level tokenizer 在中文上表現極差，因為它連把位元組組成漢字的預算都不夠。詞彙表必須大到某個門檻，中文的效率才會開始改善——這個門檻你會在程式 7B 中親自量到。

7.2.3 手動追蹤一次合併

為了讓合併順序這件事變得具體，我們用一個純 ASCII 的迷你例子完整走一遍（用 ASCII 是因為中文的位元組會讓表格太長，但原理完全相同）。

語料：aaabdaaabc，初始詞彙表為位元組 a=97、b=98、c=99、d=100。

步驟	當前序列	最高頻配對	動作
初始	a a b d a a b a c	—	長度 11
合併 1	Z a b d Z a b a c	(a,a) 出現 4 次	aa → Z (新編號 256)
合併 2	Y b d Y b a c	(Z,a) 出現 2 次	Za → Y (新編號 257)
合併 3	X d X a c	(Y,b) 出現 2 次	Yb → X (新編號 258)

注意合併 2 那一步：此時 (Z,a)、(a,b)、(Z,a) 之後產生的配對頻次都是 2，出現了平手。BPE 必須有

一個確定性的決勝規則（例如取字典序最小、或取先出現者），否則同一份語料訓練兩次會得到不同的 tokenizer。這正是程式 7A 的「確定性」測試要驗證的事。

編碼時要按同樣的順序：先套用合併 1，再合併 2，再合併 3。如果順序顛倒，「aaab」可能被切成完全不同的樣子。這就是下一節那行 min 的意義——它永遠選出訓練時最早學到的那個配對。

7.2.4 完整實作

formosa/bpe.py — 迷你 BPE trainer 與 codec

```
from collections import Counter
from dataclasses import dataclass, field

def get_stats(ids: list[int]) -> Counter:
    """統計相鄰配對出現次數"""
    return Counter(zip(ids, ids[1:]))

def merge(ids: list[int], pair: tuple[int, int], new_id: int) -> list[int]:
    """把序列中所有 pair 替換成 new_id"""
    out, i = [], 0
    while i < len(ids):
        if i < len(ids) - 1 and ids[i] == pair[0] and ids[i + 1] == pair[1]:
            out.append(new_id); i += 2
        else:
            out.append(ids[i]); i += 1
    return out

@dataclass
class MiniBPE:
    merges: dict = field(default_factory=dict) # (a, b) -> new_id
    vocab: dict = field(default_factory=dict) # id -> bytes

    def train(self, text: str, vocab_size: int, min_freq: int = 2,
              verbose: bool = False):
        assert vocab_size >= 256, "位元組層級的詞彙表至少要 256"
        ids = list(text.encode("utf-8"))
        self.vocab = {i: bytes([i]) for i in range(256)}

        for k in range(vocab_size - 256):
            stats = get_stats(ids)
            if not stats:
                break
            pair, freq = stats.most_common(1)[0]
            if freq < min_freq: # 低頻配對不值得占一個名額
                break
            new_id = 256 + k
            ids = merge(ids, pair, new_id)
            self.merges[pair] = new_id
            self.vocab[new_id] = self.vocab[pair[0]] + self.vocab[pair[1]]
            if verbose and k % 500 == 0:
                token = self.vocab[new_id].decode("utf-8", errors="replace")
                print(f"合併 {k:5d}: {token!r} 頻次 {freq}")
        return self
```



```

def encode(self, text: str) -> list[int]:
    ids = list(text.encode("utf-8"))
    while len(ids) >= 2:
        stats = get_stats(ids)
        # 選出訓練時最早學到的那個配對（順序必須一致）
        pair = min(stats, key=lambda p: self.merges.get(p, float("inf")))
        if pair not in self.merges:
            break
        ids = merge(ids, pair, self.merges[pair])
    return ids

def decode(self, ids: list[int]) -> str:
    raw = b"".join(self.vocab[i] for i in ids)
    return raw.decode("utf-8", errors="replace")

```

encode 中那行 min 是整段程式最容易寫錯的地方。編碼時必須按照訓練時學到合併規則的順序來套用——先學到的先合併。如果順序不同，同一段文字會被切成不同的樣子，而模型會完全不認得。

一個必做的驗證

編解碼可逆性測試：對任意文字，decode(encode(text)) 必須等於原文。這個測試在中文上特別重要，因為 byte-level 的合併可能在漢字中間切開，若實作有誤就會產生亂碼。程式 7A 的第一個測試就是這個，請務必包含含有表情符號、罕用字與臺羅的測試案例。

7.2.4 三種主流演算法的差異

演算法	選擇合併的依據	特性	代表
BPE	配對出現頻率最高	簡單、快速、確定性	GPT 系列、Llama
WordPiece	合併後對語言模型概似的提升最大	更貼近語言模型目標	BERT
Unigram LM	從大詞彙表逐步刪除，保留概似損失最小者	可給出多種切法與機率	T5、多語模型

三者在最終效果上的差異，通常小於「詞彙表大小」與「訓練語料組成」帶來的差異。所以本書的建議是：不必糾結演算法選擇，把精力放在語料配比與詞彙表大小上——那才是你能真正做出差異的地方。

Unigram LM 有一個值得知道的特性：它能對同一段文字給出多種可能的切法及其機率。這被用在 subword regularization（訓練時隨機選用不同切法，作為一種資料增強），在低資源語言上偶爾有幫助。第 39 章談本土語言時會提到。

7.3 Pre-tokenization：切之前的切

7.3.1 為什麼需要它

如果直接把整份文件丟給 BPE，它可能會學出跨越詞界、跨越標點、甚至跨越句子的合併規則。例如「。臺灣」可能因為高頻而被合併成一個詞元——這顯然不是我們要的。

Pre-tokenization 是在 BPE 之前先做一次粗切，規定「哪些邊界不可跨越」。

規則	英文的常見做法	中文需要調整嗎	本書建議
空白切開	空白是天然邊界	中文沒有空白	保留規則，對中文無作用
標點獨立	標點不與詞合併	中文標點同理	必須做
數字分組	數字每 1–3 位切開	同樣適用	建議做，避免數字爆炸
大小寫	英文大小寫視為不同	中文無此問題	保留
CJK 邊界	不適用	漢字之間是否允許合併	允許，但限制最大長度

最後一列是中文專屬的設計決策。如果完全不限制，BPE 可能學出「臺灣科技大學智慧機器人學程」這種很長的單一詞元——它在你的語料中或許高頻，但那是過擬合，換一份語料就沒用了。實務上會限制單一詞元的最大字元數（例如 4 到 8 個漢字）。

7.3.2 數字的處理

數字是一個容易被忽略但影響顯著的細節。如果不做特別處理，BPE 會學出「2024」「2025」「114」這類特定數字的詞元，白白浪費名額。

策略	做法	優點	缺點
不處理	讓 BPE 自由學	高頻數字很省	浪費名額、罕見數字很貴
逐位切開	每個數字獨立成詞元	一致、可預測	長數字的序列很長
每 3 位一組	1,234,567 切成三組	折衷	需處理不足三位的情況
右對齊分組	從右往左每 3 位	符合位值直覺	實作稍複雜

對臺灣文本而言還有一個特殊考量：民國紀年。「114 年」中的 114 若被切成一個詞元，模型會把它

當成一個獨立符號而非數字；若逐位切開，模型比較有機會學到「114 比 113 大 1」這類關係。第 6.5.2 節建議保留原文並另加標準化欄位，正是為了讓模型同時看到兩種表示。

7.3.3 空白的表示

空白處理的三種慣例

```
# 慣例一：在詞前加特殊符號（SentencePiece 風格）
"Hello world" -> ["_Hello", "_world"]
# 好處：解碼時可完美還原空白位置

# 慣例二：把空白併入下一個詞元（GPT 風格）
"Hello world" -> ["Hello", " world"]

# 慣例三：空白獨立成詞元
"Hello world" -> ["Hello", " ", "world"]
# 缺點：序列變長

# 中文的情況：因為沒有空白，三種慣例對純中文差異不大
# 但中英混寫時會有差異：
"執行 SPC 分析" -> ["執行", "_SPC", "_分析"]      # 慣例一
"執行 SPC 分析" -> ["執行", " SPC", " 分析"]      # 慣例二
```

本書建議採用第一種或第二種，因為它們能保證解碼時完美還原空白。第三種在中英混寫的臺灣技術文件中會顯著增加序列長度——每一個中英交界處都多一個詞元。

7.4 特殊詞元與對話模板

7.4.1 基本的特殊詞元

詞元	用途	什麼時候需要	常見錯誤
BOS	序列開始	生成時提供起點	訓練與推論時用法不一致
EOS	序列結束	讓模型學會停止	忘了加，模型永遠不停
PAD	批次對齊	批次中序列長度不同時	沒有做 loss mask (第 5.2 節)
UNK	未知字	無 byte fallback 時	有 byte fallback 就不需要
MASK	遮罩位置	遮罩式訓練	自回歸模型不需要

第二列是初學者最常踩的坑。如果訓練資料的每個樣本結尾都沒有 EOS，模型就永遠學不到「這裡該停了」——生成時它會一直寫下去直到達到長度上限。你在第 13 章第一次生成文字時，如果發現模

型停不下來，先檢查這個。

7.4.2 對話模板

指令模型需要區分「誰在說話」。做法是用特殊詞元把對話結構標記出來，這套標記稱為對話模板（chat template）。

一個對話模板的結構

```
<|system|>
你是一位協助臺灣中學生的學習助理，回答需使用臺灣繁體中文。
<|end|>
<|user|>
請說明什麼是超額比序。
<|end|>
<|assistant|>
超額比序是指當某校報名人數超過招生名額時.....
<|end|>
```

對應的 loss mask（第 5.2.3 節）：
system 與 user 區段不計分，assistant 區段計分

模板的設計有三個必須注意的地方：

1. 模板詞元必須是不可分割的單一詞元，且不能出現在正常文本中。若使用者輸入的文字剛好包含 `<|user|>`，就可能造成提示注入——第 29 與 37 章會處理這個攻擊面。
2. 訓練時用什麼模板，推論時就必須用完全相同的模板。差一個換行都可能讓模型表現大幅下降。
3. 模板要在 tokenizer 訓練時就保留名額，或在之後以「加入特殊詞元」的方式擴充，並同步調整嵌入層大小。

7.4.3 工具呼叫與結構化輸出的標記

現代模型需要能呼叫工具、輸出 JSON、標記思考過程。這些都需要專屬的特殊詞元。

用途	典型標記	功能	章節
工具呼叫	<code>< tool_call >...< end ></code>	標示要執行的函式與參數	第 29、32 章
工具回傳	<code>< tool_result >...< end ></code>	把外部結果餵回模型	第 32 章
思考過程	<code>< think >...</think ></code>	區分內部推理與最終回答	第 27 章
引用來源	<code>< cite >...< end ></code>	標示答案的出處	第 30 章

本書的建議是：在訓練 tokenizer 時預留一批未使用的特殊詞元（例如 64 個），以備日後需要。事

後才要加特殊詞元的話，你必須擴充嵌入層並重新初始化那些位置——雖然做得到（7.6 節會講），但預留名額簡單得多。

7.5 量測：fertility、壓縮率與覆蓋率

7.5.1 三個核心指標

要比較 tokenizer 的好壞，必須先有可量測的指標。以下三個是本書的標準組合，缺一不可。

指標	定義	越高越好？	意義
token fertility	詞元數 ÷ 字元數	越低越好	每個字元平均花掉多少詞元
壓縮率	位元組數 ÷ 詞元數	越高越好	一個詞元平均裝了多少位元組
詞彙覆蓋率	被詞彙表直接涵蓋的字元比例	越高越好	有多少內容不必退回 byte

fertility 是本章最重要的指標，因為它直接對應成本。fertility = 1.0 表示平均每個字元一個詞元；fertility = 2.5 表示每個字元要花 2.5 個詞元——你的成本就是前者的 2.5 倍。

一個參考範圍：英文在多數 tokenizer 上的 fertility 約為 0.25 到 0.3（因為一個詞元通常涵蓋三四個字母）。繁體中文則從 0.9（繁中優化的 tokenizer）到 2.5（英文為主的 tokenizer）不等。這個落差就是 token 稅的來源。

三個指標的計算

```
def tokenizer_metrics(tok, texts: list[str]) -> dict:
    total_tokens = total_chars = total_bytes = 0
    fallback_tokens = 0

    for t in texts:
        ids = tok.encode(t)
        total_tokens += len(ids)
        total_chars += len(t)
        total_bytes += len(t.encode("utf-8"))
        # 退回 byte 的詞元 (id < 256 表示是單一位元組)
        fallback_tokens += sum(1 for i in ids if i < 256)

    return {
        "fertility": total_tokens / total_chars,
        "compression": total_bytes / total_tokens,
        "byte_fallback_ratio": fallback_tokens / total_tokens,
        "chars_per_token": total_chars / total_tokens,
        "n_texts": len(texts),
        "n_chars": total_chars,
    }
```

7.5.2 分域 fertility：本章的核心量測

一個總體的 fertility 數字會掩蓋巨大的領域差異。用第 5.4.4 節建立的五個分域驗證集，分別量測，你會看到完全不同的圖像。

領域	A：英文為主	B：多語通用	C：繁中優化	解讀
英文技術文件	約 0.26	約 0.28	約 0.35	C 犧牲了英文效率
繁中教科書	約 2.4	約 1.4	約 0.9	差距達 2.7 倍
公文與法規	約 2.6	約 1.5	約 0.9	專有名詞多，差距更大
製造技術文件	約 2.2	約 1.5	約 1.1	中英混寫，兩邊都不討好
臺羅與族語	約 3.5	約 3.0	約 2.8	所有 tokenizer 都很差

(上表為示意用的典型量級，實際數字必須由程式 7B 在你自己的語料與 tokenizer 上量出。本書要求所有引用的 fertility 數字都必須附上量測時間、tokenizer 版本與語料版本。)

最後一列值得特別注意：臺羅與族語在所有 tokenizer 上的 fertility 都極高，因為那些帶調字母幾乎不出現在任何訓練語料中，只能一個位元組一個位元組地退回。這意味著本土語言的使用者付出的 token 稅是最重的——這是第 39 章要處理的問題之一，而它的根源就在 tokenizer。

7.5.3 fertility 之外：切得好不好

fertility 低不代表切得好。一個把「臺灣科技大學智慧機器人學程」整段合併成單一詞元的 tokenizer，fertility 會非常低，但它是過擬合——換一份語料就完全沒用。

品質面向	好的表現	不好的表現	如何檢查
語意邊界	切在詞與詞之間	切在詞的中間	人工檢視高頻詞的切法
一致性	同一個詞總是切成一樣	在不同語境切法不同	對同一詞在多種上下文測試
泛化	在未見語料上 fertility 相近	只在訓練語料上很低	用保留集量測
公平性	各領域 fertility 差距合理	某些領域極差	分域量測

第三列的檢查方法特別重要：一定要用「訓練 tokenizer 時沒看過的語料」來量 fertility。用訓練語料量出來的數字必然偏低，那是自欺。這與第 5 章講的資料切分是同一個道理——tokenizer 也會過擬合。

7.5.4 看進詞彙表裡面

除了量化指標，還有一件必做的事：實際打開詞彙表看看裡面有什麼。這比任何數字都能快速發現問題。

詞彙表的組成分析

```
import unicodedata as ud
from collections import Counter

def classify_token(piece: bytes) -> str:
    try:
        s = piece.decode("utf-8")
    except UnicodeDecodeError:
        return "partial_bytes"          # 不完整的多位元組片段
    if not s:
        return "empty"
    cats = set()
    for ch in s:
        cp = ord(ch)
        if 0x4E00 <= cp <= 0x9FFF: cats.add("cjk")
        elif ch.isascii() and ch.isalpha(): cats.add("latin")
        elif ch.isdigit(): cats.add("digit")
        elif ch.isspace(): cats.add("space")
        else: cats.add("symbol")
    return "mixed" if len(cats) > 1 else cats.pop()

def vocab_profile(tok) -> dict:
    cnt = Counter(classify_token(p) for p in tok.vocab.values())
    total = sum(cnt.values())
    # 中文詞元的字數分布：1 字詞元 vs 多字詞元
    cjk_len = Counter()
    for p in tok.vocab.values():
        try:
            s = p.decode("utf-8")
        except UnicodeDecodeError:
            continue
        if s and all(0x4E00 <= ord(c) <= 0x9FFF for c in s):
            cjk_len[len(s)] += 1
    return {"by_type": {k: round(v / total, 4) for k, v in cnt.items()},
            "cjk_token_lengths": dict(sorted(cjk_len.items()))}
```

cjk_token_lengths 這個欄位特別有診斷價值。一個健康的繁中 tokenizer，應該有一批單字詞元（涵蓋常用漢字）、一大批雙字詞元（「臺灣」「學生」「政府」）、少量三四字詞元（「智慧製造」「教育部」），以及幾乎沒有超過六字的詞元。

如果你看到大量的六字以上詞元，那多半是 7.3.1 節說的過擬合——tokenizer 把訓練語料中反覆出

現的長句記了下來。如果單字詞元太少，那代表常用漢字沒被完整涵蓋，罕用字會頻繁退回 byte。這兩種問題用 fertility 都看不出來，只能靠打開來看。

7.6 從零訓練、詞彙擴充與嵌入初始化

7.6.1 三條路線的選擇

路線	做法	適用	代價
沿用既有 tokenizer	直接用底模附帶的	做 LoRA 或提示工程	繁中 fertility 可能很差
詞彙擴充	在既有詞彙表上加入繁中詞	持續預訓練 (第 22 章)	需重新初始化新增的嵌入
從零訓練	完全自訓 tokenizer	從零預訓練 (第 13 章)	無法直接使用他人權重

對臺灣的多數專案而言，第二條路是最務實的：在一個能力不錯的開放權重模型上，加入繁中的常用詞與領域詞，然後做持續預訓練。這樣既能繼承底模的能力，又能改善繁中效率。

但要注意一個重要限制：詞彙擴充之後，模型必須經過足夠的訓練才能用好新的詞元。剛加進去的詞元對模型而言是全新的符號，它不知道那是什麼意思。如果只擴充詞彙卻不訓練，效果會比不擴充還差。

7.6.2 嵌入初始化：三種策略

新增的詞元需要新的嵌入向量。怎麼初始化，會顯著影響收斂速度。

策略	做法	效果	建議
隨機初始化	從常態分布取樣	收斂最慢	不建議
平均初始化	取所有既有嵌入的平均	比隨機好	可接受的下限
子詞平均	用該詞在舊 tokenizer 下的切法，取其嵌入平均	收斂最快	本書建議
多語對應	用對應英文詞的嵌入	在翻譯任務上有效	需對照表

子詞平均初始化的實作

```
import torch
```



```

def expand_vocab(model, old_tok, new_tokens: list[str]):
    """在既有模型上加入新詞元，用子詞平均初始化"""
    emb = model.get_input_embeddings()
    old_size, d = emb.weight.shape
    new_size = old_size + len(new_tokens)

    model.resize_token_embeddings(new_size)
    emb = model.get_input_embeddings()

    with torch.no_grad():
        for i, tok_str in enumerate(new_tokens):
            # 用舊 tokenizer 把新詞切成子詞
            sub_ids = old_tok.encode(tok_str)
            if sub_ids:
                # 取這些子詞嵌入的平均作為新詞的起點
                emb.weight[old_size + i] = \
                    emb.weight[sub_ids].mean(dim=0)
            else:
                emb.weight[old_size + i] = emb.weight[:old_size].mean(dim=0)

    # 輸出層（若未與嵌入層共享權重）需同樣處理
    head = model.get_output_embeddings()
    if head is not None and head.weight is not emb.weight:
        with torch.no_grad():
            for i, tok_str in enumerate(new_tokens):
                sub_ids = old_tok.encode(tok_str)
                head.weight[old_size + i] = head.weight[sub_ids].mean(dim=0)
    return model

```

子詞平均的直覺很簡單：新詞元「智慧製造」在舊 tokenizer 下可能被切成「智」「慧」「製」「造」四個詞元。這四個詞元的嵌入平均，已經是一個相當合理的「智慧製造」的起點——至少比隨機向量好太多。第 22 章的程式 22A 會用這個函式並量測收斂速度差異。

7.6.3 該加哪些詞

詞彙擴充的名額有限（每加一個詞就多兩份參數），所以要挑效益最高的。挑選原則是：高頻、且在舊 tokenizer 下被切得很碎。

擴充候選詞的挑選

```

from collections import Counter

def pick_expansion_candidates(texts, old_tok, top_k=8000,
                              min_freq=50, min_split=3):
    """找出高頻且在舊 tokenizer 下被切得很碎的詞"""
    # 這裡假設已有候選詞表（可來自斷詞、詞典或 n-gram 統計）
    freq = Counter()
    for t in texts:
        for w in candidate_words(t):          # 由使用者提供的候選產生器
            freq[w] += 1

```

```

scored = []
for w, f in freq.items():
    if f < min_freq:
        continue
    n_split = len(old_tok.encode(w))
    if n_split < min_split:          # 本來就切得不錯，不必加
        continue
    # 效益 = 頻次 × 可節省的詞元數
    saving = f * (n_split - 1)
    scored.append((saving, w, f, n_split))

scored.sort(reverse=True)
return scored[:top_k]

```

請注意 `saving` 的計算：加入一個詞元後，它從 `n_split` 個詞元變成 1 個，每次出現省下 `n_split - 1` 個。乘上頻次就是總節省量。按這個分數排序，你會發現最值得加的往往是「臺灣」「政府」「學生」這類高頻詞，以及你的領域專有名詞。

7.6.4 訓練 tokenizer 的語料配比

一個常被忽略的事實：tokenizer 的訓練語料與模型的訓練語料不必相同，而且通常不應該相同。

tokenizer 學的是「什麼字串常一起出現」，這件事只需要相對少量但有代表性的資料就能學好——數百 MB 通常就足夠。真正重要的不是量，而是配比：因為配比直接決定了詞彙表的名額分配給誰。

語料組成	若繁中占 90%	若繁中占 50%	後果
繁中詞彙名額	多	少	影響繁中 fertility
英文詞彙名額	少	多	影響技術文件處理
程式碼符號	幾乎沒有	有	影響第 34 章的程式任務
本土語言	需刻意加入	幾乎沒有	臺羅與族語會全部退回 byte

最後一列點出一個重要的操作：如果你希望臺羅或族語能被合理地切分，就必須在 tokenizer 的訓練語料中刻意提高它們的比例——遠高於它們在真實語料中的自然比例。這叫做上取樣 (upsampling)，是第 9 章語料配比的一個特例。

一個實務建議的起點配比（實際數字應由程式 7B 的實驗決定）：繁體中文 60 到 70%、英文 15 到 20%、程式碼 5 到 10%、本土語言 3 到 5%、其他語言與符號 5%。請注意本土語言的 3 到 5% 遠高於它在網路文本中的自然比例——那是刻意的設計選擇，而不是資料的自然分布。

tokenizer 的訓練配比與模型的訓練配比是兩件不同的事，可以（也常常應該）不同。tokenizer 只是決定「怎麼切」，把本土語言上取樣不會讓模型的本土語言能力變好——那需要第 9 章的模型語料配比與第 22 章的適配訓練。但如果 tokenizer 切不好，後面再怎麼訓練都事倍功半。

7.7 token 稅：把 fertility 換算成錢

7.7.1 三條成本傳導路徑

fertility 高不只是一個技術指標，它會沿著三條路徑轉成實際的成本。

路徑	機制	影響	章節
API 費用	按詞元計價	直接等比例增加	第 33 章
上下文容量	上下文以詞元計算	能放入的實質內容變少	第 20、30 章
訓練與推論成本	計算量正比於序列長度	GPU 小時增加	第 17、35 章

7.7.2 一個完整的試算

情境：某校務問答服務，每日 5,000 次查詢，每次提示（含檢索片段）約 1,800 個詞元、回覆約 400 個詞元。假設輸入單價每千詞元 0.9 元、輸出 3.6 元（新臺幣）。

項目	現況	換用高效 tokenizer	差異
每次輸入詞元	1,800	1,170（減少 35%）	-630
每次輸出詞元	400	260	-140
每次成本	約 3.06 元	約 1.99 元	-35%
每月成本	約 45.9 萬元	約 29.8 萬元	每月省 16.1 萬
每年成本	約 558 萬元	約 363 萬元	每年省 195 萬

請注意這個節省完全來自 tokenizer，模型能力沒有任何改變。這就是為什麼本章不是一個可以跳過的技術細節——它是臺灣使用大模型時最大的一筆隱形成本。

7.7.3 上下文容量的損失

第二條路徑更隱蔽。假設模型的上下文長度是 32,000 個詞元：

fertility	可放入的中文字數	相當於	實務意義
0.9	約 35,500 字	一本小冊子	可放入整份法規
1.4	約 22,800 字	一篇長論文	需要取捨
2.3	約 13,900 字	幾份公文	長文件必須切塊

同樣的 32K 上下文，繁中使用者實際能放進去的内容可能只有英文使用者的四成。這在 RAG (第 30 章) 與長文件處理 (第 20 章) 中會直接限制你能做到的事——你必須切更多塊、做更多次檢索、承擔更多的資訊遺失。

7.7.4 訓練成本的傳導

第三條路徑影響的是自建模型的可行性。第 3.2.5 節給了粗估公式：訓練的 FLOPs 約為 $6 \times \text{參數量} \times \text{詞元數}$ 。如果同樣的語料在你的 tokenizer 下產生 1.5 倍的詞元，訓練成本就是 1.5 倍。

反過來說，改善 tokenizer 是少數「純賺」的優化：它不增加參數、不改變架構、不需要更多資料，只需要一次前處理的工作，就能讓每一次訓練與每一次推論都變便宜。第 17 章做算力預算時，tokenizer 效率會是一個關鍵的輸入參數。

一個決策原則

如果你的專案要長期運作（超過一年），且以繁體中文為主，那麼投入時間優化 tokenizer 幾乎一定划算——因為節省是累積的、每天發生的。如果只是短期原型，那就先用現成的，把時間花在驗證需求上。這個判斷與第 1.4 節的路線選擇是同一套邏輯。

7.8 臺灣專屬的 tokenizer 公平性測試

7.8.1 為什麼需要公平性測試

fertility 的總體數字會被高頻的常見文本主導。但一個實際的臺灣應用，經常要處理的是那些低頻但關鍵的詞：機關名稱、法規條號、晶片料號、機械規格、校名系名、原住民族地名。

如果這些詞在你的 tokenizer 下被切得極碎，那麼每一次涉及它們的查詢都會特別貴、特別容易出錯。公平性測試就是要把這件事量出來。

7.8.2 六類測試詞彙

類別	範例	為什麼重要	目標
行政區與地名	雲林縣、斗六市、豐濱鄉、達邦部落	任何在地應用都會用到	fertility 應接近一般中文
機關與校名	教育部國民及學前教育署、雲林科技大學	公文與校務應用的核心	不應被切得比一般詞碎
法規用語	第二十七條之一、但書、準用、廢止	法規檢索的關鍵詞	需完整保留
半導體與電子	曝光機、光阻、封裝測試、晶圓	臺灣的主力產業	領域專有名詞
機械與製造	五軸加工、公差、熱處理、伺服馬達	同上	同上
本土語言	Tâi-uân、ㄉㄤㄇㄣˊ、客語拼音、族語地名	文化保存的技術基礎	至少要能被表示

ch07/fairness_test.py (節錄)

```

import json
from pathlib import Path

def fairness_report(tok, wordlist_path: Path) -> dict:
    """wordlist.jsonl 每行: {"category": "...", "word": "...}"""
    by_cat = {}
    for line in wordlist_path.open(encoding="utf-8"):
        item = json.loads(line)
        w, cat = item["word"], item["category"]
        ids = tok.encode(w)
        by_cat.setdefault(cat, []).append({
            "word": w,
            "n_tokens": len(ids),
            "n_chars": len(w),
            "fertility": len(ids) / len(w),
            "pieces": [tok.decode([i]) for i in ids],
            "has_fallback": any(i < 256 for i in ids),
        })

    summary = {}
    for cat, rows in by_cat.items():
        f = [r["fertility"] for r in rows]
        worst = max(rows, key=lambda r: r["fertility"])
        summary[cat] = {
            "n": len(rows),
            "mean_fertility": round(sum(f) / len(f), 3),
            "max_fertility": round(max(f), 3),
            "fallback_rate": round(
                sum(r["has_fallback"] for r in rows) / len(rows), 3),
            "worst_word": worst["word"],
            "worst_pieces": worst["pieces"],
        }
    return summary

```

worst_pieces 這個欄位是報告中最有說服力的部分：它直接顯示某個重要的詞被切成了什麼樣子。當你看到某個機關全稱被切成十幾塊碎片，那個畫面比任何數字都有說服力。

7.8.3 判讀與行動

觀察	意義	行動	章節
某類別 fertility 特別高	該領域詞彙未被涵蓋	加入詞彙擴充候選	本章 7.6.3 節
fallback_rate 高	有字元不在詞彙表中	確認是否為罕用字或族語	第 6.4.3 節
最差詞被切成碎片	該詞在語料中頻率不足	評估是否值得加入	本章 7.6.3 節
各類別差距大	tokenizer 對某些領域不友善	調整訓練語料配比	第 9 章

這份公平性報告應該成為你選擇 tokenizer 的主要依據之一，而不是只看總體 fertility。一個總體數字漂亮但把所有機關名稱都切碎的 tokenizer，在校務或公文應用中是不合格的。

7.8.4 tokenizer 選型決策表

把本章所有的量測整合成一張決策表。實務上你面對的通常不是「從零設計」，而是「從幾個現成選項中選一個」。

你的情境	主要考量	該量什麼	傾向的選擇
做提示工程或 RAG	API 成本與上下文容量	分域 fertility、公平性	選繁中 fertility 低的模型
做 LoRA 微調	沿用底模的 tokenizer	公平性、fallback 率	接受既有，但要知道代價
做持續預訓練	長期效率	fertility、擴充候選詞的節省量	詞彙擴充
從零預訓練	完全自主	全部指標 + 參數成本曲線	自訓，本章的系統案例
邊緣部署小模型	參數量與序列長度都要小	嵌入層占比、fertility	中等詞彙表，需實測取捨

倒數第二列值得展開：從零預訓練是唯一能完全自主決定 `tokenizer` 的路線，但它也是成本最高的（第 1.4 節）。所以現實中最常見的組合是「開放權重底模 + 詞彙擴充 + 持續預訓練」——這既能繼承別人的能力，又能改善繁中效率，正是第 22 章的主線。

最後一列則是臺灣產業最關心的情境。邊緣裝置上，嵌入層的參數占比會很高（一個 0.5B 模型配 128K 詞彙表，光嵌入就可能占兩成以上），所以不能只看 `fertility`，必須把參數量一起放進取捨。第 26 章會把這個取捨完整展開。

程式實作

本章三個實作都在 A 級硬體上完成。7A 只用純 Python，7B 需要一份數十 MB 的語料，7C 是一個決策工具。三者的產出會被第 9、13、22 章直接使用。

程式 7A 迷你 BPE : trainer、encoder 與 decoder

目標：不使用任何 tokenizer 函式庫，從零實作 BPE 並驗證編解碼可逆。這是本書「先手寫一次再用函式庫」原則的第二次實踐（第一次是第 3 章的 softmax）。

必須通過的五個測試：

測試	內容	為什麼重要	難度
可逆性	<code>decode(encode(x)) == x</code>	最基本的正確性	低
中文完整性	漢字不被切在位元組中間	byte-level 的核心風險	中
特殊字元	表情符號、臺羅、罕用字皆可還原	涵蓋第 6 章的所有陷阱	中
順序一致性	編碼順序與訓練時的合併順序相同	最容易寫錯的地方	高
確定性	同一段文字每次編碼結果相同	可重現性的前提	低

tests/test_bpe.py (部分)

```
import pytest
from formosa.bpe import MiniBPE

CORPUS = open("data/tw_sample.txt", encoding="utf-8").read()

HARD_CASES = [
    "臺北市府教育局",
    "M8×1.25P 公差 ±0.02mm",
    "Tâi-uân ê bûn-hòa",
    "ㄊㄞˋ ㄙㄣˋ ㄨㄣˊ",
    "👨 家庭",
    "第二十七條之一",
    "執行 SPC 分析並更新 CIM 系統",
]

@pytest.fixture(scope="module")
def tok():
    return MiniBPE().train(CORPUS, vocab_size=4096)
```



```

@pytest.mark.parametrize("text", HARD_CASES)
def test_roundtrip(tok, text):
    assert tok.decode(tok.encode(text)) == text

def test_deterministic(tok):
    s = "臺灣科技大學智慧機器人學程"
    assert tok.encode(s) == tok.encode(s)

def test_no_broken_cjk(tok):
    """每個詞元解碼後不應含有替換字元"""
    s = "臺灣的智慧製造產業"
    for i in tok.encode(s):
        piece = tok.vocab[i]
        # 允許單一位元組 (byte fallback)，但多位元組詞元必須是合法 UTF-8
        if len(piece) > 1:
            piece.decode("utf-8") # 若非法會拋出例外

```

延伸挑戰：加入 `min_freq` 參數並掃描它的影響。你會發現設得太低時，詞彙表裡會塞滿只出現兩三次的長字串——那是過擬合，在未見語料上完全用不到。

程式 7B 三種詞彙表大小的比較實驗

目標：在同一份臺灣語料上訓練 8K、16K、32K 三種詞彙表，比較 fertility、覆蓋率、速度與參數成本。這是本章的核心實驗，結果會直接決定你在第 13 章的設定。

詞彙表	嵌入參數 (d=512)	預期繁中 fertility	預期優點	預期缺點
8K	約 4.2 M	較高	模型小、訓練快	中文切得碎
16K	約 8.4 M	中等	平衡	—
32K	約 16.8 M	較低	序列短、推論快	嵌入層占比高

(表中參數量為輸入嵌入層，若輸出層不共享權重則需乘二。實際的 fertility 必須由實驗量出。)

ch07/lab7b_vocab_sweep.py (主流程)

```

import json, time
from pathlib import Path
from formosa.bpe import MiniBPE
from formosa.metrics import tokenizer_metrics

DOMAINS = ["gov", "textbook", "social", "manufacturing", "local_lang"]

```

```
def main(train_path: Path, eval_dir: Path, sizes=(8192, 16384, 32768)):
    train_text = train_path.read_text(encoding="utf-8")
    # 重要：評測語料必須是 tokenizer 訓練時沒看過的（見 7.5.3 節）
    evals = {d: (eval_dir / f"{d}.txt").read_text(encoding="utf-8")
              .split("\n\n") for d in DOMAINS}

    rows = []
    for V in sizes:
        t0 = time.perf_counter()
        tok = MiniBPE().train(train_text, vocab_size=V)
        train_sec = time.perf_counter() - t0

        for domain, texts in evals.items():
            m = tokenizer_metrics(tok, texts)
            rows.append({"vocab_size": V, "domain": domain,
                        "train_seconds": round(train_sec, 1),
                        "emb_params_d512": V * 512,
                        **{k: round(v, 4) if isinstance(v, float) else v
                           for k, v in m.items()}})
            print(f"V={V:<6} {domain:<14} "
                  f"fertility={m['fertility']:.3f} "
                  f"fallback={m['byte_fallback_ratio']:.3f}")

    Path("outputs/vocab_sweep.json").write_text(
        json.dumps(rows, ensure_ascii=False, indent=2), encoding="utf-8")
```

必須回答的四個問題（寫進報告）：

- fertility 隨詞彙表增大而下降的曲線是什麼形狀？它在哪裡開始趨緩？那個轉折點就是你的甜蜜點。
- 五個領域的改善幅度是否一致？哪個領域從擴大詞彙表中獲益最多、哪個最少？為什麼？
- 把嵌入參數的增加與 fertility 的下降放在一起看，哪個詞彙表大小的總體效益最好？請定義你的「效益」並說明理由。
- 在訓練語料上量的 fertility 與在保留語料上量的差多少？差距說明了什麼？

程式 7C token 稅計算器

目標：把 fertility 換算成三種實際成本——API 費用、上下文容量、訓練 GPU 小時。這個工具會在第 17 章的算力預算與第 33 章的應用成本估算中被直接引用。

formosa/token_tax.py (節錄)

```
from dataclasses import dataclass

@dataclass
class TokenTax:
    fertility: float          # 詞元 ÷ 字元，由程式 7B 量得
    price_in_per_1k: float    # 輸入單價（新臺幣）
    price_out_per_1k: float

    def api_cost(self, in_chars: int, out_chars: int,
```

```

        calls_per_day: int, days: int = 30) -> dict:
    tin = in_chars * self.fertility
    tout = out_chars * self.fertility
    per_call = (tin / 1000 * self.price_in_per_1k
                + tout / 1000 * self.price_out_per_1k)
    return {"tokens_in": round(tin), "tokens_out": round(tout),
            "per_call": round(per_call, 4),
            "period_total": round(per_call * calls_per_day * days)}

def context_capacity(self, ctx_tokens: int) -> int:
    """這個上下文長度實際能放入多少中文字"""
    return int(ctx_tokens / self.fertility)

def training_flops(self, n_params: int, corpus_chars: int,
                   epochs: int = 1) -> dict:
    """訓練 FLOPs 約為 6 × 參數量 × 詞元數（見第 3.2.5 節）"""
    tokens = corpus_chars * self.fertility * epochs
    flops = 6 * n_params * tokens
    return {"tokens": round(tokens), "flops": flops,
            "petaflops": round(flops / 1e15, 2)}

def compare(self, other: "TokenTax", **kw) -> dict:
    """兩個 tokenizer 的成本比較"""
    a, b = self.api_cost(**kw), other.api_cost(**kw)
    saved = a["period_total"] - b["period_total"]
    return {"baseline": a["period_total"], "improved": b["period_total"],
            "saved": saved,
            "saved_pct": round(100 * saved / a["period_total"], 1)}

```

延伸挑戰：把 GPU 小時也算進去。給定一張 GPU 的實際吞吐量（用第 4 章程式 4B 量得的 TFLOPS 乘上一個現實的 MFU，例如 0.35），把 training_flops 換算成小時，再乘上電費或雲端租金。這樣你就得到了一個完整的「從 fertility 到新臺幣」的換算鏈。

系統案例：為 MiniFormosaGPT 設計詞彙表

本章的系統案例是為第 13 章要建構的 MiniFormosaGPT 設計 tokenizer。這不是一個練習——你在這裡做的決定，會決定第 13 章之後所有實驗的效率上限。

設計需求

需求	具體要求	衝突之處	如何取捨
繁體中文效率	fertility 盡量低	會擠壓其他語言的名額	依應用場景權衡
英文可用	技術詞彙不被切碎	與上一項競爭名額	保留基本英文詞片
程式碼支援	常見符號與關鍵字	符號組合多	至少保留縮排與括號
臺羅可表示	帶調字母不變成一堆 byte	低頻，難以爭取名額	至少保證可還原

需求	具體要求	衝突之處	如何取捨
料號與型號	不被切成無意義碎片	組合近乎無限	接受一定程度的碎切
特殊詞元預留	對話模板與工具呼叫	占用名額	預留 64 個

必須交付的六項

1. tokenizer 檔案：可載入、可編解碼、通過程式 7A 的五個測試。
2. 詞彙表統計：各類字元的占比（漢字、英文、數字、符號、特殊詞元）。
3. 特殊詞元規格：完整清單、用途、以及對話模板的確切格式（含換行位置）。
4. 分域 fertility 報告：五個領域的量測結果，附量測時間與語料版本。
5. 公平性報告：六類詞彙的測試結果，含 worst_pieces 的實例。
6. 設計決策紀錄：每一個決定（詞彙表大小、語料配比、數字處理、空白慣例、最大詞元長度）的理由與被否決的替代方案。

評分重點

- 設計決策紀錄的品質。本案例的核心不是做出「最好」的 tokenizer，而是能說清楚每個取捨。一個 fertility 稍差但理由充分的設計，優於一個數字漂亮但說不出所以然的設計。
- 是否用保留語料量測。用訓練語料量出的漂亮數字不予採計。
- 公平性報告是否誠實。如果報告說所有類別都很好，助教會拿刁鑽的詞來測。
- 對話模板是否精確到換行層級。這一項在第 23 章會被實際使用，格式不精確會直接導致微調失敗。

章末評量

一、概念題

1. 比較五種切分粒度的取捨，並說明為什麼 byte fallback 對繁體中文特別重要。
2. 用自己的話描述 BPE 的訓練與編碼流程，並說明為什麼編碼時必須遵守訓練時的合併順序。
3. 為什麼 byte-level BPE 在中文上有「開場成本」？這對詞彙表大小的選擇有什麼意涵？
4. 什麼是 pre-tokenization？為什麼需要限制單一詞元的最大長度？
5. 為什麼 fertility 低不代表 tokenizer 好？請提出三種其他的品質檢查。
6. 詞彙擴充後為什麼必須繼續訓練？只擴充不訓練會發生什麼？

二、計算題

1. 一個詞彙表 32K、 $d = 768$ 的模型，輸入嵌入與輸出投影不共享權重。計算這兩層合計的參數量。若詞彙表改為 128K，增加多少？
2. 某 tokenizer 在繁中的 fertility 為 2.3。一份 10,000 字的文件需要多少詞元？若模型上下文為 32K 詞元，最多能放入多少字？
3. 承上題，若改用 fertility 為 0.9 的 tokenizer，同樣的 32K 上下文能放入多少字？增加了多少倍？
4. 某服務每日 5,000 次呼叫，每次輸入 1,800 詞元、輸出 400 詞元，輸入單價每千詞元 0.9 元、輸出 3.6 元。計算月成本。若 tokenizer 改善使詞元數減少 35%，月成本變成多少？
5. 一份 5 億字的繁中語料，用 fertility 1.4 的 tokenizer 處理後有多少詞元？若要訓練一個 1B 參數的模型跑一個 epoch，需要多少 FLOPs？（用 6ND 公式）

三、除錯題

1. 同學的 BPE 在英文上運作正常，但中文解碼後出現替換字元。請指出最可能的兩個原因與檢查方法。
2. 同學的 tokenizer 在訓練語料上 fertility 是 0.85，但在新文件上是 1.6。請解釋這個落差，並說明應該怎麼修正實驗設計。
3. 同學做了詞彙擴充，但微調後模型的表現反而比擴充前差。請列出三個可能原因。
4. 同學的模型生成時永遠不會停止，一直寫到長度上限。請說明最可能的原因與檢查方式。
5. 同學發現同一段文字用他的 tokenizer 編碼兩次得到不同結果。請指出最可能的原因（提示：與 7.2.3 節的某一行有關）。

四、系統設計題

1. 你要為一個「智慧製造維修助理」選擇 tokenizer。文本包含中文說明、英文設備術語、料號（如 SKD11、M8×1.25P）與 G-code。請設計你的評估流程：測哪些指標、用哪些測試詞、如何決定詞彙表大小、以及在什麼情況下你會選擇詞彙擴充而非從零訓練。
2. 你的團隊要為一個臺語教學應用設計 tokenizer。臺羅文本的 fertility 極高（約 2.8），但語料量遠小於華語。請提出至少三種改善方案，並分析各自的代價與可行性。

五、臺灣情境與倫理題

1. 本章指出繁中使用者付出的 token 稅是英文使用者的兩倍以上，而本土語言使用者更高。請討論：這是技術的必然結果，還是可以改變的設計選擇？誰有能力改變它？誰應該負擔改變的成本？
2. 假設你發現某個廣泛使用的商業模型，其 tokenizer 對臺灣的機關名稱與法規用語切得特別碎，導致公部門使用時成本偏高。請說明你會如何量化這件事、向誰說明、以及有哪些可能的因應方案。
3. 有人主張「反正 tokenizer 都是既有的，我們配合就好」。請提出三個反駁，以及這個主張在什麼情況下是合理的。

評量提示（給自學者）

- 計算題第 1 題：32K 時為 $2 \times 32,000 \times 768 \approx 4,915$ 萬參數；128K 時為 $2 \times 128,000 \times 768 \approx 1.97$ 億；增加約 1.47 億。
- 計算題第 2 題： $10,000 \times 2.3 = 23,000$ 詞元； $32,000 \div 2.3 \approx 13,913$ 字。
- 計算題第 3 題： $32,000 \div 0.9 \approx 35,556$ 字，約為前者的 2.56 倍。
- 計算題第 4 題：每次成本 = $1.8 \times 0.9 + 0.4 \times 3.6 = 3.06$ 元；月成本 $3.06 \times 5,000 \times 30 \approx 45.9$ 萬元。減少 35% 後約 29.8 萬元。
- 計算題第 5 題：5 億 $\times 1.4 = 7$ 億詞元； $FLOPs = 6 \times 1e9 \times 7e8 = 4.2 \times 10^{18}$ ，即 4,200 PetaFLOPs。
- 除錯題第 5 題：encode 中選擇合併配對時，若用了不穩定的排序或字典迭代順序，同一輸入可能得到不同結果。應確保 min 的比較鍵是確定性的。

研究延伸與版本注意事項

- BPE 的原始論文（Sennrich 等人 2016，用於神經機器翻譯）與 GPT-2 的 byte-level BPE 實作，是本章的直接來源。前者篇幅短、論證清楚，適合精讀。
- SentencePiece 的設計文件說明了為什麼要做語言無關的 tokenizer，以及 Unigram LM 的訓練

方式。

- 多語 tokenizer 的公平性研究：近年有多篇論文量測不同語言的 fertility 差異與其成本意涵，這些研究對本章 7.7 節有直接補充。
- 詞彙擴充與嵌入初始化的實證研究：第 22 章會用到，屆時建議一併閱讀。
- 本章的演算法不會過時，但 tokenizer 函式庫的 API 變動較快。程式碼請以課程倉庫為準；引用任何模型的 fertility 數字時，務必註明模型版本與量測日期。

常見問題

問：既然有現成的 tokenizer 函式庫，為什麼要手寫？答：三個理由。第一，BPE 的編碼順序一致性是一個很容易寫錯、也很容易誤解的細節，寫過一次你就永遠不會搞混。第二，你需要能讀懂與修改別人的 tokenizer 設定，而那需要知道它在做什麼。第三，第 13 章你要從零訓練模型，屆時 tokenizer 是你自己的一部分，不是黑盒子。

問：詞彙表大小到底該選多少？答：沒有普世答案，取決於你的語料組成、模型大小與應用場景。但有兩個經驗法則：一是模型越小，詞彙表占參數的比例越高，應該選小一點；二是繁中為主的應用通常需要比純英文應用更大的詞彙表，因為漢字的基礎單位就很多。程式 7B 的目的就是讓你在自己的資料上找到答案。

問：我用的是別人的模型，不能改 tokenizer，這一章還有用嗎？答：非常有用。第一，你需要量測它在你的領域上的 fertility，才能正確估算成本與上下文容量。第二，7.8 節的公平性測試能幫你在選模型時做出更好的決策——兩個能力相近的模型，繁中 fertility 可能差一倍。第三，若你之後要做持續預訓練（第 22 章），詞彙擴充是最先要考慮的步驟之一。

問：把詞彙表加到很大不就好了？答：不行。詞彙表增大會讓嵌入層與輸出層的參數線性增加，而這些參數的訓練效率很低——低頻詞元的嵌入可能整個訓練過程只被更新幾次。此外輸出層的 softmax 計算量也會增加。實務上是一條有明顯報酬遞減的曲線，程式 7B 就是要你把它畫出來。

教師使用建議

本章排在第 7 週，建議 2 小時講授加 1 小時實驗。本章的教學重點是「讓學生對成本產生實感」——token 稅的試算通常是全書最能引起共鳴的一段。

時段	內容	互動設計	注意事項
前 20 分鐘	7.1 節，為什麼需要 tokenizer	現場把同一句中英文丟進幾個 tokenizer 比較	這個對比是全章的動機
中間 35 分鐘	7.2 至 7.3 節，BPE 與 pre-tokenization	用白板手動跑三次合併	合併順序必須讓學生親手做一次
後 25 分鐘	7.4 至 7.5 節，特殊詞元與量測	請學生預測五個領域的 fertility 排序	預測後再看數據效果最好
最後 40 分鐘	7.7 至 7.8 節，token 稅與公平性	程式 7C 現場算某個真實服務的年成本	這一段的震撼效果最強
實驗課 1 小時	程式 7A	兩人一組，互相出刁鑽的可逆性測試	7B 需較長時間，作為課後作業

一個效果極佳的課堂示範：拿一段臺灣的公文，用三個不同的 tokenizer 編碼，把切分結果並排投影出來。學生會直接看到「臺北市政府」在某個 tokenizer 下被切成七八塊碎片。這個畫面比任何數字都能說明問題。

高中授課的調整：7.1、7.2 完整教（BPE 的合併過程很直觀，適合白板演示）；7.3、7.4 挑重點講（EOS 忘了加會怎樣、對話模板長什麼樣）；7.5 只講 fertility 一個指標；7.7 的 token 稅試算務必保留，這是最能讓高中生理解「技術決策有社會後果」的一段；7.6、7.8 可略。程式 7A 對高中生完全可行，7C 只需要填數字。

本章小結

1. tokenizer 決定序列長度、詞彙表參數量、切分粒度與未知字處理，其影響貫穿成本、上下文容量與訓練效率。
2. 五種切分粒度中，subword 加 byte fallback 是現代主流；byte fallback 對繁體中文尤其重要，它避免了罕用字與族語文字變成 UNK。
3. 中文沒有天然邊界、基礎單位多、位元組成本高，這三點使繁中 tokenizer 的設計比英文困難。
4. BPE 反覆合併最高頻的相鄰配對；編碼時必須遵守訓練時的合併順序，否則模型完全不認得。
5. byte-level BPE 在中文上有「開場成本」：前幾百個合併規則幾乎都用在把位元組組成漢字。
6. pre-tokenization 規定不可跨越的邊界；中文需要限制單一詞元的最大長度，避免過擬合的長詞元。

7. EOS 忘了加會讓模型停不下來；對話模板必須在訓練與推論時完全一致，並預留特殊詞元名額。
8. fertility (詞元÷字元) 是最重要的指標，但必須配合壓縮率、覆蓋率與分域量測一起看，且必須用保留語料。
9. 詞彙擴充應優先加入「高頻且在舊 tokenizer 下被切得碎」的詞；嵌入用子詞平均初始化，收斂最快。
10. token 稅沿三條路徑傳導：API 費用、上下文容量、訓練與推論成本。改善 tokenizer 是少數「純賺」的優化。
11. 臺灣專屬的公平性測試涵蓋行政區、機關校名、法規用語、半導體、機械與本土語言六類；worst_pieces 是報告中最有說服力的部分。

成果檢核

完成本章後你必須繳交：

- MiniBPE 實作 (程式 7A)：通過五個測試，含中文、臺羅、表情符號、料號的可逆性驗證。
- 詞彙表比較報告 (程式 7B)：8K/16K/32K 三種大小、五個領域的 fertility 與覆蓋率，含曲線圖與甜蜜點的判斷理由。
- token 稅報告 (程式 7C)：把你的專題應用場景代入，算出年成本、上下文容量與訓練 FLOPs。
- MiniFormosaGPT 的 tokenizer：檔案、詞彙表統計、特殊詞元規格 (含對話模板的精確格式)。
- 臺灣 tokenizer 公平性報告：六類詞彙的測試結果，含至少五個 worst_pieces 的實例。
- 設計決策紀錄：每一個設計選擇的理由與被否決的替代方案。

這份 tokenizer 會被第 9 章 (語料統計)、第 13 章 (MiniFormosaGPT 訓練)、第 17 章 (算力預算)、第 22 章 (詞彙擴充) 與第 33 章 (應用成本) 直接使用。它是第二篇最具體的一項交付物——從這一章開始，你手上有了一個真正屬於你自己的、為臺灣文本設計的工具。

第 8 章

語料取得、授權、個資與資料治理

Corpus Acquisition, Licensing, Privacy, and Data Governance

難度：核心 | 建議授課：第 8 週 | 硬體需求：A 級 | 注意：本章法律內容僅供工程教學，實際判斷須經專業確認

叫獸開講

「網路上找得到」不等於「可以拿來訓練」。這一章不會給你法律意見——我不是律師，本書也不是法律教材——但它會讓你在被主管、被合作單位、被記者問到「這些資料哪裡來的」時，能立刻打開一份說得清楚的清冊，而不是支支吾吾地說「就.....網路上抓的」。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 從模型用途反推資料需求，包括語域、時間範圍、人口涵蓋與品質門檻。
2. 區分政府開放資料、公開授權、合作授權、自有資料與合成資料五種來源的特性與風險。
3. 辨識著作權、契約、網站使用條款與資料庫相關權利的風險點，並知道哪些判斷必須交給法務。
4. 說明個人資料的定義、敏感資料的特殊性、以及最小化、目的限定與保存期限三項原則。
5. 設計並實作繁體中文的個資偵測與遮罩管線，並理解去識別的極限。
6. 建立 provenance ledger，記錄來源、時間、版本、授權、處理與退出機制。
7. 撰寫 Data Card，並設計資料責任人、審批流程、存取權限與稽核軌跡。
8. 設計訓練、驗證、測試、紅隊與保留資料的隔離原則，並說明為什麼隔離同時是科學問題與治理問題。
9. 辨識校園場域的特殊風險，特別是學生作品、課堂錄音、輔導紀錄與未成年人資料。

一個必須先說清楚的界線

本章討論的法律概念僅供工程教學使用，目的是讓你知道「哪裡有風險、該問誰」，而不是讓你自行做出法律判斷。任何實際的資料使用決策，都必須經過所屬機構的法務、個資窗口或倫理審查程序。本章所有涉及法律的段落都會標示需要確認的事項；請把它們當成待辦清單，而不是結論。此外，法規與實務見解會隨時變動，尤其人工智慧訓練與著作權的關係在各國都仍在演進中，使用前務必查核最新狀態。

8.1 從用途反推資料需求

8.1.1 先問要做什麼，再問要什麼資料

學生做專題時最常見的順序是：先去抓一堆資料，再想能做什麼。這個順序會製造兩個問題——你抓了大量用不到的資料（卻承擔了它們的法律與個資風險），同時缺少真正需要的資料。

正確的順序是從第 1 章那份需求規格書出發，逐層往回推。

層次	要回答的問題	對資料的意涵	對應章節
使用者與任務	誰會用？問什麼？	決定語域與主題範圍	第 1 章
語言與用語	繁中？含臺語？中英混寫？	決定語言配比	第 6、7 章
時間範圍	知識要多新？會過期嗎？	決定更新頻率與時間切分	第 5.4.2 節
涵蓋人群	誰的說話方式要被理解？	決定代表性與偏誤風險	第 25、36 章
正確性要求	錯了會怎樣？	決定品質門檻與是否需 RAG	第 30 章
敏感程度	含個資嗎？營業秘密？	決定治理層級與部署位置	本章 8.4 節

8.1.2 一份資料需求規格

把上表填成一份可審查的文件。以下是校務問答助理的範例。

configs/data_requirements.yaml

```

purpose: 校務與課程問答助理
owner: 智慧科技學院 AI 實驗室
reviewed_by: [系主任, 個資窗口]      # 誰簽過字
review_date: 2026-08-20

domains:                                # 需要哪些語域
- name: 學則與規章
  priority: high
  freshness: 現行版本，每學期更新
  accuracy: 必須逐條可追溯
- name: 課程大綱
  priority: high
  freshness: 當學年
- name: 一般校園生活問答
  priority: medium
  freshness: 不敏感

languages:
zh-Hant: 0.85
en: 0.15

excluded:                                # 明確排除，這一欄比包含的更重要
- 學生個人資料與成績
- 輔導與健康紀錄

```

- 人事與薪資資料
- 未經授權的第三方教材

```
sensitivity: low # low / medium / high
deployment: on_premise
retention: 語料保存至專案結束後 1 年，屆時重新評估
```

excluded 那一欄是整份文件最重要的部分。明確寫下「我們不用什麼」，比列出「我們用什麼」更能保護你——因為它是一個可被檢驗的承諾，也是你在資料收集時的第一道過濾條件。

8.1.3 資料量要多少

這個問題沒有普世答案，但有可以推估的方法。關鍵是先確定你走哪條技術路線（第 1.4 節），因為不同路線的資料需求差了好幾個數量級。

技術路線	資料量級	品質要求	主要瓶頸
RAG	數百至數萬份文件	極高（會被直接引用）	文件的完整性與時效
指令微調	數百至數萬筆對話	極高（每一筆都影響行為）	人工撰寫與審核的成本
LoRA 領域適配	數十至數百 MB 文本	高	領域資料的取得管道
持續預訓練	數 GB 至數十 GB	中高	合法且有代表性的語料
從零預訓練	數百 GB 至 TB 級	中（靠去重與過濾補）	算力與語料同時受限

請注意品質要求那一欄的趨勢：資料量越少，每一筆的品質就越關鍵。一千筆精心撰寫的指令資料，效果可能勝過十萬筆隨便湊的——這在第 23 章會被實證。對臺灣的多數專案而言，這是好消息：我們缺的是規模，但不缺對本地情境的理解，而後者正是高品質小資料集的來源。

8.2 五種資料來源

8.2.1 來源類型與風險輪廓

來源	典型例子	主要優點	主要風險與待確認事項
政府開放資料	開放資料平台、法規資料庫、統計資料	授權條款通常明確	仍須逐案確認條款與再散布條件

來源	典型例子	主要優點	主要風險與待確認事項
公開授權內容	採用公眾授權條款釋出的文本	條款可查	不同授權對商用與衍生的規定不同
合作授權	學校、企業、出版社簽約提供	品質高、獨特性強	契約範圍、期限、終止後義務
自有資料	機構自己產生的文件	控制力最高	仍可能含個資與第三方權利
合成資料	用模型生成	可控、無直接個資	教師模型的授權限制、品質與偏誤

對臺灣的教育與中小企業專案而言，第三與第四類是最有價值的——因為它們是別人拿不到的差異化來源（第 1.6.5 節）。但它們也是治理要求最高的，因為往往涉及真實的人與真實的營業活動。

8.2.2 政府開放資料的實務注意事項

臺灣有相當豐富的政府開放資料，但「開放」不代表「無條件可用」。以下是幾個必須逐案確認的面向。

面向	要確認什麼	為什麼重要	誰能回答
授權條款	採用哪一種授權？允許商用嗎？	決定產出能否商業化	資料提供機關
再散布	能否把處理後的資料公開？	影響開源交付	同上
標示義務	是否須標示來源？如何標示？	合規的基本要求	同上
資料時效	多久更新一次？舊版是否保留？	影響知識時效與版本管理	同上
內含個資	公開資料中是否仍有可識別資訊	公開不等於可任意再利用	個資窗口 + 提供機關

最後一列特別容易被忽略。有些公開資料中包含人名（例如得獎名單、判決書、公告的承辦人），這些內容雖然已被合法公開，但把它們拿去訓練模型是另一個目的的處理行為，可能有不同的規範要求。這正是需要向個資窗口確認的典型情況。

8.2.3 網路爬取：最需要謹慎的一類

本書不鼓勵未經評估的大規模爬取。如果你的專案確實需要爬取公開網頁，以下是必須先處理的事項——注意這裡列的是「工程與倫理上的最低要求」，法律面仍須另行確認。

1. 閱讀目標網站的使用條款與 robots.txt，並記錄你在什麼時間看到了什麼版本。條款會變，紀錄是你的依據。
2. 尊重爬取速率限制，不要對他人的伺服器造成負擔。這是基本的網路禮儀，也可能涉及法律責任。
3. 記錄每一筆內容的來源網址、擷取時間與當時的授權宣告。沒有這些，你之後無法回答任何追問。
4. 建立排除清單：新聞付費內容、個人部落格、社群貼文、含大量個資的頁面，預設排除。
5. 提供退出管道：讓內容所有者能要求移除，並實際做得到（程式 8C）。
6. 在專案文件中誠實記載爬取的範圍與方法，不要含糊帶過。

一個判斷原則

如果你不敢在報告中寫清楚某份資料是怎麼來的，那就代表你不該用它。這條原則比任何法條都好記，也比任何法條都更能保護你。第 40 章的總整專題驗收時，資料來源的可揭露性是一票否決項。

8.2.4 合成資料的三個限制

用大模型生成訓練資料看起來很誘人：沒有個資、沒有爬取、量可以無限。但它有三個必須知道的限制。

限制	內容	後果	章節
教師模型的授權	多數服務條款限制用其輸出訓練競爭模型	可能違約，須逐條確認	第 26 章
知識的天花板	學生模型難以超越教師的知識邊界	教師不懂臺灣，學生也不會懂	第 9、26 章
分布退化	大量合成資料反覆訓練會使多樣性下降	模型輸出趨於同質、罕見情況失能	第 9 章

第二列對臺灣特別關鍵。如果你用一個不熟悉臺灣制度的模型來生成臺灣情境的訓練資料，它會生成大量看起來合理但實際錯誤的內容——然後你把這些錯誤訓練進自己的模型。這是一條會放大錯誤的路徑，第 9 章會給出具體的篩選機制。

8.2.5 取得資料的標準作業流程

把前面幾節整合成一條可執行的流程。這條流程的價值在於：它讓「先確認再取得」變成預設行為，而不是靠自律。

步驟	做什麼	產出	卡關時該找誰
1	確認這份資料符合需求規格的哪一項	對應到 domains 中的哪一列	專案負責人
2	找到授權宣告或契約條款，存檔快照	terms_snapshot 檔案	資料提供方
3	初步風險分級（綠黃橙紅）	risk_level 欄位	自行判斷，存疑則升級
4	黃級以上送確認	確認人與確認日期	法務、個資窗口
5	取得資料，記錄時間與方法	source 群組欄位	—
6	計算雜湊、統計內容組成	content 群組欄位	—
7	執行個資掃描	processing 中的 pii_scan	—
8	人工覆核掃描結果與隨機抽樣	pii_review 紀錄	覆核者（非執行者）
9	設定保存期限與退出管道	lifecycle 群組	專案負責人
10	通過程式 8A 檢查後才可進入清理管線	CI 通過紀錄	—

注意第三步與第四步的設計：初步分級由工程師自己做（因為他最了解資料內容），但只要不是綠級就必須送出去確認。這個設計平衡了效率與謹慎——不是每一份資料都要驚動法務，但任何存疑的都不能自行放行。

第十步是把治理接進技術流程的關鍵。如果 manifest 沒通過檢查，資料就進不了清理管線，也就進不了訓練。這比任何規定都有效，因為它不依賴任何人記得遵守。

8.3 智慧財產權的風險辨識

這一節的目的不是教你判斷合法與否，而是教你辨識「哪裡有風險、該問什麼問題」。所有實際判斷都必須交給有資格的人。

8.3.1 四種可能同時存在的權利

權利類型	可能涉及什麼	常見誤解	該問誰
著作權	文章、程式碼、圖片、教材的重製與改作	「有標示來源就沒問題」	法務、著作權專責機關資訊
契約與服務條款	網站條款、API 條款、資料提供契約	「網站沒明講就是可以」	法務
資料庫相關保護	對資料集整體的投入與編排	「單筆資料不受保護所以整份也不受」	法務
營業秘密	合作企業的製程、配方、客戶資料	「他們給我們就是同意公開」	合作方 + 法務

最後一列在產學合作中極為常見。企業提供技術文件給你做研究，不代表你可以把訓練出來的模型公開，也不代表你可以在論文中揭露那些內容。這些必須在合作契約中寫清楚，而且應該在專案開始前就談妥，不是等到要發表時才發現。

8.3.2 合理使用：一個必須謹慎對待的概念

許多學生聽過「教育目的可以主張合理使用」，然後就把它當成通行證。這是危險的誤解。

合理使用在我國著作權法中有明文規定，判斷時需綜合考量利用之目的與性質、著作之性質、所利用之質量占著作全體之比例、以及利用結果對著作潛在市場與現在價值之影響等因素。重點是：這是一個個案判斷，需要具體衡量，不是一個可以事先套用的通則。

對本書讀者而言，實務上的意義是三句話：第一，不要自行斷定某個利用構成合理使用；第二，教育或研究目的可能是有利因素之一，但不是唯一因素，也不保證結論；第三，用於訓練後又對外提供服務，性質與純粹的課堂使用不同，風險評估必須重做。

本書的處理方式

本書所有實作使用的語料，都以「授權明確可用」為前提，而非依賴合理使用主張。這不是因為合理使用不重要，而是因為在教學情境中，讓學生養成「先確認授權」的習慣，遠比教他們如何論證合理使用更有價值。若你的專題確實需要處理灰色地帶，請走機構的正式諮詢管道。

8.3.3 風險分級與處置

實務上你會遇到大量無法立刻判定的情況。與其卡住不動，不如建立分級機制，讓不同風險等級走不同的流程。

等級	判斷特徵	處置	例子
綠	授權明確且涵蓋你的用途	記錄後直接使用	明確允許商用與改作的開放資料
黃	授權存在但條款需解讀	送法務確認後再用	條款提及非商業用途但定義不明
橙	授權不明或無授權宣告	暫緩使用，嘗試聯繫取得授權	個人網站的技術文章
紅	明確禁止或高度敏感	不使用，並記錄為何排除	付費內容、明示禁止爬取者

關鍵是：橙色與紅色的資料也要記錄。記錄「我們看過這份資料但決定不用，理由是……」與記錄「我們用了這份資料」同樣重要——它證明你有做過評估，而不是碰巧沒抓到。

8.4 個人資料：定義、原則與實作

8.4.1 什麼算個人資料

個人資料的範圍比多數人以為的廣。它不限於姓名與身分證字號，而是包括所有「可以直接或間接識別到特定個人」的資訊。

類別	例子	為什麼算	偵測難度
直接識別	姓名、身分證字號、護照號碼	可直接指向個人	中（格式明確）
聯絡資訊	電話、地址、電子郵件、社群帳號	可聯繫到個人	中
準識別資訊	出生年月、職稱、所屬單位、班級座號	組合起來可鎖定個人	高
行為紀錄	借閱紀錄、上課出席、系統操作日誌	可推知個人狀態	高
特種資料	健康、醫療、基因、性生活、犯罪紀錄等	法律上有更嚴格的規範	高

第三列的準識別資訊是最容易被忽略的。「資工系三年級、住斗六、修過某某老師的課、外號小明」——每一項單獨看都不算個資，組合起來卻可能只對應到一個人。這在小規模社群（一個系、一間公司、一個村）中尤其成立，而臺灣的許多應用場景正是這種規模。

最後一列在我國個人資料保護法中屬於受特別規範的類型，其蒐集、處理與利用有更嚴格的要件。校

園場域中的輔導紀錄、健康檢查資料、特殊教育需求紀錄都可能落入此類，處理時必須格外謹慎，並務必經由學校的個資窗口確認。

8.4.2 三項應該內化的原則

1. 目的限定：資料是為了某個目的蒐集的，換一個目的使用（例如把校務系統的資料拿去訓練模型）通常需要重新檢視其合法性基礎。這是最常被違反的一項，因為工程師往往覺得「資料已經在我們手上了」。
2. 最小化：只蒐集與保留完成目的所必需的資料。多留的每一筆都是額外的風險，卻沒有額外的價值。
3. 保存期限：資料不是永久保存的。應該在一開始就決定保存多久、到期後怎麼處理，並且真的做得到。

這三項在工程上的具體對應是：目的限定對應到你的資料需求規格（8.1.2 節的 `excluded` 欄位）、最小化對應到清理管線的欄位篩選、保存期限對應到 `provenance ledger` 中的到期日與自動提醒。它們不是抽象的原則，而是可以寫進程式的規則。

8.4.3 去識別的層次與極限

手法	做法	可逆性	限制
刪除	移除欄位	不可逆	可能連帶失去有用資訊
遮罩	換成佔位符，如 [姓名]	不可逆	仍可能從上下文推知
假名化	換成一致的代號，如 P001	持有對照表者可逆	對照表本身是高風險資產
泛化	把年齡變成年齡層、地址變成縣市	不可逆	準識別資訊仍可能組合
擾動	加入雜訊	不可逆	會影響資料效用

必須誠實面對的是：去識別不是保證。研究一再顯示，看似匿名的資料在與其他資料集交叉比對後仍可能被重新識別，尤其當資料含有豐富的準識別資訊時。所以去識別應該被視為「降低風險」而非「消除風險」，而且不能取代其他保護措施（存取控制、最小化、保存期限）。

對大模型而言還有一個特殊風險：模型可能記憶並在生成時吐出訓練資料中的內容（第 16 章會讓你量測這件事）。這意味著即使你做了遮罩，如果遮罩不完全，殘留的個資可能在某天被使用者誘發出

來。這是第 37 章紅隊測試要處理的攻擊面之一。

8.4.4 繁體中文個資偵測的難處

類型	為什麼難	可用線索	殘餘風險
中文姓名	沒有大小寫線索，姓氏與常用字重疊	姓氏表 + 上下文（「王同學」）	罕見姓氏、單名、外籍姓名
地址	格式多變，含縣市鄉鎮村里鄰	層級關鍵字 + 數字模式	簡略寫法、地標式描述
準識別組合	單項不敏感，組合才敏感	需規則或模型判斷	極高
稱謂與職稱	「三年二班的班長」可鎖定個人	需場域知識	極高

這張表說明一件事：純規則式的偵測必然有漏網之魚。實務上的做法是多層防護——規則抓格式明確的（身分證字號、電話、電子郵件），模型或人工抓語意層的（姓名、稱謂），再加上抽樣人工覆核。程式 8B 會實作前兩層，並強制產出可供人工覆核的報告。

一條不可妥協的原則

任何自動化的個資處理都必須有人工覆核的環節，而且覆核的樣本必須隨機抽取而非只看程式標記出來的部分——因為你真正該擔心的是它沒標記到的那些。本書要求：涉及真實個資的專案，人工覆核比例不得低於 1%，且必須記錄覆核者與覆核時間。

8.4.5 模型記憶：個資風險的特殊形態

傳統的資料保護思維假設資料存在某個地方，所以保護它就是控制存取。大模型打破了這個假設：訓練資料被編碼進參數，而模型可能在生成時把它吐出來。

因素	增加記憶風險	降低記憶風險	本書對策
重複次數	同一內容出現多次	徹底去重	第 9 章的去重管線
內容獨特性	罕見的字串（如身分證字號）	常見的表述	第 8B 的遮罩
模型大小	參數越多記得越牢	小模型記較少	第 26 章的取舍
訓練輪數	同一資料看越多次	單輪或少輪	第 15 章的設定

因素	增加記憶風險	降低記憶風險	本書對策
資料量	總量少時每筆權重高	大量資料稀釋	第 17 章的配比

第二列有一個違反直覺的意涵：越獨特的資訊越容易被記住，而個人資料恰恰是高度獨特的。一個在語料中只出現一次的身分證字號，反而比出現一萬次的常見詞更可能被逐字複述——因為模型無法用一般化的規律來壓縮它，只能記下來。

這件事的實務結論是：遮罩必須在訓練前做，而且要做徹底。事後才發現遺漏，你面對的就不再是「刪除一筆資料」的問題，而是第 28 章的機器遺忘問題——那要困難得多，而且效果需要驗證。

第 16 章會教你量測模型的記憶化程度，第 37 章會教你用紅隊測試主動探測殘留的個資。但最有效的防線永遠在最上游：不要讓不該進去的東西進去。

8.5 Provenance ledger：資料的身分證

8.5.1 為什麼需要它

半年後有人問你：「模型講出這句話，是從哪份資料學來的？」如果你答不出來，你就無法修正、無法回應投訴、無法配合退出請求、也無法在論文中誠實描述你的方法。

Provenance ledger（來源帳本）就是為了回答這類問題而存在的。它不是一份文件，而是一個貫穿整個資料生命週期的紀錄機制——每一筆資料進來時登記，每一次處理時追加，每一次使用時引用。

8.5.2 必要欄位

欄位群	內容	回答什麼問題	缺了會怎樣
來源	提供者、網址、取得方式、取得時間	這份資料哪裡來的？	無法回應任何追問
權利	授權類型、條款版本、確認人、風險等級	我們可以用嗎？	無法證明合規
內容	筆數、字元數、語言比例、時間範圍	這份資料裡有什麼？	無法解釋模型行為
處理	套用過的每一步、影響筆數、程式版本	它被改過什麼？	無法重現

欄位群	內容	回答什麼問題	缺了會怎樣
個資	掃描結果、遮罩方式、覆核人與時間	有沒有個資？處理了嗎？	無法回應個資查詢
使用	進了哪些切分、哪些模型用過	影響範圍多大？	無法評估退出的影響
生命週期	保存期限、到期處置、退出紀錄	什麼時候該刪？	資料無限期累積

data/manifest/gov_regulations_v3.yaml

```
dataset_id: gov-regulations
version: "2026-08-15-a"

source:
  provider: 某中央機關法規資料庫
  method: 官方 API
  url: https://example.gov.tw/api/laws          # 實際專案請填真實位址
  retrieved_at: "2026-08-15T09:30:00+08:00"
  terms_snapshot: docs/terms/gov_2026-08-15.pdf # 當時的條款存檔

rights:
  license: 政府資料開放授權條款
  license_version: "1.0"
  commercial_use: true
  redistribution: true
  attribution_required: true
  risk_level: green          # 見 8.3.3 節
  confirmed_by: 法務室      # 誰確認的
  confirmed_at: "2026-08-16"

content:
  n_documents: 12480
  n_chars: 48_320_115
  sha256: "a3f2c1..."
  languages: {zh-Hant: 0.98, en: 0.02}
  time_range: ["1949-01-01", "2026-08-15"]

processing:
  - {op: decode, from: utf-8, at: "2026-08-15"}
  - {op: normalize, spec: tw-text-spec-v1.0, changed_docs: 12480}
  - {op: pii_scan, tool: formosa.pii@0.3.1, findings: 214}
  - {op: pii_review, reviewer: 助教-李天宇, at: "2026-08-18",
      confirmed: 12, false_positive: 202}

usage:
  splits: [train, valid_gov]
  used_by: [miniformosa-v1, campus-rag-v2]

lifecycle:
  retention_until: "2029-08-15"
  on_expiry: 重新評估或刪除
  opt_out_channel: data-request@example.edu.tw
```

請注意 `terms_snapshot` 這個欄位：它保存了你取得資料當時的條款內容。條款會改，而你需要證明的是「取得當時的條款允許」，不是「今天的條款允許」。這個小細節在爭議發生時可能是關鍵。

同樣值得注意的是 `pii_review` 那一行的 `false_positive` 數字：202 筆誤判、12 筆確認。這個比例告訴你偵測器的精確度，也讓後續的人知道這份資料的個資風險已經被實際檢視過，而不只是跑過一支程式。

8.5.3 從 ledger 到可回答的問題

有人問	你查什麼	如何回答	章節
這句話從哪來的？	usage 反查 + 第 16 章記憶化偵測	指出可能的資料集與版本	第 16 章
我的資料在裡面嗎？	來源與 <code>opt_out</code> 紀錄	查詢並回覆，必要時執行退出	本章 8.7 節
你們合法嗎？	rights 群組	出示授權、確認人與日期	本章 8.3 節
能重現嗎？	processing + sha256	提供完整處理鏈與雜湊	第 4.7.3 節
資料多久刪？	lifecycle	出示保存期限與處置方式	本章 8.4.2 節

8.6 Data Card、責任人與審批流程

8.6.1 Data Card 與 provenance 的分工

Provenance ledger 是給機器與稽核用的完整紀錄；Data Card 是給人看的摘要文件。兩者都需要，但用途不同。

面向	Provenance ledger	Data Card	備註
讀者	工程師、稽核人員	任何要使用這份資料的人	—
詳細度	每一步都記	重點摘要	—
格式	結構化 (YAML/JSON)	文件 (Markdown)	—
更新頻率	每次處理都追加	版本變更時	—
關鍵內容	完整可追溯鏈	用途、限制、已知偏誤	限制那一節最重要

8.6.2 Data Card 的必備段落

1. 這份資料是什麼：來源、規模、時間範圍、語言與領域組成。
2. 它被建立來做什麼：預期用途，以及明確不建議的用途。
3. 怎麼收集與處理的：管道、清理步驟、去識別方式。
4. 裡面有什麼樣的人與內容：涵蓋範圍與代表性，誰被過度呈現、誰被遺漏。
5. 已知的限制與偏誤：這一節最重要，也最常被寫得敷衍。
6. 權利與授權：可做什麼、不可做什麼、標示義務。
7. 個資處理：掃描與遮罩的方式、殘餘風險、退出管道。
8. 維護資訊：責任人、更新頻率、保存期限、聯絡方式。

第五段的寫法有一個檢驗標準：如果你的「已知限制」寫的是「資料量可能不足」這種放諸四海皆準的話，那等於沒寫。有價值的限制描述是具體的，例如「本資料集的社群語料主要來自兩個論壇，年齡層偏向 20 至 35 歲，可能無法代表長者與國中小學生的用語」。

8.6.3 角色與審批

角色	負責什麼	在專題中由誰擔任	常見的失職
資料責任人	這份資料的一切問題由他回答	組內固定一人，不輪流	沒有指定，出事時互推
權利確認者	確認授權與契約範圍	指導老師轉介法務	工程師自行判斷
個資窗口	確認個資處理的合法性基礎	學校個資窗口	完全略過
技術執行者	實作清理、遮罩、切分	組員	未依規格執行卻未回報
覆核者	抽樣人工檢查	另一位組員（不可與執行者同人）	由執行者自己覆核

最後一系列的「不可與執行者同人」是一條簡單但有效的規則。自己檢查自己的工作，會系統性地看不見自己的盲點。在課程專題中這只是換個人看一眼的成本，但它抓到的問題往往是最嚴重的那些。

8.6.4 存取控制與稽核軌跡

不是所有人都該看得到所有資料。分級的原則很簡單：依敏感度分層，依需要授權。

層級	內容	誰可存取	技術措施
公開	已公開授權的語料	全組	一般版控
內部	機構自有的非敏感文件	專案成員	存取紀錄
受限	含準識別資訊的資料	指定成員 + 責任人核准	加密儲存、存取紀錄、定期檢視
機密	含個資或營業秘密的原始資料	最小必要人員	隔離環境、不可下載、完整稽核

稽核軌跡的最低要求是：誰、在什麼時間、存取了哪一份資料、做了什麼。這在課程專題中可以簡單到只是一份共用的存取登記表，重點不在工具有多先進，而在於這件事真的有被記錄。第 37 章會把它擴充成完整的 LLMOps 稽核機制。

8.7 退出機制與資料生命週期

8.7.1 為什麼退出很難

假設有人要求「把我的資料從你們的模型裡移除」。這件事在技術上比想像的困難得多，因為資料一旦進入訓練，它就被揉進了數億個參數裡，無法像刪除資料庫紀錄那樣拿掉。

層次	能做到什麼	成本	章節
資料集層	從語料中移除，未來重訓不再包含	低	本章
檢索層	從 RAG 索引中移除，立即生效	低	第 30、31 章
輸出層	加入過濾規則，阻止相關輸出	低但脆弱	第 33 章
模型層	機器遺忘技術，移除特定知識	高且效果需驗證	第 28 章
重新訓練	從乾淨語料重訓	極高	第 15 章

實務上的做法是組合：立即在資料集與檢索層移除（幾小時內可完成），同時評估模型層是否需要處理。誠實地告知請求者「哪些已立即生效、哪些需要時間、哪些技術上有限制」，比含糊承諾「已完全移除」要負責得多。

8.7.2 退出流程的設計

1. 提供明確的聯絡管道，並寫進 Data Card 與服務說明中。
2. 收到請求後確認請求者身分與請求範圍（避免有人代他人請求移除）。
3. 在資料集中定位受影響的內容，記錄找到幾筆、在哪些切分中。
4. 執行移除：資料集、檢索索引、快取，並記錄執行時間。
5. 評估模型層影響：這批資料是否已進入某個模型的訓練？若是，評估重訓或編輯的必要性。
6. 回覆請求者：說明已完成什麼、預計何時完成什麼、以及技術上的限制。
7. 把整個過程記入 provenance ledger，包括未能完成的部分與理由。

第七步特別重要。誠實記錄「我們無法從已訓練的模型中完全移除這些內容」不是承認失敗，而是專業的表現——它讓後續的人知道現況，也讓組織能評估是否需要投入資源處理。

8.7.3 保存期限與到期處置

資料不該無限期存放。設定期限的理由有三個：降低外洩風險、控制儲存成本、以及強迫定期重新評估這份資料是否還有價值。

formosa/lifecycle.py — 到期檢查

```
from datetime import date
from pathlib import Path
import yaml

def check_expiry(manifest_dir: Path, warn_days: int = 90) -> dict:
    today = date.today()
    expired, expiring, ok = [], [], []

    for f in sorted(manifest_dir.glob("*.yaml")):
        m = yaml.safe_load(f.read_text(encoding="utf-8"))
        life = m.get("lifecycle", {})
        until = life.get("retention_until")
        if not until:
            expired.append((f.name, "未設定保存期限"))    # 這也算問題
            continue
        d = date.fromisoformat(str(until))
        days = (d - today).days
        if days < 0:
            expired.append((f.name, f"已逾期 {-days} 天"))
        elif days <= warn_days:
            expiring.append((f.name, f"{days} 天後到期"))
        else:
            ok.append(f.name)

    return {"expired": expired, "expiring": expiring,
            "ok_count": len(ok)}
```

請注意「未設定保存期限」被歸類為 `expired`。這是刻意的設計：沒有期限不等於永久有效，而等於「還沒有人想過這件事」，那本身就是一個需要處理的狀態。

8.8 資料隔離：科學問題也是治理問題

8.8.1 五種資料的隔離

第 5 章從科學角度講過切分，這裡從治理角度再看一次。兩者的要求剛好一致，但理由不同。

資料	科學上的理由	治理上的理由	隔離要求
訓練集	模型學習的來源	需完整的授權與個資處理	—
驗證集	調參的依據	同上	不得與訓練集重疊
測試集	最終評估	需能證明結論可信	嚴格隔離，看的次數要記錄
紅隊資料	安全測試	可能含刻意設計的攻擊內容	不得進入訓練，需另行管控
保留集	偵測污染與自欺	第三方稽核的依據	完全不看，鎖存

第四列值得展開：紅隊資料中會包含各種誘發不當行為的提示（第 37 章會建立它們）。這些內容如果不小心混進訓練資料，會讓模型學會產生你正想防止的東西。所以紅隊資料的儲存位置、存取權限與流向都必須獨立管控。

8.8.2 隔離的技術實作

用雜湊分桶保證隔離（延續第 5.4.5 節）

```
import hashlib

SPLIT_SALT = "formosa-2026-v1"    # 記錄在設定檔與環境指紋中

def assign_split(doc_id: str) -> str:
    """同一份文件永遠落在同一個切分，新增資料不影響既有分配"""
    h = hashlib.blake2b((SPLIT_SALT + doc_id).encode("utf-8"),
                        digest_size=8).digest()
    b = int.from_bytes(h, "big") % 1000
    if b < 940:    return "train"
    if b < 970:    return "valid"
    if b < 995:    return "test"
    return "holdout"                # 5% 保留集，永不查看

def verify_isolation(splits: dict) -> list[str]:
    """檢查各切分之間沒有重疊的 doc_id"""
    problems = []
    names = list(splits)
    for i, a in enumerate(names):
        for b in names[i + 1:]:
            overlap = set(splits[a]) & set(splits[b])
```

```

    if overlap:
        problems.append(
            f"{a} 與 {b} 有 {len(overlap)} 份重疊文件")
    return problems

```

verify_isolation 應該在每次資料更新後自動執行，並在有問題時讓建置流程失敗。這是把治理要求變成技術強制的典型做法——依賴人記得檢查是不可靠的，讓程式在出問題時直接擋下來才可靠。

8.9 校園場域的特殊風險

本書的目標讀者有相當比例會在教育場域工作或做專題，而校園資料有一批特別需要謹慎的類型。

資料類型	風險	本書立場	必須諮詢
學生作業與作品	著作權屬學生；可能含個人經驗敘述	需個別同意，不預設可用	學生本人 + 教務
課堂錄音錄影	含所有在場者的聲音與影像	需全體同意，且應可退出	教務 + 個資窗口
輔導與健康紀錄	屬高度敏感資料	不使用，無例外	不適用——不要用
成績與出缺勤	可識別個人學習狀況	不使用於訓練	個資窗口
未成年人資料	同意能力受限，需法定代理人	額外的保護要求	個資窗口 + 法務
校務公開資訊	學則、課程、公告	通常可用，仍須確認	教務

第三列的「不使用，無例外」是本書少數的絕對立場。輔導紀錄涉及學生最脆弱的時刻，把它們用於任何形式的模型訓練，都可能造成無法挽回的傷害，而技術上的去識別無法充分保護。如果你的專案「需要」這類資料才能運作，那個專案的設計本身就該重新檢討。

第五列的未成年人議題在臺灣的教育應用中無法迴避。取得同意的方式、法定代理人的角色、以及學生日後成年時的權利，這些都不是工程師能自行決定的。本書的建議是：涉及未成年人資料的專案，一律在專案啟動前就取得書面的機構核可，不要邊做邊補。

一個給指導老師的提醒

學生做專題時，最容易出問題的不是惡意，而是「反正是自己學校的資料，應該沒關係」這種想法。建議在專題第一週就明確劃出可用與不可用的資料範圍，並要求每組在提案時填寫 8.1.2 節那份含 excluded 欄位的規格。事前劃線的成本遠低於事後處理。

程式實作

本章三個實作都在 A 級硬體上完成。它們是治理機制的技術骨架，會被第 9、22、36、37、40 章直接使用。

程式 8A 語料清冊檢查器

目標：讀取所有 manifest 檔，自動找出缺少授權、來源、保存期限或個資處理紀錄的資料集，並依風險等級排序輸出。

scripts/check_manifests.py (節錄)

```
import sys, yaml
from pathlib import Path
from datetime import date

REQUIRED = {
    "來源": ["source.provider", "source.method", "source.retrieved_at"],
    "權利": ["rights.license", "rights.risk_level",
            "rights.confirmed_by", "rights.confirmed_at"],
    "內容": ["content.n_documents", "content.sha256"],
    "個資": ["processing"],
    "生命週期": ["lifecycle.retention_until"],
}

BLOCKING = {"red", "orange"}          # 這兩級不得進入訓練

def get(d, path):
    cur = d
    for k in path.split("."):
        if not isinstance(cur, dict) or k not in cur:
            return None
        cur = cur[k]
    return cur

def check_one(m: dict, name: str) -> list[dict]:
    issues = []
    for group, fields in REQUIRED.items():
        for f in fields:
            if get(m, f) in (None, "", []):
                issues.append({"level": "error", "dataset": name,
                              "msg": f"缺少 {group} 的 {f}"})

    risk = get(m, "rights.risk_level")
    if risk in BLOCKING:
        issues.append({"level": "error", "dataset": name,
                      "msg": f"風險等級 {risk}，不得用於訓練"})

    # 個資掃描做了嗎？有沒有人覆核？
    ops = [p.get("op") for p in (get(m, "processing") or [])]
    if "pii_scan" not in ops:
        issues.append({"level": "error", "dataset": name,
```

```

        "msg": "未執行個資掃描"})
    elif "pii_review" not in ops:
        issues.append({"level": "warn", "dataset": name,
                       "msg": "個資掃描結果尚未經人工覆核"})

    return issues

def main(manifest_dir="data/manifest"):
    all_issues = []
    for f in sorted(Path(manifest_dir).glob("*.yaml")):
        m = yaml.safe_load(f.read_text(encoding="utf-8"))
        all_issues += check_one(m, f.stem)

    errors = [i for i in all_issues if i["level"] == "error"]
    for i in all_issues:
        mark = "X" if i["level"] == "error" else "!"
        print(f"{mark} [{i['dataset']}] {i['msg']}")
    print(f"\n 錯誤 {len(errors)} 項，警告 {len(all_issues) - len(errors)} 項")
    sys.exit(1 if errors else 0)      # 讓 CI 能擋下有問題的資料

```

最後那行 `sys.exit` 是這支程式的關鍵：把它接進第 4 章的 CI 流程，任何缺少授權或未做個資掃描的資料集都無法進入訓練管線。這是「把治理變成技術強制」的具體實踐——不依賴任何人記得檢查。

程式 8B 繁體中文個資掃描與遮罩

目標：建立多層次的個資偵測管線，產出可供人工覆核的差異報告。這支程式的設計目標不是「抓到全部」（那做不到），而是「抓到大部分，並讓沒抓到的部分容易被人發現」。

formosa/pii.py (節錄)

```

import re
from dataclasses import dataclass

PATTERNS = {
    "tw_id": re.compile(r"\b[A-Z][12]\d{8}\b"),
    "phone_mob": re.compile(r"\b09\d{2}[-\s]?d{3}[-\s]?d{3}\b"),
    "phone_land": re.compile(r"\b0\d{1,2}[-\s]?d{6,8}\b"),
    "email": re.compile(r"\b[\w.+-]+@[\w-]+\.[\w.-]+\b"),
    "unified": re.compile(r"(?!d)\d{8}(?!d)"),
    "address": re.compile(
        r"[\u4e00-\u9fff]{2,4}(?:縣|市)[\u4e00-\u9fff]{1,4}"
        r"(?:鄉|鎮|市|區)[\u4e00-\u9fff0-9]{0,20}?d+號"),
    "name_ctx": re.compile(
        r"(?:聯絡人|承辦人|導師|學生|申請人)[ : \s]*"
        r"([\u4e00-\u9fff]{2,4})"),
}

SURNAMES = set("陳林黃張李王吳劉蔡楊許鄭謝郭洪曾邱廖賴徐周葉蘇莊"
               "呂江何蕭羅高潘簡朱鍾游詹胡施沈余趙盧梁顏柯翁魏孫戴")

def _valid_tw_id(s: str) -> bool:

```

```

"""用公開的格式檢核規則降低誤判，非用於產生號碼"""
letters = "ABCDEFGHJKLMNPQRSTUVWXYZIO"
if len(s) != 10 or s[0] not in letters or s[1] not in "12":
    return False
n = letters.index(s[0]) + 10
total = n // 10 + (n % 10) * 9
for i, ch in enumerate(s[1:9]):
    total += int(ch) * (8 - i)
return (total + int(s[9])) % 10 == 0

@dataclass
class Finding:
    kind: str
    span: tuple
    raw: str
    confidence: str      # high / medium / low

def scan(text: str) -> list[Finding]:
    out = []
    for kind, pat in PATTERNS.items():
        for m in pat.finditer(text):
            raw = m.group(0)
            conf = "high"
            if kind == "tw_id" and not _valid_tw_id(raw):
                conf = "low"          # 格式像但檢核碼不符
            if kind == "unified":
                conf = "low"          # 八位數字誤判率高
            if kind == "name_ctx":
                name = m.group(1)
                conf = "high" if name[0] in SURNAMES else "medium"
            out.append(Finding(kind, m.span(), raw, conf))
    return sorted(out, key=lambda f: f.span[0])

def mask(text: str, findings: list[Finding],
        min_conf: str = "medium") -> tuple[str, int]:
    order = {"low": 0, "medium": 1, "high": 2}
    keep = [f for f in findings if order[f.confidence] >= order[min_conf]]
    out, last, n = [], 0, 0
    for f in sorted(keep, key=lambda f: f.span[0]):
        if f.span[0] < last:          # 重疊，跳過
            continue
        out.append(text[last:f.span[0]])
        out.append(f"[{f.kind}]")
        last = f.span[1]; n += 1
    out.append(text[last:])
    return "".join(out), n

```

設計上的三個重點值得說明。第一，confidence 分級讓你可以調整遮罩的積極程度：處理高敏感資料時連 low 都遮，處理一般資料時只遮 high 與 medium。第二，身分證字號的格式檢核只用於降低誤判，這是保護導向的用途。第三，八位數字的統一編號誤判率極高（任何八位數都會命中），所以預設標為低信心，交給人工判斷。

必須產出的三份輸出：

- 差異報告：原文與遮罩後的並排比較，標出每一處改動與其信心等級，供人工覆核。
- 統計摘要：各類型的命中數、信心分布、以及每千字的命中密度。密度異常高的文件應優先人工檢視。
- 抽樣清單：隨機抽取 1% 的「未被標記」段落，供覆核者檢查有沒有漏掉的個資。這一份比前兩份更重要。

為什麼抽樣清單最重要

前兩份報告只能告訴你「程式抓到了什麼」，但你真正該擔心的是它沒抓到的。隨機抽樣未標記的內容，是唯一能估計漏判率的方法。這與第 36 章評測的邏輯完全一致：只看你已經知道的部分，會系統性地高估自己的表現。

程式 8C 資料退出處理器

目標：接受移除請求後，自動定位受影響的資料、產生影響範圍報告、執行移除，並記錄整個過程。

scripts/handle_optout.py (節錄)

```
import json, hashlib
from datetime import datetime, timezone
from pathlib import Path

def locate(request: dict, index_dir: Path) -> dict:
    """依請求條件定位受影響的文件
    request 例: {"type": "by_source_url", "value": "https://..."}
               {"type": "by_doc_id", "value": ["d1", "d2"]}
               {"type": "by_text_match", "value": "某段文字"}
    """
    affected = []
    for f in index_dir.glob("*.jsonl"):
        for line in f.open(encoding="utf-8"):
            rec = json.loads(line)
            if matches(rec, request):
                affected.append({"dataset": f.stem,
                                "doc_id": rec["doc_id"],
                                "split": rec.get("split")})

    by_split = {}
    for a in affected:
        by_split.setdefault(a["split"], []).append(a["doc_id"])
    return {"n_affected": len(affected), "by_split": by_split,
            "details": affected}

def impact_report(located: dict, model_registry: Path) -> dict:
    """哪些模型用過這些資料？需不需要處理模型層？"""
    reg = json.loads(model_registry.read_text(encoding="utf-8"))
    datasets = {a["dataset"] for a in located["details"]}
    impacted_models = [m for m in reg
                        if datasets & set(m["trained_on"])]

    return {
        "datasets": sorted(datasets),
        "models": [m["name"] for m in impacted_models],
        "actions": {
            "corpus": "立即移除",
```

```

        "rag_index": "立即移除（見第 30 章）",
        "trained_models": ("需評估：重訓、模型編輯（第 28 章）"
                           "或輸出過濾") if impacted_models else "無",
    },
}

def execute(located: dict, ledger: Path, requester_ref: str):
    """執行移除並寫入不可否認的紀錄"""
    now = datetime.now(timezone.utc).isoformat()
    entry = {
        "type": "opt_out",
        "requester_ref": requester_ref,      # 去識別後的請求編號
        "requested_at": now,
        "n_removed": located["n_affected"],
        "by_split": {k: len(v) for k, v in located["by_split"].items()},
        "completed_at": now,
        "limitations": "已訓練模型中的殘留內容無法保證完全移除",
    }
    with ledger.open("a", encoding="utf-8") as f:
        f.write(json.dumps(entry, ensure_ascii=False) + "\n")
    return entry

```

注意 entry 中的 limitations 欄位。每一次退出處理都必須誠實記錄技術上的限制，而且這句話應該同時出現在給請求者的回覆中。承諾做不到的事，比誠實說明限制要糟糕得多。

系統案例：臺灣教育語料庫的治理設計

本章的系統案例是規劃一個由政府公開資料、校內自有資料與出版社授權資料三種來源組成的臺灣教育語料庫，並設計完整的權利基礎與退出流程。

三種來源的治理差異

來源	典型內容	主要治理工作	最容易出錯的地方
政府公開資料	課綱、法規、統計、公告	確認授權與標示義務	以為公開就等於可任意再利用
校內自有資料	學則、課程大綱、公告、教材	確認內部核可與個資篩除	教材中夾帶第三方著作
出版社授權	教科書、參考書、題庫	契約範圍、期限、終止後義務	契約未涵蓋模型訓練這種用途

第三列是一個真實且常見的困境：許多既有的授權契約簽訂時並未預想到「用於訓練人工智慧模型」這種利用方式，因此條款可能沒有明確涵蓋。這時候不能自行推定可以或不可以，必須回到合作方重新確認——而這正是需要提早啟動的工作，不能等到要訓練前一週才發現。

必須交付的六項

1. 資料清冊：每一份資料一個 manifest，通過程式 8A 的檢查（零錯誤）。
2. 權利基礎表：逐份說明依據什麼可以使用、由誰確認、確認日期，以及被排除的資料與排除理由。
3. 風險分級結果：綠黃橙紅四級的分布，橙紅級的處置說明。
4. 個資處理報告：程式 8B 的三份輸出，含人工覆核紀錄與覆核者簽名。
5. Data Card：依 8.6.2 節的八段撰寫，其中「已知限制與偏誤」必須具體。
6. 退出流程說明：對外可公開的版本，寫明管道、處理時程與技術限制。

評分重點

- 排除清單的品質：你排除了什麼、為什麼排除。只列出使用的資料而沒有排除紀錄者，視為未做評估。
- 「已知限制」是否具體。寫「資料可能有偏誤」不予計分。
- 個資報告中的抽樣清單是否真的隨機、覆核者是否與執行者不同人。
- 退出流程是否誠實說明技術限制。承諾「完全移除」者反而扣分。
- 所有涉及法律判斷之處是否標註「待專業確認」而非自行下結論。

章末評量

一、概念題

1. 為什麼資料需求規格中的 excluded 欄位比 included 欄位更重要？請舉一個具體情境說明。
2. 比較五種資料來源的風險輪廓，並說明為什麼合作授權與自有資料對臺灣專案特別有價值、也特別需要治理。
3. 什麼是準識別資訊？為什麼它在小規模社群中特別危險？請舉一個校園情境的例子。
4. 解釋目的限定、最小化與保存期限三項原則，並各說出一個工程上的具體對應做法。
5. 為什麼說「去識別是降低風險而非消除風險」？大模型在這件事上有什麼額外的風險？
6. 為什麼紅隊資料必須與訓練資料嚴格隔離？混入了會發生什麼？

二、判斷與分析題

1. 以下五種情況，請分別判斷風險等級（綠 / 黃 / 橙 / 紅）並說明你需要向誰確認什麼：（a）政府開放資料平台上標示可自由使用的統計資料；（b）某教授個人網站上的講義；（c）合作企業提供的設備維修工單；（d）某新聞網站的付費訂閱文章；（e）學生在課堂上繳交的作文。
2. 某團隊想用學校的圖書借閱紀錄訓練一個「閱讀推薦助理」。請列出至少五個必須先釐清的問題，並指出各該問誰。
3. 一份資料集的 manifest 中，rights.confirmed_by 寫的是「專題組長」。請說明這為什麼有問題，以及應該怎麼修正。
4. 某模型被發現會在特定提示下輸出一位學生的姓名與班級。請說明你的處理步驟與順序，以及哪些步驟有技術上的限制。

三、除錯題

1. 同學的個資掃描報告顯示「命中 0 筆」，但人工檢查明顯有電話號碼。請列出三個可能原因。
2. 同學的統一編號偵測誤判率極高，把所有八位數字都標記出來。請提出三種降低誤判的方法。
3. 同學的退出處理程式執行後回報「已完全移除」，但一週後同樣的內容仍出現在 RAG 回覆中。請列出兩個可能原因。
4. 一份 manifest 通過了程式 8A 的檢查，但助教仍判定不合格。請列出至少三種「格式完整但實質有問題」的情況。

四、系統設計題

1. 為一個「產學合作的智慧製造維修助理」設計完整的資料治理方案。企業提供工單與 SOP，其中

含技術人員姓名、客戶名稱與製程參數。請設計：資料分級、存取控制、個資處理、契約需釐清的事項、以及專案結束後的資料處置。

2. 你的學校要建立一個開放給全校師生使用的 AI 助理。請設計資料的准入流程：誰可以提交資料、要填什麼、誰審查、以什麼標準通過、以及被拒絕時如何申訴。並說明你的流程在什麼情況下會失效。

五、臺灣情境與倫理題

1. 本章對輔導紀錄採取「不使用，無例外」的立場。請提出一個支持這個立場的論證，以及一個可能的反對意見，並說明你會如何回應該反對意見。
2. 假設你在整理語料時發現某份「公開」資料中含有大量可識別到個人的內容，而該資料已被許多研究團隊使用多年。請說明你會怎麼做，以及你認為責任應如何分配。
3. 有人主張「臺灣的語料太少，如果每份都要確認授權，我們永遠做不出本土模型」。請提出三個回應，並誠實說明這個主張有道理的地方。
4. 一位學生希望把自己三年前的課堂作品從語料庫中移除。技術上你能從資料集移除，但無法保證從已訓練的模型中完全消除。請寫出你會給他的回覆（實際的回覆內容，不是說明）。

評量提示（給自學者）

- 判斷題第 1 題參考方向：（a）綠，仍須確認標示義務；（b）橙，無明確授權宣告，宜聯繫取得同意；（c）黃至橙，取決於合作契約範圍，須問合作方與法務；（d）紅，付費內容；（e）橙至紅，著作權屬學生且可能含個人敘述，須個別同意。
- 除錯題第 1 題：正規表示式的 \b 邊界在中文旁不成立（中文字不是 word boundary 的分界）> 文本中的電話用了全形數字而未先正規化（第 6 章）> 掃描時傳入的是已遮罩過的版本。
- 除錯題第 3 題：RAG 的向量索引未同步更新 > 快取未清除。這正是 8.7.1 節說的「多層次移除」的實務意義。

研究延伸與版本注意事項

- 我國個人資料保護法及其施行細則，以及主管機關發布的相關解釋與指引。這些是校園與企業應用的直接依據，但條文的具體適用必須由專業人員判斷。
- 我國著作權法中關於著作財產權限制與合理使用的規定。建議閱讀條文本身而非二手整理，並注意人工智慧相關的實務見解仍在發展中。
- Data Card 與 Datasheets for Datasets 的相關提案：這類文件格式的設計理念值得參考，第 40

章的專題交付會用到。

- 資料去識別與重新識別風險的研究：這個領域的實證結果對「去識別到什麼程度才夠」有直接參考價值。
- 機器遺忘 (machine unlearning) 的研究：第 28 章會深入，但若你的專案已預期會有退出請求，建議提早了解。
- 本章涉及的法規與實務見解會持續變動，特別是人工智慧訓練資料的相關規範在國內外都在發展中。每學期開課前應重新查核，並在課程網頁公告更新。

常見問題

問：課程作業也要做這麼嚴格的治理嗎？答：作業使用的是課程提供的、已確認授權的語料，所以你不必自己去確認。但本章要求你把流程走一遍，理由是：你將來做專題、進業界、或做研究時，這件事會變成你的責任，而那時候沒有人會從頭教你。把它當成一次演練。

問：我只是做 RAG，不訓練模型，需要這一章嗎？答：需要，而且某些方面要求更高。RAG 會把文件原文直接呈現給使用者，所以文件中的個資會被直接看到——這比模型記憶的風險更立即。此外檢索引的退出處理雖然技術上簡單，但你必須先有能力定位受影響的內容。

問：法律的部分我看不懂，怎麼辦？答：你不需要看懂到能做判斷，只需要看懂到能提出正確的問題。本章的目標是讓你知道「這裡有風險、我該問誰」。實際的判斷交給法務、個資窗口或指導老師，這不是推卸責任，而是專業分工。

問：如果我的專案因為授權問題做不下去怎麼辦？答：那是這個流程正在發揮作用。比較好的處理方式是及早發現並調整範圍——換一個資料來源、縮小應用場景、或改用合成資料。最糟的情況是做到最後才發現，那時候的沉沒成本會讓人傾向於「先做了再說」，而那正是問題的開始。

教師使用建議

本章排在第 8 週，建議 2 小時講授加 1 小時實驗。本章是全書「學生覺得最無聊、老師覺得最重要」的一章，建議大量使用真實案例與角色扮演來建立動機。

時段	內容	互動設計	注意事項
前 20 分鐘	8.1 至 8.2 節，需求反推與來源	請各組當場填寫 excluded 欄位	這一欄要在課堂上完成
中間 30 分鐘	8.3 至 8.4 節，權利與個資	用 5 個情境做風險分級投票	投票分歧處正是教學重

時段	內容	互動設計	注意事項
			點
後 30 分鐘	8.5 至 8.7 節 · ledger 與退出	角色扮演：一位學生要求移除資料	這個扮演的效果非常好
最後 40 分鐘	8.8 至 8.9 節與作業說明	校園資料的可用性分類練習	輔導紀錄的立場要說清楚
實驗課 1 小時	程式 8A 與 8B	互相檢查對方的 manifest	8B 務必用合成資料，不得用真實個資

一個效果極佳的課堂活動：準備五份 manifest，其中三份有不同類型的問題（缺授權確認人、風險等級為橙卻列入訓練、個資掃描未經覆核）。請各組在五分鐘內找出問題。學生會發現自己看漏了某一項，而那正是為什麼需要自動化檢查器。

實驗課的重要提醒：程式 8B 的測試資料必須是合成的假資料，絕不可使用真實的個人資料來練習偵測。請事先準備一份合成資料集，內含格式正確但不對應真實個人的號碼與姓名。這既是教學倫理的要求，也是給學生的示範。

高中授課的調整：8.1、8.2 完整教；8.3 只講「四種權利可能同時存在」與「不要自行判斷合理使用」；8.4 講到準識別資訊的例子即可，法條細節可略；8.5 的 ledger 概念用「資料的身分證」比喻很好懂；8.9 的校園議題與高中生高度相關，值得完整討論。程式部分只做 8A（填表與檢查），8B 由老師示範。

附：資料治理成熟度自評

不是每個專案都需要（或做得到）最高等級的治理。這張表讓你判斷自己現在在哪一級、下一步該補什麼。

等級	特徵	典型情境	升級到下一級要做什麼
L0 無	不知道資料哪裡來的	匆促完成的作業	建立最基本的來源紀錄
L1 記錄	有來源清單但不完整	一般課程專題的起點	補齊授權欄位與風險分級
L2 可追溯	manifest 完整、通過自動檢查	本書課程的要求	加入個資掃描與人工覆核

等級	特徵	典型情境	升級到下一級要做什麼
L3 可問責	有責任人、覆核紀錄、退出管道	對外服務的最低要求	加入存取控制與稽核軌跡
L4 可稽核	完整稽核軌跡、定期檢視、影響評估	正式營運與產學合作	第 37 章的治理框架

本書課程的畢業門檻是 L2，總整專題（第 40 章）若要對外展示則需達到 L3。如果你的專題涉及真實使用者或真實個資，L3 是最低要求，不是加分項。

一個誠實的觀察：多數學生專題起步時在 L0，而多數業界專案在 L1 到 L2 之間。這不是為了讓你安心，而是要說明一件事——如果你在課程中就把 L2 到 L3 走過一遍，你在這件事上的準備度會超過相當多的實務工作者。這是一個投資報酬率很高的能力。

本章小結

1. 資料需求應從模型用途反推，而規格中「明確排除什麼」比「包含什麼」更重要，因為它是可被檢驗的承諾。
2. 資料量需求依技術路線差異達數個數量級；資料量越少，每一筆的品質越關鍵——這對臺灣是有利的結構。
3. 五種來源各有風險輪廓；合作授權與自有資料是臺灣的差異化來源，也是治理要求最高的。
4. 「公開」不等於「可任意再利用」，特別是含個資的公開資料；爬取前必須處理條款、速率、紀錄、排除清單與退出管道。
5. 合成資料受限於教師模型的授權、知識天花板與分布退化，不是免費的解方。
6. 著作權、契約、資料庫相關權利與營業秘密可能同時存在；合理使用是個案判斷，不可自行推定。
7. 風險分為綠黃橙紅四級，橙紅級的排除紀錄與綠級的使用紀錄同樣重要。
8. 個人資料包含直接識別、聯絡資訊、準識別、行為紀錄與特種資料；準識別資訊在小規模社群中特別危險。
9. 目的限定、最小化、保存期限三原則各有具體的工程對應；去識別是降低風險而非消除風險。
10. Provenance ledger 記錄來源、權利、內容、處理、個資、使用與生命週期七群欄位，讓每一個追問都有答案。
11. Data Card 是給人看的摘要，其中「已知限制與偏誤」必須具體；覆核者不可與執行者為同一人。

12. 退出必須分層處理，並誠實說明技術限制；承諾「完全移除」比說明限制更糟。
13. 五種資料的隔離同時是科學要求與治理要求；用雜湊分桶並以自動檢查強制執行。
14. 校園場域中，輔導紀錄不使用且無例外；未成年人資料需在專案啟動前取得機構核可。

成果檢核

完成本章後你必須繳交：

- 資料需求規格：含 excluded 欄位與敏感度分級，並經指導老師或組內責任人簽核。
- 語料清冊：每份資料一個 manifest，通過程式 8A 檢查且零錯誤。
- 風險分級結果：四級分布與橙紅級的排除理由。
- 個資處理報告：程式 8B 的三份輸出（差異報告、統計摘要、隨機抽樣清單），含覆核者與覆核時間。
- Data Card 草案：八個段落齊備，「已知限制與偏誤」須具體到可被檢驗。
- 退出流程說明：對外可公開的版本，含技術限制的誠實說明。
- 待確認事項清單：所有涉及法律判斷之處，明確列出「要問誰、問什麼」，而非自行下結論。

這份治理套件會被第 9 章（清理與配比）、第 22 章（適配前的資料檢查）、第 36 章（評測集的授權）、第 37 章（AI 影響評估）與第 40 章（總整專題驗收）直接引用。它是本書中唯一一份「就算你的模型失敗了，它仍然有價值」的交付物——因為它證明了你是以負責任的方式在做這件事。

第 9 章

大規模資料清理、去重、混合與合成資料

Cleaning, Deduplication, Mixtures, and Synthetic Data at Scale

難度：核心 / 進階 | 建議授課：第 9 週（第一次階段評量） | 硬體需求：A 級可完成縮小版，B 級可處理完

整語料

叫獸開講

一份沒去重的語料，模型會把同一段廣告詞背得比課文還熟。垃圾進、垃圾出，這句話在大模型時代要改寫成：重複進、幻覺出。這一章是第二篇的最後一章，也是最髒的一章——但你在這裡多花的每一小時，都會在第 13 章之後以十倍奉還。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 從 HTML、PDF、Office、OCR、程式碼與對話紀錄中抽取乾淨的文字，並保留必要的結構資訊。
2. 設計規則式與模型式的品質過濾器，並用保留率與樣本抽查驗證每一階段的效果。
3. 實作精確去重與近似去重，說明 MinHash 與 LSH 的原理、參數選擇與誤差特性。
4. 執行 benchmark 去污染與時間洩漏防治，並量化污染對評測結果的影響。
5. 設計語料的領域配比、取樣溫度與課程安排，並說明每個決定的依據。
6. 評估合成資料的生成、篩選與標註流程，並辨識模型崩壞的風險徵兆。
7. 在資料規模、品質與多樣性三者之間做出可解釋的取捨，並用可觀測的指標支撐。
8. 為 MiniFormosaGPT 建立完整的預訓練語料混合方案，公開每個資料源的比例與剔除理由。
9. 產出可重現的清理管線，含逐階段統計、樣本快照與失敗案例紀錄。

本章在全書中的位置

第 6 章決定文字長什麼樣、第 7 章決定怎麼切、第 8 章決定能不能用，本章決定「最後餵進去的到底是什麼」。這是第二篇的匯流點，也是第 13 章訓練 MiniFormosaGPT 的直接前置作業。做完本章，你手上會有一份可以真正拿去訓練的語料——那是整個第二篇的成果。

9.1 內容抽取：從原始檔案到純文字

9.1.1 六種來源，六種毛病

來源	主要挑戰	要保留什麼	常見的錯誤做法
HTML	導覽列、廣告、樣板、留言區	正文、標題階層、清單結構	直接 strip 標籤，樣板全留下
PDF	斷行、字距、表格、直排、頁首頁尾	段落邊界、條號、表格內容	不處理斷行，句子全被切碎
Office	追蹤修訂、註解、頁首頁尾、隱藏文字	正文與有意義的表格	把註解與正文混在一起
OCR 影像	辨識錯誤、版面順序、缺字	可信度標記	無條件信任辨識結果
程式碼	縮排即語意、註解與程式碼混合	完整原樣	壓縮空白，程式碼失效
對話紀錄	發言者、時間、暱稱、系統訊息	對話結構、角色標記	把系統訊息當成對話內容

第一列的樣板問題是網頁語料最大的殺手。一個網站的每一頁都有相同的導覽列、頁尾、版權宣告——如果你抓了一萬頁，這些內容就重複了一萬次。模型會把「本網站著作權所有，未經同意不得轉載」背得滾瓜爛熟，而那是你最不需要的知識。

9.1.2 網頁樣板的偵測與移除

最有效的方法不是寫規則去猜哪些是樣板，而是利用「樣板會重複出現在同一網站的多個頁面」這個特性。

formosa/extract/boilerplate.py (節錄)

```
from collections import Counter
from urllib.parse import urlparse

def find_boilerplate(pages: list[dict], min_ratio: float = 0.5,
                     min_pages: int = 20) -> dict[str, set[str]]:
    """依網域統計哪些段落大多數頁面中重複出現
    pages 每筆: {"url": ..., "blocks": [段落字串, ...]}"""
    by_domain = {}
    for p in pages:
        d = urlparse(p["url"]).netloc
        by_domain.setdefault(d, []).append(p)

    boiler = {}
    for domain, ps in by_domain.items():
        if len(ps) < min_pages:      # 樣本太少，無法可靠判斷
            continue
        cnt = Counter()
        for p in ps:
```

```

        for b in set(p["blocks"]):      # 同頁重複只算一次
            cnt[b] += 1
            threshold = len(ps) * min_ratio
            boiler[domain] = {b for b, c in cnt.items() if c >= threshold}
        return boiler

def strip_boilerplate(page: dict, boiler: dict) -> dict:
    d = urlparse(page["url"]).netloc
    bset = boiler.get(d, set())
    kept = [b for b in page["blocks"] if b not in bset]
    return {**page, "blocks": kept,
            "n_removed": len(page["blocks"]) - len(kept)}

```

這個方法的優雅之處在於它不需要任何網站專屬的規則——它從資料本身學出樣板。但要注意 `min_pages` 這個保護：如果某個網域只抓了三頁，那麼「出現在兩頁中的段落」很可能是正常的內容重複，而不是樣板。

9.1.3 抽取的品質標記

抽取不是全有全無的。有些文件抽得很乾淨，有些勉強可用，有些根本是垃圾。應該在抽取階段就標記品質，讓後續的過濾器有依據。

指標	怎麼算	低品質的徵兆	處置
文字密度	正文字元數 ÷ 原始檔大小	極低代表多為標記或圖片	標記待查
段落完整度	以句末標點結尾的段落比例	極低代表斷行沒處理好	重新抽取或降權
替換字元率	第 6 章的 <code>replacement</code> 比例	大於千分之一即異常	回到編碼處理
行長分布	平均行長與變異	過短代表被切碎	合併行
重複行比例	同一份文件內重複的行	極高代表是列表或樣板	視情況保留或剔除

本書的做法是：抽取階段只標記不刪除，把刪除的決定留給下一節的過濾階段。這樣做的好處是你隨時可以調整閾值重跑，而不必重新抽取——重新抽取往往是整條管線中最慢的一步。

9.2 品質過濾

9.2.1 過濾的兩種思路

類型	做法	優點	缺點
規則式	長度、語言、符號比例、關鍵詞	快、可解釋、可調	規則寫不完，容易誤殺
分類器式	訓練一個「這是好文本嗎」的分類器	能捕捉難以言喻的品質	需標註資料，且會繼承標註者偏好
困惑度式	用參考模型算 PPL，過高或過低都剔除	不需標註	會偏好與參考模型相似的文本

第三種有一個必須知道的副作用：用某個模型的困惑度來篩選資料，等於在挑選「這個模型覺得正常的文本」。如果參考模型不熟悉臺灣用語，它可能會把道地的臺灣文本判定為高困惑度而剔除——那正好是你最需要的資料。這是一個實際發生過的失誤模式，第 9.7 節會再回來討論。

9.2.2 規則式過濾器的設計

formosa/clean/filters.py (節錄)

```
import re
from dataclasses import dataclass

@dataclass
class FilterResult:
    keep: bool
    reason: str = ""

CJK = re.compile(r"[\u4e00-\u9fff]")

def filter_length(text, min_chars=50, max_chars=100_000):
    n = len(text)
    if n < min_chars:
        return FilterResult(False, f"too_short({n})")
    if n > max_chars:
        return FilterResult(False, f"too_long({n})")
    return FilterResult(True)

def filter_cjk_ratio(text, min_ratio=0.30):
    """繁中語料庫中，中文占比過低者多為程式碼或英文內容"""
    r = len(CJK.findall(text)) / max(len(text), 1)
    return FilterResult(r >= min_ratio, f"cjk_ratio({r:.3f})")

def filter_symbol_ratio(text, max_ratio=0.15):
    sym = sum(1 for c in text if not c.isalnum() and not c.isspace())
```

```

        and not CJK.match(c))
    r = sym / max(len(text), 1)
    return FilterResult(r <= max_ratio, f"symbol_ratio({r:.3f})")

def filter_line_repetition(text, max_ratio=0.30):
    lines = [l.strip() for l in text.split("\n") if l.strip()]
    if len(lines) < 5:
        return FilterResult(True)
    uniq = len(set(lines))
    r = 1 - uniq / len(lines)
    return FilterResult(r <= max_ratio, f"line_repeat({r:.3f})")

PIPELINE = [filter_length, filter_cjk_ratio,
            filter_symbol_ratio, filter_line_repetition]

def apply_filters(text: str) -> FilterResult:
    """回傳第一個不通過的理由，方便統計是哪一關卡掉的"""
    for f in PIPELINE:
        r = f(text)
        if not r.keep:
            return FilterResult(False, f"{f.__name__}:{r.reason}")
    return FilterResult(True)

```

注意 `apply_filters` 回傳的是「第一個擋下它的過濾器」。這讓你統計出每一關各擋掉多少，進而判斷哪個閾值設得太嚴或太鬆。沒有這個資訊，你只知道「剩下 60%」，卻不知道是誰吃掉了那 40%。

9.2.3 每一階段都要看樣本

這是本章最重要的一條實務紀律：每一個過濾器，都必須隨機抽取被它剔除的樣本，人工看過。

過濾階段的統計與抽樣

```

import random, json
from collections import Counter
from pathlib import Path

def filter_with_audit(docs, out_dir: Path, sample_n=20, seed=42):
    rng = random.Random(seed)
    kept, reasons = [], Counter()
    samples = {}          # reason -> 被剔除的樣本

    for d in docs:
        r = apply_filters(d["text"])
        if r.keep:
            kept.append(d)
        else:
            key = r.reason.split(":")[0]
            reasons[key] += 1
            bucket = samples.setdefault(key, [])
            # 蓄水池抽樣：不必先把全部存起來
            if len(bucket) < sample_n:
                bucket.append(d)

```

```

elif rng.random() < sample_n / reasons[key]:
    bucket[rng.randrange(sample_n)] = d

total = len(docs)
print(f"輸入 {total:}, 保留 {len(kept):}, "
      f"({len(kept)/total:.1%}) ")
for k, v in reasons.most_common():
    print(f" {k:<24} 剔除 {v:}, ({v/total:.2%}) ")

out_dir.mkdir(parents=True, exist_ok=True)
(out_dir / "rejected_samples.json").write_text(
    json.dumps({k: [d["text"][:300] for d in v]
                for k, v in samples.items()}),
              ensure_ascii=False, indent=2), encoding="utf-8")

return kept

```

為什麼一定要看被剔除的樣本

過濾器的錯誤有兩種：留下垃圾（你會在訓練後發現）與剔除好資料（你永遠不會發現）。第二種更危險，因為它是沉默的。我看過團隊把 `cjk_ratio` 的閾值設在 0.6，結果把所有含大量英文術語的臺灣技術文件全部剔除——那正是他們的目標領域。抽樣看過就能立刻發現，不看就會白做三個月。

9.2.4 臺灣語料的特殊考量

情況	一般規則會怎樣	為什麼是錯的	調整方式
中英混寫技術文件	被 <code>cjk_ratio</code> 剔除	那正是臺灣產業文本的特徵	降低閾值或分型處理
公文格式	被 <code>symbol_ratio</code> 剔除	全形標點與編號密集	排除中文標點再計算
臺羅與族語	被語言偵測剔除	偵測器不認識這些語言	建立白名單
條列式法規	被 <code>line_repetition</code> 剔除	「第○條」的結構性重複	排除條號行再計算
表格化規格書	被 <code>symbol_ratio</code> 剔除	料號與單位符號多	標記為表格另行處理

這張表傳達一個核心訊息：直接套用為英文網頁設計的過濾規則，會系統性地剔除臺灣最有價值的文本類型。每一個閾值都必須在你自己的語料上校準，而校準的方法就是 9.2.3 節的抽樣檢視。

9.3 去重：精確與近似

9.3.1 為什麼去重如此重要

重複資料對模型的傷害有三個層面。

層面	機制	後果	章節
記憶化	同一內容看越多次記得越牢	可能逐字複述訓練資料	第 8.4.5、16 章
分布扭曲	重複內容的權重被放大	模型偏好那些表述方式	本章 9.5 節
算力浪費	同樣的資料訓練多次	有效資料量遠低於帳面	第 17 章

第三點的影響常被低估。如果你的 100 GB 語料中有 40% 是重複的，那麼你實際上只有 60 GB 的資訊量，卻付了 100 GB 的訓練成本。第 17 章做算力預算時，這個差異會直接改變你能訓練多大的模型。

9.3.2 精確去重

最簡單也最必要的一層：完全相同的文件只留一份。用雜湊值比對即可，成本極低。

精確去重與段落級去重

```
import hashlib

def doc_hash(text: str) -> str:
    """正規化後再算雜湊（見第 6 章），避免格式差異造成漏判"""
    normalized = "".join(text.split()) # 移除所有空白
    return hashlib.blake2b(normalized.encode("utf-8"),
                           digest_size=16).hexdigest()

def exact_dedup(docs):
    seen, kept, dup = set(), [], 0
    for d in docs:
        h = doc_hash(d["text"])
        if h in seen:
            dup += 1
            continue
        seen.add(h)
        kept.append(**d, "doc_hash": h)
    print(f"精確去重：移除 {dup:,} 份 ({dup/len(docs):.1%}) ")
    return kept

def paragraph_dedup(docs, min_len=60, max_repeat=1):
    """跨文件的段落級去重：同一段落最多保留 max_repeat 次"""
    counter, out = {}, []
    for d in docs:
        kept paras = []
        for p in d["text"].split("\n\n"):
            p = p.strip()
            if len(p) < min_len:
                kept paras.append(p); continue # 短段落不去重
            h = doc_hash(p)
            counter[h] = counter.get(h, 0) + 1
            if counter[h] <= max_repeat:
```

```

        kept_paras.append(p)
    out.append(**d, "text": "\n\n".join(kept_paras))
    return out

```

段落級去重值得特別說明。很多重複不是整份文件相同，而是某一段落被大量轉載——例如一段法規條文出現在一百個網站上。文件級去重抓不到這種情況，段落級可以。但要注意 `min_len` 的保護：短段落（例如「注意事項」這個標題）本來就會大量重複，不應剔除。

9.3.3 近似去重：MinHash 與 LSH

真正的挑戰是「幾乎相同但不完全相同」的文件：同一篇新聞的不同轉載、加了一句話的公告、格式略有差異的規格書。這些用雜湊抓不到，需要相似度比對。

直接兩兩比對是不可行的——一百萬份文件需要五千億次比對。MinHash 加 LSH 是標準解法，它的核心想法分成兩步。

► 第一步：MinHash 估計 Jaccard 相似度

把每份文件表示成一組字串片段（shingle）的集合，兩份文件的相似度定義為 Jaccard 係數：

$$J(A, B) = |A \cap B| / |A \cup B|$$

MinHash 的性質是：用同一個雜湊函數對兩個集合取最小值，兩者相等的機率恰好等於 Jaccard 係數。所以用 128 個不同的雜湊函數各取一次最小值，得到 128 維的簽章，兩個簽章相同位置的比例就是 Jaccard 的估計值。

► 第二步：LSH 只比對可能相似的對

把 128 維簽章切成 b 個帶（band），每帶 r 列（ $b \times r = 128$ ）。只要有任何一帶完全相同，兩份文件就成為候選對。

$$P(\text{成為候選} \mid \text{相似度 } s) = 1 - (1 - s^r)^b$$

這個式子是一條 S 形曲線，轉折點大約在：

$$\text{閾值} \approx (1/b)^{1/r}$$

b	r	簽章長度	約略閾值	s=0.7 時命中率	s=0.9 時命中率
20	5	100	0.549	0.975	約 1.000
16	8	128	0.707	0.613	約 1.000

b	r	簽章長度	約略閾值	s=0.7 時命中率	s=0.9 時命中率
50	2	100	0.141	約 1.000	約 1.000

(上表數值由 $P = 1 - (1 - s^r)^b$ 直接計算得出。)

從表中可以看出參數的意義：b 大 r 小會讓閾值降低、召回率高但候選對變多（計算成本高）；b 小 r 大則相反。第三列的設定閾值只有 0.14，幾乎所有文件對都會成為候選——那等於沒有做 LSH 的加速。實務上常用 b=20、r=5 或 b=16、r=8，取決於你想抓多相似的重複。

9.3.4 完整實作

formosa/dedup/minhash.py (節錄)

```
import hashlib, random
from collections import defaultdict

MOD = (1 << 61) - 1      # 梅森質數，取模運算快

def shingles(text: str, k: int = 5) -> set[str]:
    """中文用字元級 shingle，k=5 是常用起點"""
    t = "".join(text.split())
    return {t[i:i + k] for i in range(max(0, len(t) - k + 1))}

def _hash(s: str) -> int:
    return int.from_bytes(
        hashlib.blake2b(s.encode("utf-8"), digest_size=8).digest(), "big")

def make_params(num_perm: int = 128, seed: int = 42):
    rnd = random.Random(seed)      # 種子必須固定並記錄
    return [(rnd.randrange(1, MOD), rnd.randrange(0, MOD))
            for _ in range(num_perm)]

def minhash(sh: set[str], params) -> list[int]:
    if not sh:
        return [0] * len(params)
    hs = [_hash(s) for s in sh]
    return [min((a * h + b) % MOD for h in hs) for a, b in params]

def lsh_buckets(signatures: dict, b: int = 20, r: int = 5):
    """回傳候選對。signatures: doc_id -> 簽章"""
    assert all(len(s) == b * r for s in signatures.values())
    buckets = defaultdict(list)
    for doc_id, sig in signatures.items():
        for band in range(b):
            key = (band, tuple(sig[band * r:(band + 1) * r]))
```



```

        buckets[key].append(doc_id)

    pairs = set()
    for docs in buckets.values():
        if len(docs) < 2:
            continue
        for i in range(len(docs)):
            for j in range(i + 1, len(docs)):
                pairs.add(tuple(sorted((docs[i], docs[j]))))
    return pairs

def sig_similarity(s1, s2) -> float:
    return sum(x == y for x, y in zip(s1, s2)) / len(s1)

```

一個實測的參考：兩段只差三個字的繁中句子，真實 Jaccard 為 0.875，128 維 MinHash 的估計值為 0.922；完全無關的兩段則兩者皆為 0。估計值與真值有數個百分點的誤差是正常的，這就是為什麼實務上會在 LSH 找出候選對之後，再用真實 Jaccard 做一次確認。

9.3.5 去重的取捨

問題	去重太鬆	去重太嚴	如何判斷
重複內容	記憶化風險高	—	第 16 章的記憶化偵測
有效資料量	虛高	過度減少	看去重前後的詞元數
領域平衡	高重複領域被放大	結構化文本被誤殺	分域統計去重率
品質	低品質內容被放大	—	抽樣檢視

第三列在臺灣語料上特別明顯：法規、公文、契約範本這類文本的結構性重複很高（每份公文都有類似的格式），如果去重閾值太嚴，可能把整個領域的資料削掉大半。所以去重必須分域統計——本書要求你報告每個領域的去重率，而不只是總體數字。

9.4 去污染與時間洩漏

9.4.1 去重與去污染的差別

兩者用的技術相似，目的完全不同。去重是為了訓練效率與記憶化；去污染是為了評測的可信度。

面向	去重	去污染	備註
目的	減少重複、避免記憶化	確保評測結果可信	—
比對對象	訓練集內部	訓練集 vs 評測集	—
處置	保留一份	從訓練集移除	評測集不動
嚴格程度	可容忍少量漏判	寧可誤殺不可漏判	評測可信度優先
章節	本章 9.3 節	第 5.6 節與本節	—

第四列的原則值得記住：去污染時寧可誤殺。從訓練集中多剔除一些內容，代價只是損失一點資料；但如果漏掉污染，你所有的評測結論都不可信。這是一個明顯不對稱的取捨。

9.4.2 把去污染接進管線

formosa/clean/decontaminate.py (節錄)

```
from formosa.contamination import build_index, contamination_ratio

def decontaminate(train_docs, eval_sets: dict,
                  n: int = 13, threshold: float = 0.3):
    """eval_sets: {"name": [評測文本, ...]}
    注意閾值比第 5 章的偵測用途更嚴 (0.3 而非 0.5) """
    # 對每個評測集建索引
    indexes = {name: build_index(texts, n)
               for name, texts in eval_sets.items()}

    kept, removed = [], []
    for d in train_docs:
        hits = {}
        for name, idx in indexes.items():
            r = contamination_ratio(d["text"], idx, n)
            if r >= threshold:
                hits[name] = round(r, 3)
        if hits:
            removed.append(**d, "contaminated_by": hits)
        else:
            kept.append(d)

    print(f"去污染：移除 {len(removed):,} 份"
          f" ({len(removed)/len(train_docs):.2%}) ")
    by_eval = {}
    for r in removed:
        for name in r["contaminated_by"]:
            by_eval[name] = by_eval.get(name, 0) + 1
    for name, c in sorted(by_eval.items(), key=lambda x: -x[1]):
        print(f" 因 {name} 而移除：{c:,} 份")
    return kept, removed
```

注意 removed 是被保留下來的，而不是直接丟棄。你需要它來回答兩個問題：被移除的是什麼樣的

內容？某個評測集是不是特別容易造成污染（那可能代表題目取材自太常見的來源）？

9.4.3 時間洩漏

一種特殊的污染：訓練資料中包含了評測期之後的資訊。這在有時效性的應用中會嚴重高估模型表現。

情境	洩漏方式	後果	處置
新聞問答	訓練資料含事件後續報導	模型「預測」得異常準	嚴格時間切分
法規查詢	訓練資料含修法後版本	評測舊版問題時答新版	記錄版本與生效日
校務資訊	訓練資料含次學年公告	看似知道未來	以學年度切分
價格與統計	訓練資料含後期數據	趨勢預測失真	時間切分 + 標註時間戳

防治時間洩漏的關鍵是：每一筆資料都要有可信的時間戳，而且那個時間戳應該是「內容的時間」而非「抓取的時間」。一篇 2024 年的文章在 2026 年被抓取，它的內容時間是 2024。這個區分在建立 provenance 時就必須做好（第 8.5 節）。

9.5 語料配比：決定模型會變成什麼樣子

9.5.1 配比就是優先順序的宣告

一份語料的組成不是自然形成的，而是一連串選擇的結果。而模型會忠實地反映這些選擇——你餵得多的，它就學得好；你餵得少的，它就不熟。

這意味著配比表其實是一份價值宣告：你認為這個模型應該擅長什麼。對一個臺灣本土模型而言，這份宣告特別重要，因為自然的網路資料分布會讓繁體中文與本土語言被嚴重稀釋（第 1.6.3 節）。

9.5.2 取樣溫度：控制配比的數學工具

假設某語料源的自然占比為 p 。若直接按自然比例取樣，稀有的領域會幾乎不出現。常用的調整方式是取樣溫度：

$$q_i = p_i^\alpha / \sum_j p_j^\alpha \quad \cdot \text{其中 } 0 \leq \alpha \leq 1$$

$\alpha = 1$ 表示完全按自然比例； $\alpha = 0$ 表示所有來源等量；介於中間則是部分平衡。多語模型常用 $\alpha = 0.3$ 到 0.7 。

語料源	自然占比	$\alpha=1$	$\alpha=0.5$	$\alpha=0.3$
通用中文	0.70	0.700	0.490	0.392
英文	0.20	0.200	0.262	0.269
程式碼	0.08	0.080	0.166	0.204
臺語與族語	0.02	0.020	0.083	0.135

(表中數值由上式直接計算。可看出 α 越小，稀有來源被提升得越多。)

看最後一列：臺語與族語的自然占比只有 2%，若按自然比例取樣，模型幾乎學不到。把 α 調到 0.3，它的比例提升到約 13.5%——這是刻意的干預，不是資料的自然分布。

但要注意代價：提升稀有領域的比例，等於重複使用那些資料更多次，而這會增加記憶化風險（第 9.3.1 節）。所以上取樣不是免費的，本書建議搭配三個保護：限制最大重複輪數、對上取樣的資料做更嚴格的去重、以及在第 16 章量測記憶化程度。

9.5.3 本書建議的臺灣語料配比

類別	建議比例	主要來源	理由與注意事項
通用繁體中文	35–45%	百科、新聞、書籍	語言能力的基礎
教育與教材	10–15%	課文、講義、題庫	本書的主要應用場景
公文與法規	8–12%	法規資料庫、公告	制度知識的來源，重複率高需嚴格去重
產業與技術	8–12%	規格書、技術報告	差異化來源，取得最難
社群與口語	5–10%	論壇、討論串	語感與口語表達，個資風險高
英文	12–18%	技術文件、百科	維持英文與程式能力
程式碼	5–10%	開源專案	推理與結構化能力
本土語言	2–5%	教材、辭典、文獻	需上取樣，自然占比遠低於此

(以上為教學用的起點建議，實際比例必須依你的應用目標與可取得的資料調整，並用第 5 章的分域驗證集實測驗證。)

兩個值得說明的決定。第一，英文占 12 到 18% 看起來不少，但這是必要的——完全不餵英文會讓模型的技術詞彙與程式能力大幅退化，而臺灣的技術文本本來就中英混寫。第二，程式碼的比例高於它在應用中的占比，因為有證據顯示程式碼資料對一般推理能力有幫助，第 27 章會再提到。

9.5.4 課程安排：資料的先後順序

除了「餵多少」，還有「什麼時候餵」。課程學習 (curriculum) 的想法是：先餵簡單的、乾淨的，後餵困難的、專業的。

階段	資料特徵	目的	注意
早期	高品質、通順、通用	建立基本語言能力	避免過早接觸雜訊
中期	多樣化、含專業領域	擴展知識廣度	主要的學習階段
後期	高品質、貼近目標任務	收斂到期望的行為	常稱為退火階段

最後一列的退火 (annealing) 在近年相當受重視：在訓練的最後階段，把資料換成精選的高品質內容，同時把學習率降下來。這個做法的效益在多個模型上被報告過，成本也不高——只影響最後幾個百分比的訓練步數。第 15 章講學習率排程時會與它配合。

對臺灣的實務而言，退火階段是一個很好的介入點：把你最珍貴的本土高品質語料 (人工審過的教材、正式公文、專業技術文件) 留到最後階段使用，效果通常優於平均分散在整個訓練過程中。

9.6 合成資料

9.6.1 什麼時候值得用

情境	合成資料能做些什麼	限制	章節
指令資料不足	生成問答配對	品質參差，須篩選	第 23 章
特定格式缺乏	生成結構化輸出範例	容易同質化	第 29 章
邊緣案例稀少	刻意生成罕見情境	可能不符真實分布	第 25、37 章
隱私顧慮	用合成取代真實個資	合成品可能仍洩漏模式	第 8 章
預訓練語料不足	大量生成通用文本	效果通常不佳，不建議	本節

最後一列必須說清楚：用合成資料補預訓練語料，在多數情況下不是好主意。預訓練需要的是真實世界的多樣性與長尾知識，而模型生成的內容天然缺乏這兩者。合成資料在後訓練階段（指令、偏好、特定格式）的價值遠高於預訓練階段。

9.6.2 三段式流程：生成、篩選、標註

formosa/synth/pipeline.py (節錄)

```
from dataclasses import dataclass

@dataclass
class SynthItem:
    prompt: str
    response: str
    source_model: str          # 哪個模型生成的（授權追溯用）
    generated_at: str
    checks: dict               # 各項篩選的結果
    human_reviewed: bool = False
    reviewer: str = ""

def rule_checks(item: SynthItem) -> dict:
    """規則層篩選：便宜、可解釋，先擋掉明顯的問題"""
    r = {}
    r["length_ok"] = 20 <= len(item.response) <= 3000
    r["no_refusal"] = not any(k in item.response for k in
                              ("我無法", "作為一個 AI", "抱歉，我不能"))
    r["tw_terms"] = term_drift(item.response)[0] == 0  # 第 1 章的檢核
    r["no_repeat"] = max_ngram_repeat(item.response, n=8) <= 2
    r["has_answer"] = not item.response.strip().endswith("?")
    return r

def filter_synth(items, min_pass_rate=1.0):
    kept, rejected = [], []
    for it in items:
        it.checks = rule_checks(it)
        rate = sum(it.checks.values()) / len(it.checks)
        (kept if rate >= min_pass_rate else rejected).append(it)
    print(f"規則篩選：保留 {len(kept)}/{len(items)} "
          f"({len(kept)/max(len(items),1):.1%}) ")
    return kept, rejected
```

注意 `tw_terms` 這一項：它直接沿用第 1 章的用語漂移檢查。如果教師模型不熟悉臺灣用語，它生成的內容會混入其他中文區的詞彙，而這些內容一旦進入訓練資料，就會把錯誤固化進你的模型。這是合成資料在臺灣情境下最實際的風險。

9.6.3 人工覆核的比例

規則與模型篩選都無法保證正確性——它們只能過濾形式上的問題，無法判斷內容是否符合臺灣的制度與事實。所以人工覆核是必要的。

用途	建議覆核比例	覆核重點	理由
預訓練補充	抽樣 1%	整體品質與多樣性	量大，逐筆不可行
指令微調	100%	正確性、用語、拒答邊界	每一筆都直接塑造模型行為
偏好資料	100%	偏好判斷是否合理	第 25 章的核心
評測題目	100% 且雙人	答案正確性與出處	第 36 章的要求

第二列的 100% 看起來昂貴，但請回想第 8.1.3 節的觀察：指令資料的量通常只有數百到數千筆，而每一筆都會直接影響模型行為。一千筆資料由兩人各花一天覆核，是完全可行的，而它的效益遠高於再生成一萬筆未經檢查的資料。

9.6.4 模型崩壞的風險

如果一代模型用上一代模型的輸出訓練，反覆數次，會發生什麼？研究顯示分布會逐漸退化：罕見的表述先消失，然後多樣性下降，最後模型只會產生一小部分高機率的內容。這個現象通常稱為模型崩壞。

徵兆	觀察方式	出現階段	處置
多樣性下降	生成內容的 n-gram 多樣性指標	早期	提高真實資料比例
長尾消失	罕見詞彙的出現率	中期	刻意保留罕見樣本
同質化	不同提示得到相似回答	中期	檢查合成資料的來源多樣性
事實漂移	重複的錯誤被強化	後期	人工覆核與事實查核

實務上的防護是三條：合成資料在總語料中的比例設上限（本書建議預訓練階段不超過 10%）、保留足夠的真實資料作為錨點、以及在每一代都量測多樣性指標而非只看損失值。

模型崩壞不只發生在「刻意用合成資料訓練」的情況。當網路上的內容越來越多是模型生成的，你去爬取的「真實」資料裡本來就混有大量合成內容——而你可能無從分辨。這是一個整個領域正在面對的問題，也是為什麼「有明確來源與時間戳的資料」（第 8 章）在未來會越來越有價值。

9.7 規模、品質與多樣性的三角

9.7.1 三者無法同時最大化

優先	做法	得到什麼	失去什麼
規模	放寬過濾、少去重	訓練詞元數多	品質低、記憶化風險高
品質	嚴格過濾、高門檻	每個詞元價值高	資料量銳減、可能失去多樣性
多樣性	刻意保留邊緣領域	涵蓋廣、泛化好	整體品質變異大

第二列的括號值得展開：過度追求品質會傷害多樣性，因為「品質」的判準往往偏向某一種文本風格。用困惑度篩選會偏好與參考模型相似的文本；用分類器篩選會繼承標註者的偏好；用規則篩選會偏好格式規整的文本。三種方法都會系統性地剔除某些類型的內容。

對繁體中文而言，這個風險特別具體：如果你的品質篩選器主要在正式書面文本上校準，它會傾向剔除口語、方言、網路用語與非典型格式——而那些正是模型最需要學會的臺灣語感來源。

9.7.2 可觀測的指標

面向	指標	如何量	警訊
規模	去重後的有效詞元數	tokenizer 統計	低於尺度定律建議值（第 17 章）
品質	各過濾階段的保留率與抽樣評分	程式 9A 的統計	某一關剔除超過三成
多樣性	領域分布、來源數、詞彙覆蓋	分域統計	某一來源占比超過兩成
重複度	分域去重率	程式 9B	某一域去重率超過五成
污染	與各評測集的重疊	程式 9C	任何非零值都要檢視

第三列的「某一來源占比超過兩成」是一個實用的警訊。當單一來源占比過高，模型會過度學習那個來源的風格與觀點。這在臺灣的實務中很容易發生——因為某些政府網站或大型論壇的資料特別容易大量取得。

9.7.3 一份語料統計報表該長什麼樣

outputs/corpus_report.json (結構)

```
{
  "corpus_version": "2026-08-20-a",
  "built_at": "2026-08-20T14:00:00+08:00",
  "tokenizer": "formosa-32k-v1",

  "pipeline_stages": [
    {"stage": "raw", "docs": 2_400_000, "chars": 8.2e9},
    {"stage": "extract", "docs": 2_310_000, "chars": 6.1e9,
     "note": "移除樣板與導覽"},
    {"stage": "normalize", "docs": 2_310_000, "chars": 6.0e9},
    {"stage": "quality", "docs": 1_680_000, "chars": 4.8e9,
     "reject_by": {"too_short": 320000, "cjk_ratio": 180000,
                  "symbol_ratio": 82000, "line_repeat": 48000}},
    {"stage": "exact_dedup", "docs": 1_450_000, "chars": 4.2e9},
    {"stage": "near_dedup", "docs": 1_180_000, "chars": 3.5e9},
    {"stage": "decontam", "docs": 1_176_000, "chars": 3.49e9,
     "removed_by_eval": {"tw_gov_eval": 2800, "edu_eval": 1200}}
  ],

  "final_mixture": {
    "general_zh": {"chars": 1.45e9, "ratio": 0.415, "sources": 12},
    "education": {"chars": 0.44e9, "ratio": 0.126, "sources": 8},
    "gov_legal": {"chars": 0.35e9, "ratio": 0.100, "sources": 4},
    "industry": {"chars": 0.31e9, "ratio": 0.089, "sources": 6},
    "social": {"chars": 0.24e9, "ratio": 0.069, "sources": 3},
    "english": {"chars": 0.52e9, "ratio": 0.149, "sources": 5},
    "code": {"chars": 0.14e9, "ratio": 0.040, "sources": 2},
    "local_lang": {"chars": 0.04e9, "ratio": 0.012, "sources": 7,
                  "upsample_factor": 3.0}
  },

  "dedup_rate_by_domain": {
    "gov_legal": 0.48, "education": 0.21, "general_zh": 0.19,
    "industry": 0.12, "social": 0.31, "local_lang": 0.06
  },

  "warnings": [
    "gov_legal 去重率 48%，接近警戒值，請確認未過度刪除",
    "local_lang 上取樣 3 倍，記憶化風險需於第 16 章複查"
  ]
}
```

這份報表是本章的核心交付物。它讓任何人（包括三個月後的你）能一眼看出：資料從哪裡來、每一關剔除了多少、最終的組成是什麼、以及有哪些需要注意的地方。第 40 章的專題驗收會直接檢查這份報表。

程式實作

本章三個實作構成完整的清理管線。7B 需要數 GB 的語料才能看出效果，若硬體受限可用課程提供的縮小版資料集。

程式 9A 可平行化的清理管線

目標：建立一條可分階段執行、逐階段輸出統計與樣本、可中斷續跑的清理管線。

scripts/clean_corpus.py (骨架)

```
import json, argparse
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor
from formosa.clean import extract, normalize, quality

STAGES = ["extract", "normalize", "quality"]

def process_shard(args):
    """單一分片的處理，可平行執行"""
    shard_path, stage, out_dir = args
    fn = {"extract": extract.run, "normalize": normalize.run,
         "quality": quality.run}[stage]
    stats = {"in": 0, "out": 0, "reject": {}}
    out_path = out_dir / shard_path.name

    with shard_path.open(encoding="utf-8") as fin, \
         out_path.open("w", encoding="utf-8") as fout:
        for line in fin:
            doc = json.loads(line)
            stats["in"] += 1
            result = fn(doc)
            if result is None:
                r = doc.get("_reject", "unknown")
                stats["reject"][r] = stats["reject"].get(r, 0) + 1
                continue
            fout.write(json.dumps(result, ensure_ascii=False) + "\n")
            stats["out"] += 1
    return shard_path.name, stats

def run_stage(stage: str, in_dir: Path, out_dir: Path, workers: int = 8):
    out_dir.mkdir(parents=True, exist_ok=True)
    shards = sorted(in_dir.glob("*.jsonl"))
    # 續跑：已完成的分片跳過
    todo = [s for s in shards if not (out_dir / s.name).exists()]
    print(f"[{stage}] 共 {len(shards)} 片，待處理 {len(todo)} 片")

    total = {"in": 0, "out": 0, "reject": {}}
    with ProcessPoolExecutor(workers) as ex:
        for name, st in ex.map(process_shard,
                               [(s, stage, out_dir) for s in todo]):
            total["in"] += st["in"]; total["out"] += st["out"]
            for k, v in st["reject"].items():
                total["reject"][k] = total["reject"].get(k, 0) + v
```

```

rate = total["out"] / max(total["in"], 1)
print(f"[{stage}] 保留率 {rate:.1%}")
(out_dir / "_stats.json").write_text(
    json.dumps(total, ensure_ascii=False, indent=2), encoding="utf-8")
return total

```

三個設計要點：分片處理讓你能平行化也能中斷續跑；每一階段獨立輸出讓你能只重跑某一關而不必從頭來；統計與樣本自動產出讓你不必額外寫分析程式。這三點在處理數 GB 語料時是必需的，因為完整跑一次可能要好幾小時。

驗收條件：

- 中斷後重新執行，已完成的分片不重跑，且最終結果與不中斷時完全相同。
- 每一階段輸出保留率、各剔除理由的數量、以及每種理由 20 筆隨機樣本。
- 對同一份輸入執行兩次，輸出的文件順序與內容完全一致（可重現性）。
- 能只重跑 quality 階段而不重跑 extract（證明階段確實獨立）。

程式 9B MinHash 近似去重

目標：實作完整的 MinHash + LSH 去重，並量測去重前後對困惑度與記憶化的影響。

scripts/dedup.py (主流程)

```

from formosa.dedup.minhash import (shingles, minhash, make_params,
                                   lsh_buckets, sig_similarity)

def run_dedup(docs, num_perm=128, b=20, r=5,
              jaccard_threshold=0.8, k=5, seed=42):
    assert b * r == num_perm
    params = make_params(num_perm, seed)

    # 1. 計算簽章
    sigs, shs = {}, {}
    for d in docs:
        sh = shingles(d["text"], k)
        shs[d["doc_id"]] = sh
        sigs[d["doc_id"]] = minhash(sh, params)

    # 2. LSH 找候選對
    pairs = lsh_buckets(sigs, b, r)
    print(f"候選對 {len(pairs):,} 組")

    # 3. 用真實 Jaccard 確認 (LSH 只是加速，仍有誤判)
    dup_edges = []
    for a, c in pairs:
        sa, sc = shs[a], shs[c]
        j = len(sa & sc) / max(len(sa | sc), 1)
        if j >= jaccard_threshold:
            dup_edges.append((a, c, j))
    print(f"確認重複 {len(dup_edges):,} 組")

```

```
# 4. 連通分量：每組重複只留一份（保留最長的）
parent = {d["doc_id"]: d["doc_id"] for d in docs}

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]; x = parent[x]
    return x

for a, c, _ in dup_edges:
    ra, rc = find(a), find(c)
    if ra != rc:
        parent[rc] = ra

groups = {}
by_id = {d["doc_id"]: d for d in docs}
for d in docs:
    groups.setdefault(find(d["doc_id"]), []).append(d["doc_id"])
kept = [max((by_id[i] for i in g), key=lambda d: len(d["text"]))
        for g in groups.values())
print(f"去重：{len(docs):,} -> {len(kept):,} "
      f"（移除 {1 - len(kept)/len(docs):.1%}）")
return kept, dup_edges
```

必須完成的實驗與回答的問題：

- 掃描 jaccard_threshold 從 0.6 到 0.95，畫出「移除比例對閾值」的曲線。曲線在哪裡開始陡降？那個轉折點通常是好的選擇。
- 比較 (b=20, r=5) 與 (b=16, r=8) 兩組參數的候選對數量與最終去重結果。用 9.3.3 節的公式解釋差異。
- 分域統計去重率。哪個領域最高？為什麼？這符合 9.3.5 節的預期嗎？
- 用去重前與去重後的語料各訓練一次小模型（可用第 2 章的 Tiny Transformer），比較驗證困惑度與逐字複述訓練資料的比例。

程式 9C 合成資料工廠

目標：建立一條「生成、規則篩選、人工覆核」的合成資料流程，產出臺灣情境的問答資料並記錄完整的來源資訊。

scripts/synth_factory.py (節錄)

```
import json, hashlib
from datetime import datetime, timezone
from pathlib import Path

SEED_TOPICS = [
    ("教育制度", ["升學管道", "學分規定", "獎助學金", "轉學考"]),
    ("公文寫作", ["函的格式", "簽的用法", "受文者", "副本"]),
    ("產業術語", ["工具機", "半導體封裝", "自動化控制", "品質管理"]),
```

```

    ("生活資訊", ["健保", "勞保", "報稅", "戶政"]),
]

def build_prompt(category: str, topic: str, style: str) -> str:
    return (
        f"你是臺灣的{category}專家。請以臺灣繁體中文與臺灣慣用術語，"
        f"針對「{topic}」出一個{style}的問題並提供正確回答。\\n"
        f"若你不確定臺灣的實際規定，請在回答中明確標示「需查證」，"
        f"不要編造具體的條號、金額或日期。\\n"
        f"格式：\\n 問題：...\\n 回答：..."
    )

def record(item: dict, model_name: str) -> dict:
    """每一筆都記錄來源，供第 8 章的 provenance 使用"""
    return {
        **item,
        "synth_id": hashlib.blake2b(
            (item["prompt"] + item["response"]).encode("utf-8"),
            digest_size=8).hexdigest(),
        "source_model": model_name,
        "generated_at": datetime.now(timezone.utc).isoformat(),
        "human_reviewed": False,
        "review_note": "",
    }

def review_queue(items, out: Path):
    """輸出成方便人工覆核的格式：一筆一個區塊，留下勾選欄位"""
    lines = []
    for i, it in enumerate(items, 1):
        lines.append(f"## {i}. [{it['synth_id']}]")
        lines.append(f"***問題**：{it['prompt_text']}")
        lines.append(f"***回答**：{it['response']}")
        lines.append("- [ ] 事實正確    - [ ] 臺灣用語    "
                    "- [ ] 無編造條號    - [ ] 可用")
        lines.append("覆核意見：\\n")
    out.write_text("\\n".join(lines), encoding="utf-8")

```

build_prompt 中那句「不要編造具體的條號、金額或日期」是關鍵的防護。教師模型對臺灣制度的知識往往不可靠（第 1 章開場那個案例），而它最容易編造的正是這類看起來很具體的細節。明確要求它標示「需查證」，可以讓人工覆核者知道該查哪裡。

延伸挑戰：統計覆核結果，算出教師模型在各主題上的「可用率」。你會發現某些主題（例如公文格式）的可用率遠低於其他主題——那正是模型不熟悉臺灣制度的具體證據，也是第 22 章要優先補強的領域。

系統案例：MiniFormosaGPT 預訓練語料混合方案

本章的系統案例是第二篇的總結：把第 6 到 9 章的所有工具串起來，產出一份可以真正拿去訓練的語

料，並公開每個資料源的比例、來源與被剔除的原因。

完整流程

步驟	做什麼	使用的工具	產出
1	確認所有資料的 manifest 通過檢查	程式 8A	CI 通過紀錄
2	內容抽取與樣板移除	程式 9A (extract)	純文字 + 品質標記
3	文字正規化	程式 6A	正規化文本 + transforms 紀錄
4	個資掃描與遮罩	程式 8B	遮罩後文本 + 覆核紀錄
5	品質過濾	程式 9A (quality)	保留率統計 + 剔除樣本
6	精確與近似去重	程式 9B	分域去重率
7	評測去污染	程式 5C + 9.4.2 節	移除清單與原因
8	配比與上取樣	9.5 節的取樣溫度	最終混合比例
9	tokenizer 編碼與統計	程式 7A/7B	詞元數與 fertility
10	產出語料統計報表	9.7.3 節的格式	corpus_report.json

必須交付的六項

1. 語料統計報表：依 9.7.3 節的結構，含每一階段的輸入輸出與剔除原因。
2. 配比說明：每個領域的比例、選擇理由、以及上取樣倍數與其風險評估。
3. 剔除清冊：被剔除的資料類型、數量、原因，以及每種原因的樣本。這一項與保留的資料同樣重要。
4. 去重報告：分域去重率、閾值選擇的依據、以及去重前後的有效詞元數對照。
5. 去污染報告：與各評測集的重疊情況、移除數量、以及對評測可信度的說明。
6. 可重現腳本：從原始資料到最終語料的完整流程，能一鍵重跑並得到相同結果。

評分重點

- 剔除清冊的完整性。只交出「我們用了什麼」而沒有「我們剔除了什麼、為什麼」者，視為未完成。

- 是否抽樣檢視過被剔除的樣本，並在報告中說明有無誤殺。
- 配比決定是否有依據。「憑感覺」不予計分，必須連結到應用目標或分域驗證結果。
- 可重現性。助教會在乾淨環境重跑，結果必須一致。
- 警訊處理。統計報表中若有警訊（去重率過高、單一來源占比過大），必須有回應說明。

章末評量

一、概念題

1. 為什麼網頁樣板對語料品質的傷害特別大？用「重複」與「分布扭曲」兩個角度說明。
2. 解釋 MinHash 為什麼能估計 Jaccard 相似度，以及 LSH 如何把兩兩比對的成本降下來。
3. 去重與去污染的技術相似，但為什麼去污染要「寧可誤殺不可漏判」？
4. 什麼是取樣溫度？ α 從 1 降到 0.3 對稀有語料源有什麼影響？代價是什麼？
5. 為什麼用困惑度篩選資料可能剔除掉你最需要的臺灣文本？
6. 模型崩壞的四個徵兆是什麼？為什麼說它不只發生在「刻意使用合成資料」的情況？

二、計算題

1. 用 $P = 1 - (1 - s^r)^b$ 計算： $b = 20$ 、 $r = 5$ 時，相似度 0.6、0.7、0.8 的文件成為候選對的機率各是多少？並說明這組參數適合抓多相似的重複。
2. 承上題，若改為 $b = 16$ 、 $r = 8$ （簽章長度 128），相似度 0.7 的命中率變成多少？請解釋為什麼下降。
3. 某語料四個來源的自然占比為 0.70、0.20、0.08、0.02。用 $\alpha = 0.5$ 的取樣溫度計算調整後的比例。
4. 一份 100 GB 的語料，經過品質過濾保留 70%、精確去重保留 86%、近似去重保留 81%。計算最終的資料量與整體保留率。若 fertility 為 1.4，估算最終的詞元數（假設 1 GB 約 3.3 億個繁中字元）。
5. 某評測集有 500 題，去污染時從訓練集移除了 3,200 份文件。若不做去污染，模型在受污染的 8% 題目上正確率為 95%，其餘為 68%。計算整體正確率，並與去污染後的 68% 比較。

三、除錯題

1. 同學的清理管線跑完後，保留率只有 12%。請列出四個最可能的原因與檢查順序。
2. 同學的近似去重把大量法規文件判定為重複並移除，導致該領域資料只剩兩成。請說明原因與三種修正方式。
3. 同學的 MinHash 對同一份語料跑兩次得到不同的去重結果。請指出最可能的原因。
4. 同學的合成資料在規則篩選中通過率高達 98%，但人工覆核發現超過三成內容有事實錯誤。請說明這代表什麼，以及應該如何調整流程。
5. 同學報告「去重後模型的驗證困惑度上升了」，因此主張不該去重。請指出這個推論的問題。

四、系統設計題

1. 你要為一個「臺灣中小學教育助理」建立預訓練語料。可取得的資料包括：教育部公開資料、某出版社授權的教材、教師社群的討論串、以及大量的一般網頁。請設計配比方案，說明每個來源的比例、上取樣或降權的決定、以及你會如何驗證這個配比是好的。
2. 你的團隊只有一台 32 GB 記憶體的工作站，要處理 200 GB 的原始語料。請設計整條管線的執行策略：如何分片、哪些階段可平行、去重階段的記憶體瓶頸在哪裡、以及如何在中斷後續跑。

五、臺灣情境與倫理題

1. 本章 9.2.4 節指出，直接套用為英文網頁設計的過濾規則會系統性別除臺灣最有價值的文本。請再舉出兩種本章未提到的情況，並提出調整方式。
2. 你在清理語料時發現某個大型論壇的資料品質很好、量也很大，但它的使用者結構偏向特定年齡與性別。若不加限制，它可能占最終語料的三成。請說明你的處置，以及你會在 Data Card 中如何記載。
3. 合成資料可以緩解繁中語料不足的問題，但也可能把國際模型對臺灣的誤解固化進本土模型。請討論這個兩難，並提出一套你認為可行的使用原則。

評量提示 (給自學者)

- 計算題第 1 題： $s=0.6$ 時約 0.802； $s=0.7$ 時約 0.975； $s=0.8$ 時約 0.9996。閾值約在 0.55，適合抓相似度 0.6 以上的重複。
- 計算題第 2 題：約 0.613。因為 r 從 5 增為 8， s^r 大幅變小 (0.7 的 8 次方約 0.058)，需要更高的相似度才容易命中。
- 計算題第 3 題：先算各項的平方根 (0.837、0.447、0.283、0.141)，總和 1.708，正規化後約為 0.490、0.262、0.166、0.083。
- 計算題第 4 題： $100 \times 0.70 \times 0.86 \times 0.81 \approx 48.8$ GB，整體保留率約 48.8%。字元數約 161 億，詞元數約 225 億。
- 計算題第 5 題： $(0.08 \times 0.95 + 0.92 \times 0.68) = 0.7016$ ，約 70.2%，比真實的 68% 高出約 2.2 個百分點。
- 除錯題第 5 題：去重後困惑度上升是預期的，因為重複資料讓模型「背」得更好、困惑度虛低。應該比較的是泛化能力與記憶化程度，而非訓練集或含重複的驗證集困惑度。

研究延伸與版本注意事項

- 大規模語料清理的公開實踐：多個開放語料專案都公布了完整的處理管線與過濾規則，是最好的參考來源，但套用到繁體中文前務必依 9.2.4 節重新校準。
- MinHash 與 LSH：相關的資料探勘教材對這兩個技術有完整的數學處理，若想深入理解誤差界限值得一讀。
- 去重對模型的影響：近年有多篇實證研究量化去重與記憶化、下游表現的關係，第 16 章會再引用。
- 資料配比與課程學習：關於領域配比、取樣溫度與退火階段的實證研究，第 15、17 章會延伸。
- 模型崩壞：這是一個活躍的研究方向，結論仍在演進中，引用時務必註明來源與時間。
- 本章的演算法穩定，但具體的過濾閾值必須依語料重新校準，不可直接沿用他人的數字。課程提供的預設值僅為起點。

常見問題

問：我的專題只有幾百 MB 資料，需要做這麼複雜的管線嗎？答：需要，但可以簡化。去重、去污染與抽樣檢視這三項無論規模大小都必做——它們保護的是結論的可信度，與資料量無關。平行化與分片處理則可以省略。

問：過濾閾值該設多少？答：本章給的數字都是起點，不是答案。正確的做法是：先用預設值跑一次，抽樣看被剔除的內容，然後調整。這個循環通常要跑三到五次才會穩定。願意做這件事的人，最終的語料品質會明顯優於直接套用他人參數的人。

問：去重要做到什麼程度？答：先做精確去重（必做、成本低），再做段落級去重（強烈建議），最後才考慮近似去重（成本較高）。如果資源有限，前兩項的投資報酬率最高。

問：合成資料到底能不能用？答：在後訓練階段（指令、偏好、特定格式）可以，而且往往是必要的；在預訓練階段建議上限 10%，且必須有真實資料作為錨點。最重要的是每一筆都要記錄來源模型與生成時間，讓你日後能追溯與評估。

教師使用建議

本章排在第 9 週，是第一學期的第一次階段評量週。建議 2 小時講授加 1 小時實驗，並在課堂上完成階段評量的成果檢視。

時段	內容	互動設計	注意事項
前 25 分鐘	9.1 至 9.2 節，抽取與過濾	現場展示一個網頁的樣板占比	樣板的視覺化效果很好
中間 30 分鐘	9.3 節，去重與 MinHash	用兩句相近的中文手算 Jaccard	公式要讓學生自己代一次
後 25 分鐘	9.4 至 9.5 節，去污染與配比	請各組決定自己專題的配比並說明理由	配比理由的討論最有價值
最後 40 分鐘	9.6 至 9.7 節 + 階段評量一	各組展示語料統計報表	這是第一次正式的成果檢視
實驗課 1 小時	程式 9A 與 9B	互相檢查對方的剔除樣本	互查最容易發現誤殺

第一次階段評量的建議做法：各組展示三樣東西——語料統計報表、剔除清冊、以及自訓的 tokenizer 與其 fertility 報告。評量重點不是數字漂亮，而是「說得清楚每個決定的理由」。這與第 5 章建立的紀律一致，也為第二學期的專題定下基調。

一個效果很好的課堂活動：事先準備一份「過度清理」的語料（品質閾值設得極嚴），與一份「未清理」的語料，各用它們訓練一個極小的模型，展示兩者的生成結果。前者可能通順但單調，後者可能混入廣告詞與亂碼。學生會直接看到三角取舍的具體樣貌。

高中授課的調整：9.1、9.2 完整教（樣板與過濾很直觀）；9.3 只講精確去重與「為什麼重複有害」，MinHash 的數學可略，改用「指紋比對」的比喻；9.4 用第 5 章的污染示範帶過；9.5 的配比討論很適合高中生，可以讓他們決定「一個臺灣模型該學什麼」；9.6、9.7 挑重點。程式部分做 9A 的簡化版即可。

本章小結

1. 六種來源各有典型毛病；網頁樣板可用「跨頁面重複」的特性自動偵測，不需為每個網站寫規則。
2. 抽取階段只標記品質不刪除，把刪除的決定留給過濾階段，這樣調整閾值時不必重新抽取。
3. 過濾有規則式、分類器式與困惑度式三種思路；困惑度式會偏好與參考模型相似的文本，對臺灣語料有系統性風險。
4. 每一個過濾器都必須抽樣檢視被剔除的內容。留下垃圾你會發現，剔除好資料你不會——後者才是真正的危險。
5. 直接套用為英文設計的過濾規則，會系統性剔除中英混寫、公文格式、本土語言與條列式法規，

必須重新校準。

6. 重複資料造成記憶化、分布扭曲與算力浪費；精確去重必做，段落級去重能抓到跨文件的轉載。
7. MinHash 用多個雜湊的最小值估計 Jaccard；LSH 用分帶比對把成本降下來，閾值約為 $(1/b)$ 的 r 次方根。
8. 去重可容忍少量漏判，去污染則寧可誤殺——因為評測可信度的損失是不對稱的。
9. 時間洩漏需要可信的「內容時間」而非「抓取時間」，這要在建立 provenance 時就做好。
10. 取樣溫度 α 控制配比： α 越小稀有來源被提升越多，代價是重複次數增加與記憶化風險。
11. 課程安排的退火階段是介入的好時機：把最珍貴的本土高品質語料留到最後使用。
12. 合成資料在後訓練階段價值高，預訓練階段建議上限 10%；模型崩壞的風險不只來自刻意使用合成資料。
13. 規模、品質與多樣性無法同時最大化；過度追求品質會傷害多樣性，而且傷害的往往是非典型但重要的文本。
14. 語料統計報表是本章的核心交付物，必須包含每一階段的輸入輸出、剔除原因、最終配比與警訊。

成果檢核

完成本章後你必須繳交 MiniFormosaGPT 的預訓練語料與完整文件：

- 語料統計報表：依 9.7.3 節的結構，含十個階段的完整數字。
- 剔除清冊：每種剔除原因的數量與 20 筆樣本，並說明抽樣檢視的結果。
- 去重報告：分域去重率、閾值選擇依據、去重前後的有效詞元數。
- 去污染報告：與各分域驗證集的重疊、移除數量與影響評估。
- 配比說明：每個領域的比例與理由，上取樣倍數與風險評估。
- 合成資料紀錄（若使用）：來源模型、生成時間、篩選通過率、人工覆核結果。
- 可重現腳本與環境指紋：能一鍵從原始資料重建最終語料。

這是第二篇的最終交付物，也是第一次階段評量的核心。從第 10 章開始，你會把這份語料真正餵進模型——而模型能學到什麼，此刻已經被你手上這份報表決定了大半。第二篇的四章看起來離「做模型」很遠，但沒有它們，第三篇之後的每一步都會建立在你不了解的東西上。

索引

依詞條首字排序，數字為出現的章次。英文詞條排在中文詞條之前。索引依本冊內容編製；跨篇的完整索引見全書合訂本。

英文詞條

baseline	5	MinHash	9
BERT	5、7	n-gram	5、9
Big5	6	NFC	6
BOS	7	NFD	6
BPC	5、6	NFKC	6
BPE	6、7	PAD	5、7
byte fallback	6、7	pre-tokenization	7
causal mask	5	provenance	6、8、9
checkpoint	6	RAG	5、6、7、8
Data Card	5、6、8、9	shingle	9
Dataset	8	softmax	5、7
dropout	5	T5	5、7
EOS	7	teacher forcing	5
fertility	5、7、9	token 稅	6、7
FLOPs	5、7	tokenizer	5、6、7、9
GPT	5、7、9	Transformer	5、9
Jaccard	9	Unicode	6
LoRA	7、8	Unigram	7
loss mask	5、7	UNK	6、7
LSH	9	UTF-8	6
MFU	7	WordPiece	7

中文詞條

人工覆核	5、8、9	張量	5
上下文長度	6、7	授權	5、8、9
分域驗證集	5、7、9	推理模型	5
尺度定律	5、9	教師模型	8、9
半形	6、7	族語	6、7、9
去污染	5、9	梯度	5
去重	5、6、8、9	梯度累積	5

去識別	8	梯度裁切	5
可重現	7、9	異體字	6
未成年人	8	造字區	6
正規化	5、6、7、8、9	最小化	8
目的限定	8	嵌入	5、6、7
全形	6、7、8、9	測試集	5、6、8
合成資料	8、9	著作權	8、9
合理使用	8	評測	5、8、9
因果遮罩	5	詞元	5、6、7、9
存取控制	8	詞彙表	5、6、7
污染	5、8、9	量化	5、6、7、9
位移	5	開放權重	5、7
困惑度	5、9	微調	5、6、7、8、9
批次大小	5	準識別資訊	8
罕用字	6、7	資料隔離	8
取樣溫度	9	對話模板	7
注音	6	對齊	5、6、7
注意力	5	算力	5、7、8、9
爬取	8、9	臺語	5、6、7、8、9
保存期限	8	臺羅	6、7、9
保留率	9	臺灣用語	6、9
保留集	5、7、8	語言判定	6
品質過濾	6、9	輔導紀錄	8
客語	5、6、7	模型崩壞	9
指令微調	5、8、9	樣板	9
政府開放資料	8	稽核軌跡	8
紅隊	8	課程學習	9
風險分級	8	學習率	5、9
個資	5、6、8、9	隨機種子	5
時間切分	5、8、9	檢索	5、6、7、8
消融	5	營業秘密	8
特殊詞元	5、7	環境指紋	5、6、9
特種資料	8	繁簡	5、6
訓練集	5、8、9	繁體中文	5、6、7、8、9
記憶化	5、8、9	雜湊分桶	8

退火 9

曝光偏差 5

退出機制 8

驗證集 5、7、8、9

參數量 5、7

大模型導論 Introduction to Large Language Models

第 2 篇 語言建模、臺灣語料與 Token 工程 (第 5–9 章)

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

課程與勘誤 : @prof (<https://page.line.me/prof>)

臺灣自編大學教科書系列 | 2026 年 8 月 第一版